

# CEDIMS\_SourceCode

Coding languages used: Svelte, TypeScript, JavaScript, SQL, CSS, and Python. Frameworks and services include SvelteKit, Supabase, Backblaze B2 compatible S3 storage, Tesseract.js, PDF.js, and Vite.

This source-code document includes only the main project features: authentication, role-based dashboards, document upload and OCR, duplicate protection, offline sync, analytics, verification, storage APIs, database security, and chatbot training utilities.

## Project Configuration

### package.json Lines 1-80

```
0001 {
0002   "name": "cedims",
0003   "private": true,
0004   "version": "0.0.1",
0005   "type": "module",
0006   "scripts": {
0007     "dev": "vite dev",
0008     "build": "vite build",
0009     "preview": "vite preview",
0010     "prepare": "svelte-kit sync || echo \"\"",
0011     "check": "svelte-kit sync && svelte-check --tsconfig ./tsconfig.json",
0012     "check:watch": "svelte-kit sync && svelte-check --tsconfig ./tsconfig.json --watch"
0013   },
0014   "devDependencies": {
0015     "@sveltejs/adaptor-vercel": "^6.3.2",
0016     "@sveltejs/kit": "^2.53.0",
0017     "@sveltejs/vite-plugin-svelte": "^6.2.4",
0018     "@tailwindcss/vite": "^4.1.18",
0019     "@types/qrcode": "^1.5.6",
0020     "svelte": "^5.49.2",
0021     "svelte-check": "^4.3.6",
0022     "tailwindcss": "^4.1.18",
0023     "typescript": "^5.9.3",
0024     "vite": "^7.3.1"
0025   },
0026   "dependencies": {
0027     "@aws-sdk/client-s3": "^3.1013.0",
0028     "@aws-sdk/s3-request-presigner": "^3.1013.0",
0029     "@supabase/supabase-js": "^2.96.0",
0030     "browser-image-compression": "^2.0.2",
0031     "chart.js": "^4.5.1",
0032     "docx": "^9.7.1",
0033     "exceljs": "^4.4.0",
0034     "idb-keyval": "^6.2.2",
0035     "jsqr": "^1.4.0",
0036     "lucide-svelte": "^0.575.0",
0037     "mammoth": "^1.12.0",
0038     "pdf-lib": "^1.17.1",
0039     "pdf-parse": "^2.4.5",
0040     "qrcode": "^1.5.4",
0041     "tesseract.js": "^7.0.0"
0042   }
0043 }
```

## Application Shell

### src/routes/+layout.svelte Lines 1-170

```
0001 <script lang="ts">
0002   import "../app.css";
0003   import Toast from "$lib/components/Toast.svelte";
0004   import ChatBot from "$lib/components/ChatBot.svelte";
0005   import LoadingScreen from
"$components/LoadingScreen.svelte";
0006   import { showQRScanner } from "$lib/stores/ui";
0007   import { hideLoading } from "$stores/loadingStore";
0008   import { goto } from "$app/navigation";
```

```
0009   import { initAuth, profile } from "$lib/utls/auth";
0010   import {
0011     initOfflineSync,
0012     prefetchOfflineMetadata,
0013   } from "$lib/utls/offline";
0014   import { onMount } from "svelte";
0015   import { get } from "svelte/store";
0016
0017   let { children } = $props();
0018
0019   // Loaded on demand rather than imported at the top of
this file: this is
0020   // the root layout, so a static import here pulled
QRScanner (and its CSS)
0021   // into every single page load — login, upload, all of
them — even though
0022   // the component only ever mounts once the user
opens the scanner. That's
0023   // exactly the "preloaded but not used" warning
browsers logged on pages
0024   // that never open it.
0025   let QRScannerComponent = $state<typeof
import("$lib/components/QRScanner.svelte").default | null>(null);
0026   $effect(() => {
0027     if ($showQRScanner
&& !QRScannerComponent) {
0028       import("$lib/components/QRScanner.svelte").then((m)
=> {
0029         QRScannerComponent =
m.default;
0030       });
0031     }
0032   });
0033
0034   function handleScan(data: string) {
0035     if (data.includes("/verify/")) {
0036       const hash =
data.split("/verify/").pop();
0037       if (hash) {
0038         goto(`/verify/${hash}`);
0039         showQRScanner.set(false);
0040       }
0041     } else {
0042       // Fallback for non-verify QR codes
0043       console.log("[QR] Scanned data:",
data);
0044     }
0045   }
0046
0047   onMount(() => {
0048     // Dynamic `import()` failures (stale chunk
hashes after a redeploy,
0049     // or a chunk request that got redirected/404'd)
surface as a
0050     // *rejected promise*, not a thrown error — so
the existing 'error'
0051     // listener below never sees them. Without this,
a build mismatch
0052     // left the router permanently stuck instead of
self-healing via
```

```

0053 // reload, which is exactly the "Failed to fetch
dynamically
0054 // imported module" loop seen in the console.
0055 let reloadedForStaleBuild = false;
0056 const reloadOnStaleBuild = (message: string |
undefined) => {
0057     if (reloadedForStaleBuild) return;
0058     if (
0059         message?.includes("Failed
to fetch dynamically imported module") ||
0060         message?.includes("error
loading dynamically imported module")
0061     ) {
0062         reloadedForStaleBuild =
true;
0063         console.warn("[v0] Build
mismatch detected. Reloading...");
0064         window.location.reload();
0065     }
0066 };
0067
0068 const handleModuleError = (e: ErrorEvent) =>
reloadOnStaleBuild(e.message);
0069 const handleModuleRejection = (e:
PromiseRejectionEvent) =>
0070     reloadOnStaleBuild(e.reason?.message ??
String(e.reason ?? ""));
0071
0072     window.addEventListener("error",
handleModuleError);
0073
    window.addEventListener("unhandledrejection",
handleModuleRejection);
0074
0075     (async () => {
0076         // Initialize auth first
0077         try {
0078             await initAuth();
0079         } catch (err) {
0080             console.error("[v0] Failed to
initialize auth:", err);
0081         }
0082
0083         // Then initialize offline sync
0084         initOfflineSync();
0085
0086         // Pre-fetch metadata if online (Phase
20.4)
0087         const user = get(profile);
0088         if (user && user.id) {
0089             prefetchOfflineMetadata(user.id, user.district_id ||
undefined);
0090         }
0091
0092         // Hide loading screen once app is
initialized
0093         hideLoading();
0094
0095         // Defer service worker registration
0096         setTimeout(() => {
0097             if ("serviceWorker" in
navigator && import.meta.env.PROD) {
0098                 (async () => {
0099                     try {
0100
Preview/staging hosts sometimes front every

```

```

0101 request with an auth or canonicalization //
0102 //
0103 redirect. Registering a SW whose script //
0104 request is redirected throws a SecurityError //
0105 (browsers disallow it outright), so probe //
0106 first with a manual-redirect HEAD request and //
0107 skip registration entirely when that's the //
0108 case instead of letting the browser log a //
0109 hard failure every load.
0110     const probe = await fetch("/service-worker.js", {
0111         method: "HEAD",
0112         redirect: "manual",
0113         cache: "no-store",
0114     });
0115     (
0116         probe.type === "opaqueredirect" ||
0117         (probe.status >= 300 && probe.status < 400)
0118     )
0119     {
0120         console.warn(
0121             "[v0] Skipping Service Worker registration:
0122 script is served behind a redirect on this host.",
0123         );
0124     };
0125     return;
0126 }
0127 } catch
0128 (probeError) {
0129     //
0130     Network hiccup on the probe itself isn't a
0131     //
0132     reason to skip registration — fall through
0133     //
0134     and let register() attempt it normally.
0135 }
0136 try {
0137     const registration =
0138     await navigator.serviceWorker.register(
0139         "/service-worker.js",
0140     );
0141     console.log(
0142         "Service Worker registered:",
0143     
```

```

    registration,
0137                                     );
0138                                     } catch
(error) {
0139
    console.error(
0140
    "Service Worker registration failed:",
0141
    error,
0142                                     );
0143                                     }
0144                                     }}());
0145                                     }
0146                                     }, 2000);
0147                                     }}());
0148
0149     return () => {
0150         window.removeEventListener("error",
handleModuleError);
0151         window.removeEventListener(
0152             "unhandledrejection",
0153             handleModuleRejection,
0154         );
0155     };
0156 });
0157 </script>
0158
0159 <svelte:head>
0160     <title>CEDIMS · Powered by Smart E-VISION</title>
0161 </svelte:head>
0162
0163 <LoadingScreen appName="CEDIMS" />
0164
0165 <Toast />
0166
0167 {#if $showQRScanner && QRScannerComponent}
0168     <QRScannerComponent onScan={handleScan}
onClose={() => showQRScanner.set(false)} />
0169 {/if}
0170

```

## Authentication and Role Routing

**src/lib/utils/auth.ts** Lines 1-220

```

0001 import { writable } from 'svelte/store';
0002 import { supabase } from './supabase';
0003 import type { User } from '@supabase/supabase-js';
0004
0005 export interface Profile {
0006     id: string;
0007     full_name: string;
0008     role: 'Teacher' | 'School Head' | 'Master Teacher' |
'District Supervisor' | 'Admin';
0009     school_id: string | null;
0010     district_id: string | null;
0011     avatar_url: string | null;
0012 }
0013
0014 export const user = writable<User | null>(null);
0015 export const profile = writable<Profile | null>(null);
0016 export const authLoading = writable<boolean>(true);
0017 export const isChangingPassword =
writable<boolean>(false);
0018
0019 let authInitialized = false;
0020 let authInitPromise: Promise<void> | null = null;
0021
0022 // Tracks which account last signed in on this browser, so

```

```

the instant-cache
0023 // bootstrap in performAuthInit() never guesses from
localStorage key order.
0024 const LAST_USER_KEY = 'auth_last_user_id';
0025
0026 export async function initAuth(): Promise<void> {
0027     // Prevent multiple simultaneous initialization attempts
0028     if (authInitialized) {
0029         return;
0030     }
0031
0032     if (authInitPromise) {
0033         return authInitPromise;
0034     }
0035
0036     authInitPromise = performAuthInit();
0037     return authInitPromise;
0038 }
0039
0040 async function performAuthInit(): Promise<void> {
0041     if (authInitialized) return;
0042
0043     authLoading.set(true);
0044
0045     // 1. INSTANT: Load cached profile + user from
localStorage BEFORE any network call.
0046     // This makes the dashboard render immediately even
when fully offline.
0047     //
0048     // IMPORTANT: localStorage persists across every
account that has ever
0049     // signed in on this browser/device (e.g. a shared school
computer), and
0050     // old auth_profile_<id> entries are never cleared
automatically. We must
0051     // only ever load the entry for the specific user who last
signed in here
0052     // (tracked separately in LAST_USER_KEY) — never
"whichever key happens to
0053     // be first" (Object.keys() order has nothing to do with
the current
0054     // session), or one teacher's device can briefly render
another teacher's
0055     // cached profile/data on the next load, before the real
session check
0056     // corrects it.
0057     let hasCachedProfile = false;
0058     const lastUserId =
localStorage.getItem(LAST_USER_KEY);
0059     if (lastUserId) {
0060         try {
0061             const cached =
localStorage.getItem(`auth_profile_${lastUserId}`);
0062             if (cached) {
0063                 const parsed = JSON.parse(cached);
0064                 profile.set(parsed);
0065                 // Set a minimal user object so dashboard layout
guard ($user) passes
0066                 user.set({ id: lastUserId } as any);
0067                 hasCachedProfile = true;
0068                 console.log('[v0] Auth: Instant profile + user set
from cache');
0069             }
0070         } catch (e) {
0071             console.error('[v0] Error loading cached profile:', e);
0072         }
0073     }
0074

```

```

0075 // If we have cached data, unlock the UI immediately —
network will update in background
0076 if (hasCachedProfile) {
0077     authLoading.set(false);
0078 }
0079
0080 try {
0081     const timeout = navigator.onLine ? 8000 : 3000;
0082     console.log('[v0] Auth: starting session check...');
0083     const sessionPromise = supabase.auth.getSession();
0084     const timeoutPromise = new Promise( (_, reject) =>
0085         setTimeout(() => reject(new Error('Session check
timeout')), timeout)
0086     );
0087
0088     const { data: { session }, error } = await
Promise.race([
0089         sessionPromise,
0090         timeoutPromise
0091     ]) as any;
0092
0093     if (error) {
0094         console.error('[v0] Error getting session:',
error.message);
0095     }
0096
0097     if (session?.user) {
0098         user.set(session.user);
0099         await fetchProfile(session.user.id);
0100     }
0101
0102     supabase.auth.onAuthStateChange(async (_event:
string, session: any) => {
0103         if (session?.user) {
0104             user.set(session.user);
0105             await fetchProfile(session.user.id);
0106         } else {
0107             user.set(null);
0108             profile.set(null);
0109         }
0110     });
0111
0112     authInitialized = true;
0113 } catch (err) {
0114     console.error('[v0] Auth initialization error:', err
instanceof Error ? err.message : String(err));
0115     authInitialized = true;
0116 } finally {
0117     authLoading.set(false);
0118 }
0119 }
0120
0121 async function fetchProfile(userId: string): Promise<void>
{
0122     const CACHE_KEY = `auth_profile_${userId}`;
0123
0124     // 1. Try to load from cache first if offline or for
immediate UI feedback
0125     const cached = localStorage.getItem(CACHE_KEY);
0126     if (cached) {
0127         try {
0128             const parsed = JSON.parse(cached);
0129             profile.set(parsed);
0130             console.log('[v0] Auth: loaded profile from cache');
0131         } catch (e) {
0132             console.error('[v0] Error parsing cached profile:', e);
0133         }
0134     }
0135
0136     if (!navigator.onLine && cached) return;
0137
0138     try {
0139         const { data, error } = await supabase
.from('profiles')
0140             .select('id, full_name, role, school_id, district_id,
avatar_url')
0141             .eq('id', userId)
0142             .single();
0143
0144         if (error) {
0145             console.error('Error fetching profile:',
error.message);
0146             // If offline, we already set it from cache. If it's a
real error (not just fetch), maybe don't clea
0147             if (navigator.onLine) profile.set(null);
0148         } else if (data) {
0149             profile.set(data as Profile);
0150             localStorage.setItem(CACHE_KEY,
JSON.stringify(data));
0151             localStorage.setItem(LAST_USER_KEY, userId);
0152         }
0153     } catch (err) {
0154         console.warn('[v0] Auth: fetchProfile failed (ignoring if
offline):', err);
0155     }
0156 }
0157
0158 export async function signIn(email: string, password:
string): Promise<{ error: string | null }> {
0159     try {
0160         const { error } = await
supabase.auth.signInWithPassword({ email, password });
0161         if (error) {
0162             console.error('[v0] Sign in error:', error.message);
0163             return { error: error.message };
0164         }
0165         return { error: null };
0166     } catch (err) {
0167         const errorMsg = err instanceof Error ? err.message :
'An unexpected error occurred during sign in';
0168         console.error('[v0] Sign in exception:', errorMsg);
0169         return { error: errorMsg };
0170     }
0171 }
0172
0173 export async function signOut(): Promise<void> {
0174     await supabase.auth.signOut();
0175     user.set(null);
0176     profile.set(null);
0177
0178     // Clear every cached profile on this device, not just the
current one —
0179     // this is what a shared/school computer needs so the
next person to open
0180     // the app never has a chance to see a previous
teacher's cached identity.
0181     localStorage.removeItem(LAST_USER_KEY);
0182     for (const key of Object.keys(localStorage)) {
0183         if (key.startsWith('auth_profile_')) {
0184             localStorage.removeItem(key);
0185         }
0186     }
0187
0188 export async function changePassword(currentPassword:
string, newPassword: string): Promise<{ error: string | nu

```

```

0191   isChangingPassword.set(true);
0192   try {
0193     const { error: signInError } = await
supabase.auth.signInWithPassword({
0194       email: (await
supabase.auth.getUser()).data.user?.email || "",
0195       password: currentPassword
0196     });
0197     if (signInError) return { error: 'Current password is
incorrect.' };
0198
0199     const { error: updateError } = await
supabase.auth.updateUser({ password: newPassword });
0200     if (updateError) return { error: updateError.message };
0201     return { error: null };
0202   } catch (err) {
0203     return { error: err instanceof Error ? err.message :
'Failed to change password.' };
0204   } finally {
0205     isChangingPassword.set(false);
0206   }
0207 }
0208
0209 export async function resetPasswordForEmail(email:
string): Promise<{ error: string | null }> {
0210   try {
0211     const redirectTo = typeof window !== 'undefined'
0212       ? `${window.location.origin}/auth/update-
password`
0213       : undefined;
0214     const { error } = await
supabase.auth.resetPasswordForEmail(email, {
0215       redirectTo
0216     });
0217     if (error) return { error: error.message };
0218     return { error: null };
0219   } catch (err) {
0220     return { error: err instanceof Error ? err.message :
'Failed to send reset email.' };

```

## Document Permission Rules

### src/lib/utils/documentPermissions.ts Lines 1-230

```

0001 /**
0002  * CEDIMS Document Permissions System
0003  *
0004  * UPLOAD PERMISSIONS:
0005  * - Teacher: DLL only (with teaching load)
0006  * - Master Teacher: DLL (with teaching load) + ISP/ISR
0007  * - School Head: ISP/ISR only
0008  * - District Supervisor: NONE (view/remarks only)
0009  *
0010  * ISP/ISR VISIBILITY:
0011  * - Master Teacher ISP/ISR:
0012  *   * Visible to: Self, District Supervisor (same district),
School Head (same school)
0013  *   * Remarks by: School Head, District Supervisor
0014  *
0015  * - School Head ISP/ISR:
0016  *   * Visible to: Self, District Supervisor (same district)
0017  *   * Remarks by: District Supervisor only
0018  *
0019  * - DLL: Visible to all (existing rules)
0020  */
0021
0022 export function getAllowedUploadDocTypes(role: string):
string[] {
0023   switch (role) {
0024     case 'Teacher':

```

```

0025     return ['DLL'];
0026     case 'Master Teacher':
0027       return ['DLL', 'ISP', 'ISR'];
0028     case 'School Head':
0029       return ['ISP', 'ISR'];
0030     case 'District Supervisor':
0031       return []; // District Supervisors do NOT upload
0032     default:
0033       return [];
0034   }
0035 }
0036
0037 export function canUploadDocument(role: string, docType:
string): boolean {
0038   return
getAllowedUploadDocTypes(role).includes(docType);
0039 }
0040
0041 export function requiresTeachingLoadSelection(role:
string, docType: string): boolean {
0042   return (role === 'Teacher' || role === 'Master Teacher')
&& docType === 'DLL';
0043 }
0044
0045 export function canViewArchivedDocument(role: string,
docType: string): boolean {
0046   // DLL documents: visible to all roles who can access
archive
0047   if (docType === 'DLL') {
0048     return role === 'Teacher' || role === 'Master Teacher' ||
role === 'School Head' || role === 'District Super
0049   }
0050
0051   // ISP/ISR: special rules via canViewUploadedISPISR
0052   return false;
0053 }
0054
0055 export function canAddRemarkToISPISR(
0056   role: string,
0057   docUploadedByRole: string,
0058   documentType: string
0059 ): boolean {
0060   // Master Teacher ISP/ISR: School Head and District
Supervisor can add remarks
0061   if (docUploadedByRole === 'Master Teacher' &&
(documentType === 'ISP' || documentType === 'ISR')) {
0062     return role === 'School Head' || role === 'District
Supervisor';
0063   }
0064
0065   // School Head ISP/ISR: Only District Supervisor can add
remarks
0066   if (docUploadedByRole === 'School Head' &&
(documentType === 'ISP' || documentType === 'ISR')) {
0067     return role === 'District Supervisor';
0068   }
0069
0070   return false;
0071 }
0072
0073 export function canAddReviewRemarks(role: string):
boolean {
0074   // Master Teachers review DLLs alongside their School
Head — that is the
0075   // instructional-supervision part of the role — so they can
leave remarks
0076   // too. Scope is enforced by what each role can see
rather than here: the

```

```

0077 // archive query restricts both Master Teacher and
School Head to their own
0078 // school (profiles.school_id), so this only ever grants
remarks on
0079 // documents from their own school.
0080 return role === 'Master Teacher' || role === 'School
Head' || role === 'District Supervisor';
0081 }
0082
0083 export function getUploadGuidance(role: string): string {
0084   switch (role) {
0085     case 'Teacher':
0086       return 'Teachers may upload Daily Lesson Plan
documents only.';
0087     case 'Master Teacher':
0088       return 'Master Teachers may upload Daily Lesson
Plan, ISP — Instructional Supervisory Plan, and ISR — Inst
0089     case 'School Head':
0090       return 'School Heads may upload ISP — Instructional
Supervisory Plan and ISR — Instructional Supervisory R
0091     case 'District Supervisor':
0092       return 'District Supervisors can view and manage ISP
— Instructional Supervisory Plan / ISR — Instructiona
0093     default:
0094       return 'Please sign in to continue.';
0095   }
0096 }
0097
0098 /**
0099  * Check if a user can view an uploaded ISP/ISR
document based on roles and location (school/district).
0100  *
0101  * VISIBILITY RULES:
0102  * - Master Teacher ISP/ISR:
0103  *   * Self (uploader)
0104  *   * District Supervisor (same district)
0105  *   * School Head (same school)
0106  *
0107  * - School Head ISP/ISR:
0108  *   * Self (uploader)
0109  *   * District Supervisor (same district)
0110  *
0111  * - District Supervisor can always view all ISP/ISR in their
district
0112  */
0113 export function canViewUploadedISPISR(
0114   currentUserRole: string,
0115   currentUserId: string,
0116   documentUploaderId: string,
0117   uploaderRole: string,
0118   documentType: string,
0119   currentUserSchoolId: string | null,
0120   uploaderSchoolId: string | null,
0121   currentUserDistrictId: string | null,
0122   uploaderDistrictId: string | null
0123 ): boolean {
0124   // Not ISP/ISR, use regular rules
0125   if (documentType !== 'ISP' && documentType !== 'ISR') {
0126     return canViewArchivedDocument(currentUserRole,
documentType);
0127   }
0128
0129   // Current user is the uploader (can always view own
documents)
0130   if (currentUserId === documentUploaderId) {
0131     return true;
0132   }
0133
0134   // Master Teacher ISP/ISR visibility
0135   if (uploaderRole === 'Master Teacher') {
0136     // District Supervisor from same district can view
0137     if (currentUserRole === 'District Supervisor' &&
currentUserDistrictId === uploaderDistrictId) {
0138       return true;
0139     }
0140     // School Head from same school can view
0141     if (currentUserRole === 'School Head' &&
currentUserSchoolId === uploaderSchoolId) {
0142       return true;
0143     }
0144   }
0145
0146   // School Head ISP/ISR visibility
0147   if (uploaderRole === 'School Head') {
0148     // District Supervisor from same district can view
0149     if (currentUserRole === 'District Supervisor' &&
currentUserDistrictId === uploaderDistrictId) {
0150       return true;
0151     }
0152   }
0153
0154   return false;
0155 }
0156
0157 /**
0158  * Get documents visible to a specific user based on their
role and location.
0159  * Used for filtering queries in the backend.
0160  */
0161 export function getVisibleDocumentFilter(
0162   role: string,
0163   userId: string,
0164   schoolId: string | null,
0165   districtId: string | null
0166 ): { filterType: string; params: any } {
0167   if (role === 'Teacher') {
0168     return {
0169       filterType: 'teacher_dll',
0170       params: { userId }
0171     };
0172   }
0173
0174   if (role === 'Master Teacher') {
0175     return {
0176       filterType: 'master_teacher',
0177       params: { userId, schoolId, districtId }
0178     };
0179   }
0180
0181   if (role === 'School Head') {
0182     return {
0183       filterType: 'school_head',
0184       params: { userId, schoolId, districtId }
0185     };
0186   }
0187
0188   if (role === 'District Supervisor') {
0189     return {
0190       filterType: 'district_supervisor',
0191       params: { districtId }
0192     };
0193   }
0194
0195   return {
0196     filterType: 'none',
0197     params: {}

```

```
0198 };
0199 }
```

## Upload Page State and Guards

src/routes/dashboard/upload/+page.svelte Lines 1-260

```
0001 <script lang="ts">
0002   import FileDropZone from
"$lib/components/FileDropZone.svelte";
0003   import UploadProgress from
"$lib/components/UploadProgress.svelte";
0004   import {
0005     runPipeline,
0006     type PipelinePhase,
0007     type PipelineResult,
0008   } from "$lib/utlils/pipeline";
0009   import {
0010     getQueueSize,
0011     getQueueItems,
0012     processQueue,
0013     cacheMetadata,
0014     getCacheMetadata,
0015     pendingSyncCount,
0016   } from "$lib/utlils/offline";
0017   import {
0018     syncLedgerFromServer,
0019     type LedgerEntry,
0020   } from "$lib/utlils/offlineSubmissionLedger";
0021   import { profile, type Profile } from "$lib/utlils/auth";
0022   import { isMobileDevice } from "$lib/utlils/device";
0023   import { settings } from "$lib/stores/settings";
0024   import { connectivity } from "$lib/stores/connectivity";
0025   import { addToast } from "$lib/stores/toast";
0026   import { supabase } from "$lib/utlils/supabase";
0027   import { goto } from "$app/navigation";
0028   import { onMount, untrack } from "svelte";
0029   import { fade } from "svelte/transition";
0030   import { extractMetadata, type DocMetadata } from
"$lib/utlils/ocr";
0031   import { transcodeToPdf } from "$lib/utlils/transcode";
0032   import { predictLoad, validateSelection } from
"$lib/utlils/copilot";
0033   import {
0034     canUploadDocument,
0035     getAllowedUploadDocTypes,
0036     getUploadGuidance,
0037     requiresTeachingLoadSelection,
0038   } from "$lib/utlils/documentPermissions";
0039   import { focusTrap } from "$lib/actions/focusTrap";
0040   import UploadSuccessModal from
"$lib/components/UploadSuccessModal.svelte";
0041   import { formatDocumentList, getDocumentLabel } from
"$lib/utlils/documentLabels";
0042
0043   import type { PageData } from "./$types";
0044
0045   let { data } = $props<{ data: PageData }>();
0046
0047   interface TeachingLoad {
0048     id: string;
0049     subject: string;
0050     grade_level: string;
0051   }
0052
0053   let selectedFile = $state<File | null>(null);
0054   let docType = $state("DLL");
0055   let allowedDocTypes = $state<string[]>([]);
0056   let subject = $state("");
0057   let weekNumber = $state<number | undefined>();
```

```
0058   let teachingLoadId = $state<string>("");
0059   let teachingLoads = $state<TeachingLoad[]>([]);
0060   let loadingTeachingLoads = $state(true);
0061   let currentPhase =
$state<PipelinePhase>("transcoding");
0062   let progress = $state(0);
0063   let message = $state("");
0064   let processing = $state(false);
0065   let result = $state<PipelineResult | null>(null);
0066   let showSuccessModal = $state(false);
0067   let mismatchAlert = $state<any>(null);
0068   let queueCount = $state(0);
0069   let currentDeadline = $state<any>(null);
0070   let submissionAlreadyExists = $state(false);
0071   let submissionBlockReason = $state("");
0072   let ocrConfidence = $state<number | null>(null);
0073   let fileSizeWarning = $state(false);
0074   let detectingMetadata = $state(false);
0075   let scanProgress = $state(0);
0076   let detectedMetadata = $state<any>(null);
0077   let fileHash = $state<string>("");
0078   // Caches the PDF bytes produced by the pre-upload
OCR conversion step
0079   // below, keyed by File reference, so the actual upload
doesn't send the
0080   // same docx/doc through Google Apps Script a second
time. Consecutive
0081   // conversions of the same file were tripping Apps
Script's per-user
0082   // concurrent-execution quota and causing intermittent
CORS/redirect
0083   // failures on the second call.
0084   let preConvertedPdf: { file: File; bytes: Uint8Array } |
null = null;
0085   let isOnline = $state(
0086     typeof navigator !== "undefined" ? navigator.onLine :
true,
0087   );
0088   const isMobile = isMobileDevice();
0089   let pendingItems = $state<any[]>([]);
0090   let showPendingPanel = $state(false);
0091
0092   let showLoadPicker = $state(false);
0093   let showWeekPicker = $state(false);
0094   let academicWeeks = $state<number[]>([]);
0095
0096   // Header subtitle reflects only the document types this
role can actually submit.
0097   const uploadSubtitle = $derived(
0098     allowedDocTypes.length > 0
0099     ? `Submit your
${formatDocumentList(allowedDocTypes)} for archival`
0100     : "View and manage submitted documents",
0101   );
0102
0103   // Watch for changes that could invalidate uniqueness
0104   $effect(() => {
0105     if (teachingLoadId && weekNumber && docType &&
"profile") {
0106       checkExistingSubmission();
0107     } else if (fileHash) {
0108       checkExistingSubmission();
0109     } else {
0110       submissionAlreadyExists = false;
0111       submissionBlockReason = "";
0112     }
0113   });
0114
```

```

0115   async function checkExistingSubmission() {
0116       // Check 1: Hash uniqueness (highest priority -
blocks content regardless of slot)
0117       if (fileHash) {
0118           const { hasHash } = await import(
0119               "$lib/utils/offlineSubmissionLedger"
0120           );
0121           const localMatch = await hasHash(fileHash);
0122           if (localMatch) {
0123               submissionAlreadyExists = true;
0124               submissionBlockReason = `Duplicate content:
This file was already archived as "${localMatch.file
0125               return;
0126           }
0127
0128           if (navigator.onLine) {
0129               // Cross-teacher duplicate check — needs to
see whether ANY
0130               // user already archived this hash, not just the
current
0131               // uploader's own rows, so it goes through a
narrow RPC rather
0132               // than a direct table select (see
migrations/20260910_*.sql).
0133               //
0134               // Plain array request rather than .maybeSingle():
PostgREST
0135               // only 406s when a singular representation is
requested and
0136               // 0 rows come back — routine here, not a real
failure.
0137               const { data: hashMatches } = (await
supabase.rpc(
0138                   "check_duplicate_submission_hash",
0139                   { p_hash: fileHash },
0140                   )) as {
0141                   data: { id: string; file_name: string; doc_type:
string; week_number: number | null }[] | nu
0142               };
0143               const serverHashMatch = hashMatches?.[0];
0144
0145               if (serverHashMatch) {
0146                   submissionAlreadyExists = true;
0147                   // Check whether the archived file has already
been approved
0148                   const { data: review } = await supabase
0149                       .from("dll_reviews")
0150                       .select("status")
0151                       .eq("submission_id", serverHashMatch.id)
0152                       .maybeSingle();
0153
0154                   if (review?.status === "approved") {
0155                       submissionBlockReason = `Already
approved (Checked): This file was approved as "${server
0156                       } else if (review?.status === "returned") {
0157                           submissionBlockReason = `Duplicate
content: This file was already archived and returned
0158                           } else {
0159                               submissionBlockReason = `Duplicate
content: Already archived on server as "${serverHashM
0160                           }
0161                           return;
0162                       }
0163                   }
0164               }
0165
0166               // No slot uniqueness check: multiple uploads per
week are allowed.
0167               // Only hash deduplication (above) is enforced.
0168               submissionAlreadyExists = false;
0169           }
0170
0171           // Watch weekNumber and fetch deadline
0172           $effect(() => {
0173               if (weekNumber && $profile?.district_id) {
0174                   fetchCurrentDeadline(weekNumber,
$profile.district_id);
0175               } else {
0176                   currentDeadline = null;
0177               }
0178           });
0179
0180           async function fetchCurrentDeadline(wk: number,
districtId: string) {
0181               if (navigator.onLine) {
0182                   const { data } = await supabase
0183                       .from("academic_calendar")
0184                       .select("deadline_date, description")
0185                       .eq("district_id", districtId)
0186                       .eq("week_number", wk)
0187                       .maybeSingle();
0188
0189                   if (data) {
0190                       currentDeadline = data;
0191                       // Update specific cache entry if needed or just
rely on the batch pre-fetch
0192                       return;
0193                   }
0194               }
0195
0196               // Offline Fallback
0197               const cached = await
getCachedMetadata(`calendar_open_${districtId}`);
0198               if (cached?.data) {
0199                   const entry = cached.data.find(
0200                       (e: any) => e.week_number === wk &&
e.is_active === true,
0201                   );
0202                   if (entry) {
0203                       currentDeadline = entry;
0204                       console.log("[upload] Using cached deadline for
week", wk);
0205                   }
0206               }
0207           }
0208
0209           // Mobile Optimization: Multi-stage library pre-warming
0210           // Hides the 2-5s loading delay of heavy workers on
mobile
0211           async function preWarmLibraries() {
0212               if (typeof window === "undefined") return;
0213               console.log("[upload] Pre-warming heavy libraries...");
0214
0215               // PDF.js: pre-load from CDN so OCR can use it
immediately
0216               if (!!(window as any).pdfjsLib) {
0217                   console.log("[upload] Fetching PDF.js...");
0218                   const script = document.createElement("script");
0219                   script.src =
"https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.4.120/pdf.min.js";
0220                   script.onload = () => {
0221                       (window as
any).pdfjsLib.GlobalWorkerOptions.workerSrc =
0222                       "https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.4.120/pdf.worker.
min.js";

```

```

0223     console.log("[upload] PDF.js loaded.");
0224   };
0225   document.head.appendChild(script);
0226 }
0227
0228 // Tesseract.js (dynamic import)
0229 try {
0230   const { createWorker } = await
import("tesseract.js");
0231   // Just the import is often enough to trigger Service
Worker caching
0232   console.log("[upload] Tesseract.js pre-warmed.");
0233 } catch (e) {
0234   console.warn("[upload] Pre-warm failed:", e);
0235 }
0236 }
0237
0238 // Keep this page's pending-sync panel in sync with the
shared queue store
0239 // the instant it changes (e.g. right after a background
sync finishes
0240 // elsewhere), instead of only refreshing on this page's
own online/mount
0241 // events. This also provides the initial fetch on mount.
0242 $effect(() => {
0243   $pendingSyncCount;
0244   refreshPendingItems();
0245 });
0246
0247 onMount(() => {
0248   // Delay pre-warming slightly to not block initial page
render
0249   setTimeout(preWarmLibraries, 3000);
0250
0251   const onOnline = () => {
0252     isOnline = true;
0253     refreshPendingItems();
0254   };
0255   const onOffline = () => {
0256     isOnline = false;
0257   };
0258   window.addEventListener("online", onOnline);
0259   window.addEventListener("offline", onOffline);
0260

```

## Upload Page Handlers

**src/routes/dashboard/upload/+page.svelte Lines 260-620**

```

0260
0261 // Handle files launched from OS (PWA File Handling
API)
0262 if (
0263   "launchQueue" in window &&
0264   "setConsumer" in (window as any).launchQueue
0265 ) {
0266   (window as
any).launchQueue.setConsumer((launchParams: any) => {
0267     if (launchParams.files.length > 0) {
0268       launchParams.files[0].getFile().then((file: File)
=> {
0269         selectedFile = file;
0270         addToast("info", "Opened from system:
${file.name}");
0271       });
0272     }
0273   });
0274 }
0275
0276 return () => {

```

```

0277   window.removeEventListener("online", onOnline);
0278   window.removeEventListener("offline", onOffline);
0279 };
0280 });
0281
0282 async function refreshPendingItems() {
0283   queueCount = await getQueueSize();
0284   if (queueCount > 0) {
0285     const items = await getQueueItems();
0286     pendingItems = items.map((item) => ({
0287       fileName: item.fileName,
0288       fileHash: item.fileHash,
0289       docType: item.options.docType || "DLL",
0290       weekNumber: item.options.weekNumber,
0291       teachingLoadId: item.options.teachingLoadId,
0292       timestamp: item.timestamp,
0293     }));
0294   } else {
0295     pendingItems = [];
0296   }
0297 }
0298
0299 // Handle shared file from the server-side hint (Share
Target)
0300 $effect(() => {
0301   if (data.sharedFile?.isShared) {
0302     addToast(
0303       "info",
0304       `Received shared file: ${data.sharedFile.name}.
Please confirm your selection below.`
0305     );
0306   }
0307 });
0308
0309 let dataLoadedForProfile = $state<string | null>(null);
0310
0311 $effect(() => {
0312   if ($profile && dataLoadedForProfile !== $profile.id) {
0313     untrack(() => {
0314       // District Supervisors cannot upload - redirect to
archive
0315       if ($profile!.role === 'District Supervisor') {
0316         dataLoadedForProfile = $profile!.id;
0317         addToast('info', 'District Supervisors use the
Archives tab to review submissions and add re
0318         goto('/dashboard/archive');
0319         return;
0320       }
0321       fetchInitialData($profile!);
0322       dataLoadedForProfile = $profile!.id;
0323     });
0324   }
0325 });
0326
0327 $effect(() => {
0328   if ($profile?.role) {
0329     const types =
getAllowedUploadDocTypes($profile.role);
0330     untrack(() => { allowedDocTypes = types; });
0331     if (!types.includes(untrack(() => docType))) {
0332       docType = types[0] || "DLL";
0333     }
0334   }
0335 });
0336
0337 // Sync subject from selected teaching load
0338 $effect(() => {
0339   if (teachingLoadId && teachingLoads.length > 0) {

```

```

0340     const load = teachingLoads.find(l => l.id ===
teachingLoadId);
0341     if (load?.subject) {
0342         subject = load.subject;
0343     }
0344     } else if (!teachingLoadId) {
0345         subject = "";
0346     }
0347 });
0348
0349 async function fetchInitialData(userProfile: Profile) {
0350     loadingTeachingLoads = true;
0351
0352     let loads: TeachingLoad[] = [];
0353     if (navigator.onLine) {
0354         const { data, error } = await supabase
0355             .from("teaching_loads")
0356             .select("id, subject, grade_level")
0357             .eq("user_id", userProfile.id)
0358             .eq("is_active", true)
0359             .order("subject, grade_level");
0360
0361         if (!error && data) {
0362             loads = data;
0363
0364             cacheMetadata(`teaching_loads_${userProfile.id}`, data);
0365         } else if (error) {
0366             console.error(
0367                 "[upload] Online fetch teaching loads error:",
0368                 error.message,
0369             );
0370         }
0371     }
0372     if (loads.length === 0) {
0373         const cached = await getCacheMetadata(
0374             `teaching_loads_${userProfile.id}`,
0375         );
0376         if (cached?.data) {
0377             loads = cached.data;
0378             console.log("[upload] Using cached teaching
loads");
0379         }
0380     }
0381
0382     if (loads.length > 0) {
0383         teachingLoads = loads;
0384         if (!teachingLoadId) {
0385             teachingLoadId = teachingLoads[0].id;
0386         }
0387     } else {
0388         teachingLoads = [];
0389         teachingLoadId = "";
0390         weekNumber = undefined;
0391         // Only show error if uploading DLL (which requires
teaching load)
0392         // School Head and Master Teacher uploading
ISP/ISR don't need teaching loads
0393         if (navigator.onLine && docType === 'DLL') {
0394             addToast("error", "No teaching loads found.
Please configure at least one teaching load before u
0395         }
0396     }
0397
0398     if (userProfile.district_id) {
0399         let calendarEntries: any[] = [];
0400
0401         // Try cache first for immediate UI

```

```

0402         const cacheKey =
`calendar_open_${userProfile.district_id}`;
0403         const cachedCal = await
getCacheMetadata(cacheKey);
0404         if (cachedCal?.data) {
0405             calendarEntries = (cachedCal.data as
any[]).filter(
0406                 (w) => w.is_active === true,
0407             );
0408             academicWeeks = calendarEntries.map((w) =>
w.week_number);
0409             console.log("[upload] Loaded calendar from
cache");
0410         }
0411
0412         // Sync with online if possible
0413         if (navigator.onLine) {
0414             const { data, error } = await supabase
0415                 .from("academic_calendar")
0416                 .select("week_number, deadline_date,
description, is_active")
0417                 .eq("district_id", userProfile.district_id)
0418                 .eq("is_active", true)
0419                 .order("week_number", { ascending: true });
0420
0421             if (!error && data) {
0422                 calendarEntries = data;
0423                 academicWeeks = data.map((w) =>
w.week_number);
0424                 cacheMetadata(cacheKey, data);
0425             } else if (error) {
0426                 console.error(
0427                     "[upload] Online fetch calendar error:",
0428                     error.message,
0429                 );
0430             }
0431         }
0432
0433         // Auto-detect current week only when teaching
loads are available
0434         if (
0435             teachingLoads.length > 0 &&
0436             calendarEntries.length > 0 &&
0437             !weekNumber
0438         ) {
0439             const today = new Date();
0440             today.setHours(0, 0, 0, 0);
0441
0442             const sorted = [...calendarEntries]
0443                 .filter((e) => e.deadline_date)
0444                 .sort(
0445                     (a, b) =>
0446                         new Date(a.deadline_date).getTime() -
0447                         new Date(b.deadline_date).getTime(),
0448                 );
0449
0450             const currentEntry = sorted.find((e) => {
0451                 const deadline = new Date(e.deadline_date);
0452                 deadline.setHours(23, 59, 59, 999);
0453                 return deadline >= today;
0454             });
0455
0456             if (currentEntry) {
0457                 weekNumber = currentEntry.week_number;
0458             } else if (sorted.length > 0) {
0459                 weekNumber = sorted[sorted.length -
1].week_number;
0460             }

```

```

0461     }
0462   }
0463
0464   if (navigator.onLine) {
0465     try {
0466       await syncLedgerFromServer(userProfile.id);
0467     } catch (err) {
0468       console.warn("[upload] Sync context fetch
error:", err);
0470     }
0471   }
0472
0473   loadingTeachingLoads = false;
0474 }
0475
0476 function mapTeachingLoad(metadata:
Partial<DocMetadata>): string {
0477   if (!metadata.subject) return teachingLoadId;
0478
0479   const sub = metadata.subject.toUpperCase();
0480   const grade = metadata.gradeLevel;
0481
0482   // Try exact subject match first
0483   const exactMatch = teachingLoads.find(
0484     (l) =>
0485       l.subject.toUpperCase() === sub &&
0486       (!grade || l.grade_level === grade),
0487   );
0488   if (exactMatch) return exactMatch.id;
0489
0490   // Try subject name contains or common mappings
0491   const fuzzyMatch = teachingLoads.find((l) => {
0492     const loadSub = l.subject.toUpperCase();
0493     // GMRC / EsP mapping
0494     if (sub === "GMRC" ||
sub.includes("PAGPAPAKATAO")) {
0495       return (
0496         loadSub === "GMRC" ||
0497         loadSub.includes("ESP") ||
0498         loadSub.includes("VALUES")
0499       );
0500     }
0501     // AP mapping
0502     if (sub === "AP" || sub.includes("PANLIPUNAN")) {
0503       return loadSub === "AP" ||
loadSub.includes("SOCIAL");
0504     }
0505     return loadSub.includes(sub) ||
sub.includes(loadSub);
0506   });
0507
0508   return fuzzyMatch ? fuzzyMatch.id : "";
0509 }
0510
0511 function parseDocDate(dateStr: string | null): Date | null
{
0512   if (!dateStr) return null;
0513   try {
0514     console.log("[upload] Attempting to parse date
from:", dateStr);
0515     // Extract the month and day part using regex to
ignore any surrounding garbage
0516     const match = dateStr.match(
0517       /(?:JAN|FEB|MAR|APR|MAY|JUN|JUL|AUG|SEP|OCT|NOV|DE
C)[a-zA-Z]*\s+\d{1,2}/i,
0518     );
0519
0520     if (!match) {
0521       console.log(
0522         "[upload] Could not find Month Day pattern in
string",
0523       );
0524       return null;
0525     }
0526
0527     const cleaned = match[0];
0528     const yearMatch = dateStr.match(/(\d{4})/);
0529     const year = yearMatch
? yearMatch[0]
: new Date().getFullYear().toString();
0532
0533     const finalDateStr = `${cleaned}, ${year}`;
0534     console.log("[upload] Parsed clean date string:",
finalDateStr);
0535
0536     const d = new Date(finalDateStr);
0537     return isNaN(d.getTime()) ? null : d;
0538   } catch (e) {
0539     console.error("[upload] Date parse error:", e);
0540     return null;
0541   }
0542 }
0543
0544 async function lookupWeekByDate(date: Date, districtId:
string) {
0545   // Find the week where this document's date is
between (deadline - 10 days) and (deadline)
0546   // This prevents dates from months ago jumping
forward to the first future deadline.
0547   const isoDate = date.toISOString();
0548
0549   const endDate = new Date(date);
0550   endDate.setDate(endDate.getDate() + 10);
0551   const isoEndDate = endDate.toISOString();
0552
0553   console.log(
0554     "[upload] Looking up week between: ${isoDate}
and ${isoEndDate}, district: ${districtId}",
0555   );
0556
0557   const { data, error } = await supabase
0558     .from("academic_calendar")
0559     .select("week_number, deadline_date")
0560     .gte("deadline_date", isoDate)
0561     .lte("deadline_date", isoEndDate)
0562     .eq("district_id", districtId)
0563     .eq("is_active", true)
0564     .order("deadline_date", { ascending: true })
0565     .limit(1)
0566     .maybeSingle();
0567
0568   if (error) {
0569     console.error("[upload] Calendar lookup error:",
error);
0570     return null;
0571   }
0572
0573   console.log(`[upload] Lookup result:`, data);
0574   return data ? data.week_number : null;
0575 }
0576
0577 async function onFileSelected(file: File) {
0578   if ($settings.maintenance_mode) {
0579     addToast(

```

```

0580     "error",
0581     "System is currently under maintenance.
Uploads are disabled.",
0582   );
0583   return;
0584 }
0585 selectedFile = file;
0586 result = null;
0587 ocrConfidence = null;
0588 fileSizeWarning =
0589   file.size > $settings.max_upload_size_mb * 1024 *
1024;
0590 preConvertedPdf = null;
0591
0592 // Runs OCR-based metadata detection: doc type,
week number, and teaching load auto-fill
0593 detectingMetadata = true;
0594 scanProgress = 5;
0595 try {
0596   const ext =
file.name.split('.').pop()?.toLowerCase();
0597   let ocrTarget = file;
0598   if ((ext === 'docx' || ext === 'doc') &&
navigator.onLine) {
0599     scanProgress = 20;
0600     // Same server-proxy -> Google Apps Script
fallback chain the
0601     // actual upload pipeline uses (transcode.ts), so
pre-upload
0602     // metadata detection doesn't silently skip OCR
for Word docs
0603     // just because the server-side proxy has no
working engine.
0604     try {
0605       const { pdfBytes } = await
transcodeToPdf(file);
0606       // Cached so handleUpload's pipeline run can
reuse these
0607       // bytes instead of converting this exact file
again.
0608       preConvertedPdf = { file, bytes: pdfBytes };
0609       scanProgress = 48;
0610       ocrTarget = new File([pdfBytes as BlobPart],
file.name.replace(/\.w+$/, '.pdf'), { type: 'a
0611     } catch (err) {
0612       console.warn("[upload] Pre-upload conversion
failed, skipping OCR metadata detection:", err)
0613     }
0614   }
0615   scanProgress = Math.max(scanProgress, 55);
0616   const metadata = await
extractMetadata(ocrTarget);
0617   scanProgress = 82;
0618   console.log("[upload] OCR metadata result:",
metadata);
0619   detectedMetadata = metadata;
0620   scanProgress = 92;

```

## Upload Page Interface

**src/routes/dashboard/upload/+page.svelte Lines 620-980**

```

0620   scanProgress = 92;
0621
0622   if (metadata.docType !== "Unknown") {
0623     docType = metadata.docType;
0624   }
0625
0626   // Only auto-fill week number and teaching load for
DLL documents

```

```

0627   if (docType === "DLL") {
0628     if (metadata.weekNumber) {
0629       weekNumber = metadata.weekNumber;
0630     } else if (metadata.date && $profile?.district_id)
{
0631       const docDate =
parseDocDate(metadata.date);
0632       if (docDate) {
0633         const derivedWeek = await
lookupWeekByDate(
0634           docDate,
0635           $profile.district_id,
0636         );
0637         if (derivedWeek) {
0638           weekNumber = derivedWeek;
0639           console.log(
0640             `[upload] Derived week
${weekNumber} from date ${metadata.date}`,
0641           );
0642         }
0643       }
0644     }
0645
0646     const mappedId =
predictLoad(metadata.rawText,
teachingLoads) ||
0647     mapTeachingLoad(metadata);
0648     if (mappedId) teachingLoadId = mappedId;
0649   }
0650
0651   if (metadata.confidence) {
0652     ocrConfidence = metadata.confidence;
0653   }
0654
0655   // Check for initial mismatch
0656   if (teachingLoadId) {
0657     mismatchAlert = validateSelection(
0658       teachingLoadId,
0659       metadata.rawText,
0660       teachingLoads,
0661     );
0662   }
0663
0664   const { speak } = await
import("$lib/utils/voiceGuide");
0665   speak("Smart detection complete. Fields
updated.");
0666   scanProgress = 100;
0667 } catch (err) {
0668   console.error("[upload] OCR error:", err);
0669 } finally {
0670   scanProgress = 100;
0671   detectingMetadata = false;
0672 }
0673
0674 // Hash calculation for early duplicate detection.
0675 try {
0676   const buffer = await file.arrayBuffer();
0677   const hashBuffer = await
crypto.subtle.digest("SHA-256", buffer);
0678   const hashArray = Array.from(new
Uint8Array(hashBuffer));
0679   fileHash = hashArray
0680     .map((b) => b.toString(16).padStart(2, "0"))
0681     .join("");
0682   console.log("[upload] Early hash calculation
complete:", fileHash);
0683   // checkExistingSubmission is triggered via $effect

```

```

0685     } catch (hashErr) {
0686         console.warn("[upload] Early hash calculation
failed:", hashErr);
0687     }
0688 }
0689
0690 /** Auto-wait for profile + teaching loads to finish
loading (polls every 200ms) */
0691 function waitForDataReady(timeoutMs: number):
Promise<boolean> {
0692     return new Promise((resolve) => {
0693         const start = Date.now();
0694         const check = () => {
0695             // Only wait for the loading FLAGS, not the
selection itself
0696             if ($profile && !loadingTeachingLoads) {
0697                 resolve(true);
0698             } else if (Date.now() - start > timeoutMs) {
0699                 resolve(false);
0700             } else {
0701                 setTimeout(check, 200);
0702             }
0703         };
0704         check();
0705     });
0706 }
0707 async function handleUpload() {
0708     if ($settings.maintenance_mode) {
0709         addToast(
0710             "error",
0711             "System maintenance is active. Upload
cancelled.",
0712         );
0713         return;
0714     }
0715     if (!selectedFile) {
0716         addToast("error", "Please select a file to upload");
0717         return;
0718     }
0719
0720     // Auto-wait for profile and teaching load list to be
ready (up to 5s)
0721     if (loadingTeachingLoads || !$profile) {
0722         addToast("info", "Syncing data, please wait...");
0723         const ready = await waitForDataReady(15000);
0724         if (!ready) {
0725             addToast("error", "Timeout: Could not sync
profile data.");
0726             return;
0727         }
0728     }
0729
0730     if (!canUploadDocument($profile?.role || "", docType))
{
0731         addToast("error",
getUploadGuidance($profile?.role || ""));
0732         return;
0733     }
0734
0735     if (requiresTeachingLoadSelection($profile?.role || "",
docType) && !teachingLoadId) {
0736         addToast("error", "Please select a teaching load
before uploading");
0737         showLoadPicker = true;
0738         return;
0739     }
0740
0741     // Block Word files when offline (GAS needs internet)

```

```

0742     if (!isOnline) {
0743         const ext =
selectedFile.name.split('.').pop()?.toLowerCase();
0744         if (ext === 'docx' || ext === 'doc') {
0745             addToast("error", "Word document conversion
requires an internet connection. Please upload a PDF
0746             return;
0747         }
0748     }
0749
0750     processing = true;
0751     result = null;
0752
0753     const { speak, VoicePrompts } = await
import("$lib/utls/voiceGuide");
0754     speak(VoicePrompts.UPLOAD_START);
0755
0756     // Select the appropriate pipeline based on
connectivity
0757     const pipelineOptions = {
0758         userId: $profile!.id,
0759         docType,
0760         subject: subject || undefined,
0761         weekNumber,
0762         teachingLoadId,
0763         enforceOcr: $settings.enforce_ocr,
0764         submissionWindowDays:
$settings.submission_window_days,
0765         preDetectedMetadata: detectedMetadata,
0766         preConvertedPdfBytes:
preConvertedPdf?.file === selectedFile ?
preConvertedPdf.bytes : undefined,
0767         // Show real bytes-sent during the transfer.
Without this the bar
0768         // parks at one number for the whole upload, which
on a slow phone
0769         // connection is minutes of UI that looks identical to
a hang.
0770         onTransferProgress: (loaded: number, total:
number) => {
0771             if (!total) return;
0772             const mb = (n: number) => (n / 1024 /
1024).toFixed(1);
0773             progress = 50 + Math.round((loaded / total) *
30);
0774             message = `Uploading... ${mb(loaded)} MB of
${mb(total)} MB`;
0775             },
0776         },
0777     };
0778
0779     // The pipeline runs for minutes on a phone. Without
a wake lock, a
0780     // dimming screen or an app switch mid-upload gets
the page throttled
0781     // and its in-flight requests suspended, which
surfaced as spurious
0782     // "timed out" failures on requests that never got to
run.
0783     const { acquireWakeLock, releaseWakeLock } =
await import("$lib/utls/wakeLock");
0784     await acquireWakeLock();
0785
0786     try {
0787         const pipeline = runPipeline(selectedFile,
pipelineOptions);
0788
0789         for await (const event of pipeline) {
0790             currentPhase = event.phase;

```

```

0791     progress = event.progress;
0792     message = event.message;
0793
0794     if (event.metadata) {
0795         if (event.metadata.docType !== "Unknown")
0796             docType = event.metadata.docType;
0797         if (event.metadata.weekNumber)
0798             weekNumber =
event.metadata.weekNumber;
0799         if (event.metadata.confidence !== undefined)
0800             ocrConfidence =
event.metadata.confidence;
0801         const { speak } = await
import("$lib/utls/voiceGuide");
0802         speak("Smart detection complete. Details
updated.");
0803     }
0804
0805     if (event.phase === "done" && event.result) {
0806         result = event.result;
0807         showSuccessModal = true;
0808         speak(VoicePrompts.UPLOAD_COMPLETE);
0809         selectedFile = null;
0810         await refreshPendingItems();
0811         connectivity.refreshPendingCount();
0812     }
0813
0814     if (event.phase === "error") {
0815         const { speak, VoicePrompts } = await import(
0816             "$lib/utls/voiceGuide"
0817         );
0818         speak(VoicePrompts.ERROR);
0819         addToast("error", event.message);
0820     }
0821 }
0822 } finally {
0823     await releaseWakeLock();
0824     processing = false;
0825 }
0826
0827 await refreshPendingItems();
0828 }
0829
0830 // Reactively validate selection mismatch
0831 $effect(() => {
0832     if (teachingLoadId && detectedMetadata?.rawText) {
0833         mismatchAlert = validateSelection(
0834             teachingLoadId,
0835             detectedMetadata.rawText,
0836             teachingLoads,
0837         );
0838     } else {
0839         mismatchAlert = null;
0840     }
0841 });
0842
0843 function closeSuccessModal() {
0844     showSuccessModal = false;
0845 }
0846
0847 function handleUploadAnother() {
0848     showSuccessModal = false;
0849     result = null;
0850     detectedMetadata = null;
0851     ocrConfidence = null;
0852     fileHash = "";
0853 }
0854

```

```

0855     function handleViewArchive() {
0856         showSuccessModal = false;
0857         goto("/dashboard/archive");
0858     }
0859 </script>
0860
0861 <svelte:head>
0862     <title>Upload Document â€ CEDIMS</title>
0863 </svelte:head>
0864
0865 <div>
0866     <!-- Header -->
0867     <div
0868         class="mb-6 flex flex-col sm:flex-row items-start
sm:items-center justify-between gap-4"
0869     >
0870         <div>
0871             <h1
0872                 class="text-2xl font-bold text-text-primary flex
items-center gap-2"
0873             >
0874                 Upload Document <span
0875                     class="px-2 py-0.5 rounded-md text-xs font-
semibold uppercase tracking-normal border
0876                         {isOnline
0877                             ? 'bg-gov-green/5 text-gov-green border-
gov-green/10'
0878                             : 'bg-gov-gold/10 text-gov-gold-dark
border-gov-gold/20'"
0879                     >{isOnline ? "Online" : "Offline"}</span>
0880                 >
0881             </h1>
0882             <p class="text-base text-text-secondary mt-1">
0883                 {isOnline
0884                     ? uploadSubtitle
0885                     : "Files will be saved locally and synced when
online"}
0886             </p>
0887             {#if currentDeadline}
0888                 <div
0889                     class="mt-2 inline-flex items-center gap-2 px-
3 py-1 rounded-full bg-gov-blue/5 border borde
0890                 >
0891                     Deadline: {new Date(
0892                         currentDeadline.deadline_date,
0893                     ).toLocaleDateString("en-PH", {
0894                         month: "short",
0895                         day: "numeric",
0896                         year: "numeric",
0897                     })}
0898                     {#if currentDeadline.week_number}
0899                         Â· Week {currentDeadline.week_number}
0900                     {/if}
0901                 </div>
0902             {/if}
0903         </div>
0904
0905         <!-- Hidden on mobile: every upload there is queued
by design and syncs
0906             on its own, so this panel would appear after every
submission and
0907             imply the document hadn't been handed in.
Desktop only queues when
0908             a direct upload actually failed, where it's worth
surfacing. -->
0909             {#if queueCount > 0 && !isMobile}
0910                 <div class="flex flex-col gap-2 w-full sm:w-auto">
0911                     <div>

```

```

0912         onclick={() => (showPendingPanel
= !showPendingPanel)}
0913         class="flex items-center justify-between gap-
2 px-4 py-2 rounded-xl bg-gov-gold/10 text-gov-
0914         role="button"
0915         tabindex="0"
0916         onkeydown={(e) =>
0917         e.key === "Enter" &&
0918         (showPendingPanel
= !showPendingPanel)}
0919     >
0920     <div class="flex items-center gap-2">
0921     <span class="relative flex h-2 w-2">
0922     <span
0923     class=" absolute inline-flex h-full w-full
rounded-full bg-gov-gold opacity-75"
0924     ></span>
0925     <span
0926     class="relative inline-flex rounded-full
h-2 w-2 bg-gov-gold"
0927     ></span>
0928     </span>
0929     <!-- Worded as ongoing rather than
"pending sync": an
0930     upload that fell back to the background
queue is
0931     reported to the teacher as archived, so
a label
0932     implying it had not been submitted
contradicted
0933     that. It still surfaces the real state — the
file
0934     is finishing — without framing it as a
failure. -->
0935     {queueCount} file{queueCount > 1 ? "s" : ""}
finishing in background
0936     </div>
0937     <div class="flex items-center gap-2">
0938     <svg
0939     class="w-4 h-4 transition-transform
{showPendingPanel
0940     ? 'rotate-180'
0941     : ''}"
0942     fill="none"
0943     stroke="currentColor"
0944     viewBox="0 0 24 24"
0945     ><path
0946     stroke-linecap="round"
0947     stroke-linejoin="round"
0948     stroke-width="2"
0949     d="M19 9l-7 7-7-7"
0950     /></svg>
0951     >
0952     </div>
0953     </div>
0954     {#if isOnline}
0955     <button
0956     onclick={async () => {
0957     await processQueue(true);
0958     await refreshPendingItems();
0959     }}
0960     class="w-full px-3 py-2 bg-gov-blue text-
white text-xs font-semibold rounded-lg hover:bg
0961     >
0962     Sync All Now
0963     </button>
0964     {/if}
0965     {#if showPendingPanel &&

```

```

pendingItems.length > 0}
0966     <div
0967     class="bg-surface-white border border-
border-subtle rounded-xl p-3 space-y-2 max-h-48 ov
0968     >
0969     {#each pendingItems as item}
0970     <div
0971     class="flex items-center justify-
between py-2 px-3 bg-surface-muted rounded-lg t
0972     >
0973     <div class="min-w-0">
0974     <p
0975     class="font-semibold text-text-
primary truncate"
0976     >
0977     {item.fileName}
0978     </p>
0979     <p class="text-text-muted">
0980     {getDocumentLabel(item.docType)} · Week {item.weekNumber
||

```

## Upload Pipeline Core

**src/lib/utls/pipeline.ts Lines 1-240**

```

0001 /**
0002  * Upload Pipeline Orchestrator (Worker-Powered) —
Resilient Hybrid Mode
0003  *
0004  * This orchestrator manages the high-stakes document
archival pipeline.
0005  * It is designed to be extremely resilient, especially on
mobile devices.
0006  *
0007  * HYBRID STRATEGY:
0008  * 1. Attempt secure cloud archival (Online).
0009  * 2. If the server check/upload stalls (timeout) or network
fails,
0010  *    automatically move the document to the local
persistent vault (Offline).
0011  * 3. Sync background processes will attempt to finalize
the archive later.
0012  */
0013
0014 import { transcodeToPdf } from './transcode';
0015 import PdfWorker from './pdf.worker?worker';
0016 import { compressFile } from './compress';
0017 import { supabase } from './supabase';
0018 import { env } from '$env/dynamic/public';
0019 import { createNotification } from './notificationSystem';
0020 import { getCurrentSchoolYear } from './schoolYear';
0021 import { isMobileDevice } from './device';
0022 import type { PipelineEvent, PipelineOptions } from
'$lib/types/pipeline';
0023 export type { PipelinePhase, PipelineResult } from
'$lib/types/pipeline';
0024
0025 interface CoreResult {
0026     stampedBytes: Uint8Array;
0027     fileHash: string;
0028     fileName: string;
0029     filePath: string;
0030     detectedMetadata: any;
0031     activeWeekNumber?: number;
0032     activeDocType: string;
0033     rawText: string;
0034 }
0035
0036 function runWorkerTask(worker: Worker, type: string,

```

```

payload: any, transfer: Transferable[] = []): Promise<any>
0037   return new Promise((resolve, reject) => {
0038       const id = Math.random().toString(36).substring(7);
0039       const handler = (e: MessageEvent) => {
0040           if (e.data.id === id) {
0041               worker.removeEventListener('message',
handler);
0042               if (e.data.success) resolve(e.data.payload);
0043               else reject(new Error(e.data.error));
0044           }
0045       };
0046       worker.addEventListener('message', handler);
0047       worker.postMessage({ type, payload, id }, transfer);
0048   });
0049 }
0050
0051 export async function withTimeout<T>(promise:
Promise<T>, timeoutMs: number, errorMessage: string):
Promise<T> {
0052   let timeoutId: any;
0053   const timeoutPromise = new Promise<never>((_, reject)
=> {
0054       timeoutId = setTimeout(() => reject(new
Error(errorMessage)), timeoutMs);
0055   });
0056
0057   try {
0058       const result = await Promise.race([promise,
timeoutPromise]);
0059       return result as T;
0060   } finally {
0061       clearTimeout(timeoutId);
0062   }
0063 }
0064
0065 // A single transient blip (brief signal drop, one dropped
packet) on a bad
0066 // mobile connection shouldn't fail the whole upload after
already getting
0067 // this far. Used only for calls whose failure is meant to be
a hard stop
0068 // (unlike the calendar lookups, which fail soft) — it retries
the same
0069 // bounded attempt once more before giving up for real.
0070 async function withRetry<T>(fn: () => Promise<T>,
attempts: number, delayMs: number): Promise<T> {
0071   let lastErr: unknown;
0072   for (let i = 0; i < attempts; i++) {
0073       try {
0074           return await fn();
0075       } catch (err) {
0076           lastErr = err;
0077           if (i < attempts - 1) await new Promise((r) =>
setTimeout(r, delayMs));
0078       }
0079   }
0080   throw lastErr;
0081 }
0082
0083 interface XhrUploadResult {
0084   ok: boolean;
0085   status: number;
0086   body: string;
0087 }
0088
0089 /**
0090  * Upload via XMLHttpRequest rather than fetch, for two
mobile-specific reasons.

```

```

0091  *
0092  * 1. fetch() cannot report upload progress at all, so the
bar sat frozen for
0093  * the entire transfer — on a slow phone connection
that's minutes of a UI
0094  * that looks hung, which is indistinguishable from a
dead connection.
0095  * XHR exposes upload.onprogress, so real bytes-sent
can be surfaced.
0096  * 2. A single flat timeout punishes slow-but-working
transfers: a 3MB file at
0097  * 5 KB/s legitimately needs ~10 minutes, and killing that
at 2 minutes
0098  * throws away a transfer that was succeeding. This
instead times out on
0099  * *inactivity* — it only aborts if no bytes move for
stallMs, so a slow
0100  * upload runs as long as it needs while a genuinely
dead one still fails
0101  * promptly.
0102  */
0103 function xhrUpload(
0104   url: string,
0105   body: XMLHttpRequestBodyInit,
0106   opts: {
0107       method?: string;
0108       headers?: Record<string, string>;
0109       stallMs?: number;
0110       onProgress?: (loaded: number, total: number) =>
void;
0111   } = {}
0112 ): Promise<XhrUploadResult> {
0113   const { method = 'POST', headers = {}, stallMs = 90000,
onProgress } = opts;
0114
0115   return new Promise((resolve, reject) => {
0116       const xhr = new XMLHttpRequest();
0117       let stallTimer: any;
0118
0119       const resetStallTimer = () => {
0120           clearTimeout(stallTimer);
0121           stallTimer = setTimeout(() => {
0122               xhr.abort();
0123               reject(new Error('Upload stalled — no data sent
for a while. Check your connection and try again
0124               }, stallMs);
0125           });
0126
0127       const done = () => clearTimeout(stallTimer);
0128
0129       xhr.open(method, url, true);
0130       for (const [k, v] of Object.entries(headers))
xhr.setRequestHeader(k, v);
0131
0132       xhr.upload.onprogress = (e) => {
0133           resetStallTimer();
0134           if (e.lengthComputable && onProgress)
onProgress(e.loaded, e.total);
0135       };
0136       // Bytes are still in flight while the server responds;
keep the stall
0137       // timer alive so a slow server round-trip isn't
mistaken for a stall.
0138       xhr.onprogress = resetStallTimer;
0139       xhr.onload = () => {
0140           done();
0141           resolve({ ok: xhr.status >= 200 && xhr.status <
300, status: xhr.status, body: xhr.responseText });

```

```

0142     };
0143     xhr.onerror = () => { done(); reject(new
Error('Network error during upload.')); };
0144     xhr.onabort = () => done();
0145     xhr.ontimeout = () => { done(); reject(new
Error('Upload timed out.')); };
0146
0147     resetStallTimer();
0148     xhr.send(body);
0149   });
0150 }
0151
0152 async function sha256Hex(bytes: Uint8Array):
Promise<string> {
0153   const digest = await crypto.subtle.digest('SHA-256',
bytes.buffer as ArrayBuffer);
0154   return Array.from(new Uint8Array(digest))
0155     .map((b) => b.toString(16).padStart(2, '0'))
0156     .join("");
0157 }
0158
0159 async function* runPipelineCore(
0160   file: File,
0161   options: PipelineOptions,
0162   worker: Worker
0163 ): AsyncGenerator<PipelineEvent & { _core?:
CoreResult }> {
0164
0165   // Hash the bytes the teacher actually picked, before
transcode/compress touch them —
0166   // those steps stamp new PDF dates or re-encode
content on every run, so hashing
0167   // post-transform meant the same document got a
different hash on every upload and
0168   // duplicate detection never fired.
0169   const originalBytes = new Uint8Array(await
file.arrayBuffer());
0170   const fileHash = await sha256Hex(originalBytes);
0171
0172   // Transcode (Word to PDF)
0173   yield { phase: 'transcoding', progress: 10, message:
'Converting to PDF...' };
0174   // Reuse bytes from an earlier conversion of this exact
file (e.g. the
0175   // upload page's pre-check OCR step) when available,
rather than sending
0176   // the same docx/doc through the conversion engine a
second time.
0177   let pdfBytes = file.type === 'application/pdf'
0178     ? originalBytes
0179     : options.preConvertedPdfBytes ?? (await
transcodeToPdf(file)).pdfBytes;
0180
0181   // Detect a low-power/slow-connection state and skip
heavy compression if the
0182   // file is already small, to save CPU/battery on mobile.
0183   // effectiveType is one of 'slow-2g' | '2g' | '3g' | '4g'.
Matching only
0184   // '2g' missed 'slow-2g' entirely — the very slowest class,
i.e. exactly
0185   // the connection this branch exists to protect — and
also treated a
0186   // crawling '3g' link as if it were fine.
0187   const effectiveType = (navigator as
any).connection?.effectiveType;
0188   const isSlowConnection =
0189     effectiveType === 'slow-2g' ||
0190     effectiveType === '2g' ||

```

```

0191     effectiveType === '3g' ||
0192     (navigator as any).connection?.saveData === true;
0193   const isSmallEnough = pdfBytes.byteLength < 2 * 1024
* 1024; // 2MB
0194
0195   if (isSlowConnection && isSmallEnough) {
0196     console.log('[pipeline] Low-power/Slow-connection
detected. Skipping non-essential compression.');
```

```

0197     yield { phase: 'compressing', progress: 30, message:
'Preparing your file...' };
0198   } else {
0199     yield { phase: 'compressing', progress: 30, message:
'Preparing your file...' };
0200     pdfBytes = await compressFile(pdfBytes);
0201   }
0202
0203   // Analyzing (OCR): uses the converted PDF bytes, not
the original file.
0204   yield { phase: 'analyzing', progress: 30, message:
'Reading your document...' };
0205   const { extractMetadata } = await import('./ocr');
0206   const pdfBlob = new Blob([pdfBytes as BlobPart]);
0207   const ocrFile = file.type === 'application/pdf' ? file : new
File([pdfBlob], file.name.replace(/\.w+$/, '.p
0208   const hasPreDetected =
options.preDetectedMetadata?.docType &&
options.preDetectedMetadata?.docType !== 'Unk
0209   const detectedMetadata = hasPreDetected ?
options.preDetectedMetadata : await extractMetadata(ocrFile);
0210
0211   // Compress & hash
0212   yield { phase: 'compressing', progress: 50, message:
'Securing your file...' };
0213   // Only the compressed bytes are taken from here. The
worker also returns a
0214   // hash of what it was given, but that is post-processing
bytes — the very
0215   // thing that made duplicate detection unreliable — so
fileHash above,
0216   // taken from the original upload, is what identifies the
document.
0217   const { compressedBytes } = await runWorkerTask(
0218     worker,
0219     'COMPRESS_AND_HASH',
0220     { pdfBytes },
0221     [pdfBytes.buffer]
0222   );
0223
0224   // Stamping
0225   yield { phase: 'stamping', progress: 70, message:
'Adding verification code...' };
0226   const { generateQrPng } = await import('./qr-stamp');
0227   const qrBytes = await generateQrPng(fileHash);
0228   const { stampedBytes } = await runWorkerTask(
0229     worker,
0230     'STAMP_QR',
0231     { compressedBytes, qrBytes, fileHash },
0232     [compressedBytes.buffer, qrBytes.buffer]
0233   );
0234
0235   const activeWeekNumber = options.weekNumber ||
detectedMetadata?.weekNumber;
0236   const activeDocType = options.docType ||
detectedMetadata?.docType || 'DLL';
0237   const rawText = options.rawText ||
detectedMetadata?.rawText || "";
0238   const fileName = file.name.replace(/\.w+$/, '.pdf');
0239   const sanitizedFileName = (fileName ||

```

```
'document').replace(/s+/g, '_').replace(/^[a-zA-Z0-9._-]/g, "");
0240 const filePath =
`submissions/${options.userId}/${activeDocType}/${Date.now()}
_${sanitizedFileName}`;
```

## Online and Offline Pipeline

### src/lib/utls/pipeline.ts Lines 240-520

```
0240 const filePath =
`submissions/${options.userId}/${activeDocType}/${Date.now()}
_${sanitizedFileName}`;
0241
0242 yield {
0243   phase: 'uploading',
0244   progress: 90,
0245   message: 'Ready to upload...',
0246   _core: {
0247     stampedBytes: new Uint8Array(stampedBytes),
0248     fileHash,
0249     fileName,
0250     filePath,
0251     detectedMetadata,
0252     activeWeekNumber,
0253     activeDocType,
0254     rawText
0255   }
0256 };
0257 }
0258
0259 async function* runOnlinePipelineResilient(
0260   core: CoreResult,
0261   options: PipelineOptions
0262 ): AsyncGenerator<PipelineEvent> {
0263   const { stampedBytes, fileHash, fileName, filePath,
activeWeekNumber, activeDocType, rawText } = core;
0264
0265   yield { phase: 'uploading', progress: 10, message:
'Checking your document...' };
0266
0267   const { lookupOfflineDoc, cacheVerifiedDoc,
calculateComplianceStatus } = await import('./offline');
0268   const { recordSubmission } = await
import('./offlineSubmissionLedger');
0269
0270   // Look up calendar_id + deadline from
academic_calendar using detected week number.
0271   // This is metadata only — calculateComplianceStatus()
already falls back
0272   // to 'compliant' when there's no deadline — so a
slow/failed lookup must
0273   // never abort the whole upload. Each query is time-
bounded (previously
0274   // unbounded, so a bad connection could hang here
forever with no error),
0275   // and any failure (timeout or otherwise) is swallowed:
the archive still
0276   // goes through, just without deadline-based lateness
tracking for this
0277   // submission.
0278   let calendarId = options.calendarId || null;
0279   let deadlineDate: Date | undefined;
0280   if (!calendarId && activeWeekNumber) {
0281     try {
0282       const { data: calEntry } = await withTimeout(
0283         supabase
0284           .from('academic_calendar')
0285           .select('id, deadline_date')
0286           .eq('school_year', options.schoolYear ||
getCurrentSchoolYear())
```

```
0287           .eq('week_number', activeWeekNumber)
0288           .maybeSingle() as any,
0289         10000,
0290         'Calendar lookup timed out.'
0291       ) as { data: any };
0292       if (!calEntry) {
0293         yield { phase: 'uploading', progress: 15,
message: 'Checking your document...' };
0294         const { data: profileData } = await withTimeout(
0295           supabase
0296             .from('profiles')
0297             .select('district_id')
0298             .eq('id', options.userId)
0299             .single() as any,
0300           10000,
0301           'Profile lookup timed out.'
0302         ) as { data: any };
0303         if (profileData?.district_id) {
0304           const { data: calByDistrict } = await
withTimeout(
0305             supabase
0306               .from('academic_calendar')
0307               .select('id, deadline_date')
0308               .eq('district_id', profileData.district_id)
0309               .eq('week_number', activeWeekNumber)
0310               .maybeSingle() as any,
0311             10000,
0312             'Calendar lookup timed out.'
0313           ) as { data: any };
0314           if (calByDistrict) {
0315             calendarId = calByDistrict.id;
0316             if (calByDistrict.deadline_date)
deadlineDate = new Date(calByDistrict.deadline_date);
0317           }
0318         } else {
0319           calendarId = calEntry.id;
0320           if (calEntry.deadline_date) deadlineDate = new
Date(calEntry.deadline_date);
0321         }
0322       } catch (err: any) {
0323         console.warn('[pipeline] Calendar lookup
failed/timed out, continuing without a deadline:', err?.mes
0324       )
0325     }
0326   }
0327
0328   yield { phase: 'uploading', progress: 20, message:
'Checking for duplicates...' };
0329
0330   // Two different things, deliberately handled differently:
0331   //
0332   // Same DOCUMENT (identical file hash) -> rejected.
Re-submitting the very
0333   // same file is never a new submission, and UNIQUE
(file_hash) enforces it
0334   // at the database besides.
0335   //
0336   // Same SLOT (another DLL for a week/subject that
already has one) ->
0337   // accepted and marked 'supplementary' by the slot
check further down.
0338   // That's a legitimate additional document, and
supplementary is excluded
0339   // from compliance rate, upload totals and "missing"
checks, so it can't
0340   // distort any figure.
0341   if (await lookupOfflineDoc(fileHash)) {
0342     throw new Error('This exact document has already
```

```

been uploaded. Choose a different file, or check My Fil
0343   }
0344
0345   // Cross-teacher check, via a narrow RPC rather than a
direct table select
0346   // (see migrations/20260910_*.sql). A positive result
rejects; a timeout is
0347   // advisory only and must not fail an upload the phone
has already done
0348   // minutes of work for — UNIQUE (file_hash) still
backstops it at insert.
0349   try {
0350     // Plain array request rather than .maybeSingle():
PostgREST only
0351     // returns 406 when a singular representation is
requested (the
0352     // Accept header .maybeSingle().single() set) and 0
rows come back —
0353     // a routine, expected outcome here, not an error
worth a red network
0354     // failure in the console on every non-duplicate
upload.
0355     const { data: matches } = await withRetry(
0356       () => withTimeout(
0357         supabase.rpc('check_duplicate_submission_hash', { p_hash:
fileHash }) as any,
0358         15000,
0359         'Server integrity check timed out.'
0360       ) as Promise<{ data: any[] }>,
0361       2,
0362       1500
0363     );
0364     const hashMatch = matches?.[0];
0365     if (hashMatch) throw new Error('This exact document
has already been uploaded (${hashMatch.file_name}).'
0366   } catch (err: any) {
0367     if (err?.message?.startsWith('This exact document'))
throw err;
0368     console.warn('[pipeline] Duplicate pre-check
unavailable, continuing:', err?.message);
0369   }
0370
0371   yield { phase: 'uploading', progress: 40, message:
'Uploading...' };
0372   const session = await supabase.auth.getSession();
0373   const token = session.data.session?.access_token;
0374
0375   if (!token) throw new Error('You have been signed out.
Please sign in again and retry.');
```

```

0376
0377   const contentType = 'application/pdf';
0378   const fileBlob = new Blob([stampedBytes as BlobPart],
{ type: contentType });
0379   // Upload directly to B2 so burst uploads do not occupy
Vercel memory or
0380   // pass the document body through a serverless
function. The presign route
0381   // remains lightweight and each file retries with backoff
independently.
0382   yield { phase: 'uploading', progress: 50, message:
'Uploading...' };
0383   let uploadResponse: { ok: boolean; status: number;
body?: string } | null = null;
0384   let lastUploadError: any = null;
0385   for (let attempt = 0; attempt < 3; attempt += 1) {
0386     try {
0387       const presignResponse = await withTimeout(
```

```

0388       fetch('/api/storage/presign', {
0389         method: 'POST',
0390         headers: {
0391           'Authorization': `Bearer ${token}`,
0392           'Content-Type': 'application/json'
0393         },
0394         body: JSON.stringify({ key: filePath,
contentType, intent: 'upload' })
0395       }),
0396       30000,
0397       'Pre-signed URL request timed out.'
0398     );
0399
0400     if (!presignResponse.ok) {
0401       if (presignResponse.status === 401 ||
presignResponse.status === 403) {
0402         throw new Error('You have been signed out.
Please sign in again and retry.');
```

```

0403       }
0404       throw new Error('Pre-signed URL failed
(${presignResponse.status})');
0405     }
0406
0407     const { url: presignedUrl } = await
presignResponse.json();
0408     uploadResponse = await xhrUpload(presignedUrl,
fileBlob, {
0409       method: 'PUT',
0410       headers: { 'Content-Type': contentType },
0411       onProgress: options.onTransferProgress
0412     });
0413
0414     if (uploadResponse.ok) break;
0415     throw new Error('B2 upload failed
(${uploadResponse.status})');
0416   } catch (err: any) {
0417     lastUploadError = err;
0418     if (attempt < 2) {
0419       const delay = 750 * 2 ** attempt +
Math.floor(Math.random() * 250);
0420       await new Promise((resolve) =>
setTimeout(resolve, delay));
0421     }
0422   }
0423 }
0424
0425 if (!uploadResponse?.ok) {
0426   const message = lastUploadError?.message ||
'Upload could not be completed.';
0427   if (message.includes('CORS') ||
message.includes('Access')) {
0428     throw new Error('Could not reach the storage
service. Please try again.');
```

```

0429   }
0430   throw new Error('Upload could not be completed
after 3 attempts. Please try again.');
```

```

0431 }
0432
0433 yield { phase: 'uploading', progress: 80, message:
'Finishing up...' };
0434
0435 // Detect whether this is an ADDITIONAL DLL for an
already-covered slot
0436 // (same teaching load + week + doc type). If so, mark it
'supplementary'
0437 // so it is archived but does not affect compliance /
missing / late / rate.
0438 let complianceStatus: 'compliant' | 'late' |
```

```

'supplementary' = calculateComplianceStatus(new Date(),
deadlin
0439   if (options.teachingLoadId && activeWeekNumber) {
0440     try {
0441       const { data: slotMatch } = await withTimeout(
0442         supabase
0443           .from('submissions')
0444           .select('id')
0445           .eq('teaching_load_id', options.teachingLoadId)
0446           .eq('week_number', activeWeekNumber)
0447           .eq('doc_type', activeDocType)
0448           .limit(1) as any,
0449         30000,
0450         'Duplicate slot check timed out.'
0451       ) as { data: any };
0452       if (slotMatch && slotMatch.length > 0) {
0453         complianceStatus = 'supplementary';
0454       }
0455     } catch (e) {
0456       console.warn('[pipeline] Duplicate slot check failed,
continuing as normal:', e);
0457     }
0458   }
0459   const { error: dbError } = await withTimeout(
0460     supabase.from('submissions').insert({
0461       user_id: options.userId,
0462       file_name: fileName,
0463       file_path: filePath,
0464       file_hash: fileHash,
0465       file_size: stampedBytes.byteLength,
0466       doc_type: activeDocType,
0467       week_number: activeWeekNumber,
0468       school_year: options.schoolYear ||
getCurrentSchoolYear(),
0469       subject: options.subject,
0470       calendar_id: calendarId,
0471       teaching_load_id: options.teachingLoadId || null,
0472       compliance_status: complianceStatus,
0473       raw_text: rawText || null
0474     }) as any,
0475     30000,
0476     'Database record timed out.'
0477   ) as { error: any };
0478   // 23505 = unique_violation on file_hash — the
backstop for the same
0479   // document being submitted twice when the pre-checks
above couldn't reach
0480   // the server. Note this cannot fire for an additional DLL
on an existing
0481   // week/subject: that's a different file, so a different hash,
and it is
0482   // archived as supplementary.
0483   if (dbError) {
0484     if (dbError.code === '23505') {
0485       throw new Error('This exact document has already
been uploaded. Choose a different file, or check My
0486     }
0487     throw new Error(`DB Error: ${dbError.message}`);
0488   }
0489
0490   // Success bookkeeping
0491   if (options.teachingLoadId && activeWeekNumber) {
0492     await recordSubmission({
0493       teachingLoadId: options.teachingLoadId,
0494       weekNumber: activeWeekNumber,
0495       schoolYear: options.schoolYear ||
getCurrentSchoolYear(),
0496       docType: activeDocType,

```

```

0497       fileHash,
0498       fileName,
0499       timestamp: Date.now(),
0500       status: 'syncd'
0501     });
0502   }
0503   await cacheVerifiedDoc(fileHash, { file_name: fileName,
doc_type: activeDocType, week_number: activeWeekNumb
0504
0505   // Same reasoning as the queued path: the file is
uploaded and the row is
0506   // inserted by this point, so a convenience notification —
two more un-timed
0507   // network calls — must not hold back the success the
teacher is waiting on.
0508   createNotification(options.userId, 'Archival Successful',
`Securely archived ${activeDocType} - Week ${activ
0509     .catch((e) => console.warn('[pipeline] Notification
write failed:', e));
0510
0511   yield {
0512     phase: 'done',
0513     progress: 100,
0514     message: 'Upload Successful!',
0515     result: { fileHash, filePath, fileSize:
stampedBytes.byteLength, fileName }
0516   };
0517 }
0518
0519 async function* runOfflinePipelineResilient(
0520   core: CoreResult,

```

## OCR Metadata Extraction

### src/lib/utis/ocr.ts Lines 1-260

```

0001 /**
0002  * OCR Metadata Extraction — Tesseract.js (Client-Side)
0003  * Dynamically imported to avoid blocking initial bundle.
0004  * Scans the first page ROI to extract document type,
week, and school year.
0005  */
0006 import { predictSubject, predictGradeLevel,
predictDocType } from './fuzzyClassifier';
0007 import { createWorker } from 'tesseract.js';
0008 import { PDFJS_VERSION } from './pdfjsVersion';
0009
0010 export interface DateRange {
0011   start: Date;
0012   end: Date;
0013   raw: string;
0014 }
0015
0016 export interface DocMetadata {
0017   docType: 'DLL' | 'ISP' | 'ISR' | 'Unknown';
0018   weekNumber: number | null;
0019   schoolYear: string | null;
0020   subject: string | null;
0021   subjectConfidence?: number;
0022   gradeLevel: string | null;
0023   rawText: string;
0024   confidence: number;
0025   language: 'English' | 'Filipino' | 'Unknown';
0026   school: string | null;
0027   teacher: string | null;
0028   date: string | null;
0029   dateRange: DateRange | null;
0030   weekSource?: 'calendar' | 'header-date' | 'regex' | 'none';
0031 }
0032

```

```

0033 // pdf.min.js is a few hundred KB. On the broadband
desktops were tested on,
0034 // 10s was never a real constraint — but at the mobile
data speeds this app
0035 // actually gets used on in the field (screenshots as low as
~2 KB/s), a
0036 // several-hundred-KB script can take well over a minute,
so the old 10s
0037 // timeout was guaranteed to fire on exactly the
connections this matters
0038 // most for, silently degrading every PDF upload to
"Unknown" metadata
0039 // detection instead of just taking longer to load.
0040 function loadPdfJs(timeoutMs: number = 45000):
Promise<void> {
0041   return new Promise((resolve, reject) => {
0042     if ((window as any)['pdfjsLib']) {
0043       resolve();
0044       return;
0045     }
0046     let settled = false;
0047     const timer = setTimeout(() => {
0048       if (settled) return;
0049       settled = true;
0050       reject(new Error("Timed out loading PDF.js from
CDN"));
0051     }, timeoutMs);
0052     const cleanup = () => clearTimeout(timer);
0053     const script = document.createElement('script');
0054     script.src =
'https://cdnjs.cloudflare.com/ajax/libs/pdf.js/${PDFJS_VERSION}
/pdf.min.js';
0055     script.onload = () => {
0056       if (settled) return;
0057       settled = true;
0058       cleanup();
0059       (window as
any).pdfjsLib.GlobalWorkerOptions.workerSrc =
0060       'https://cdnjs.cloudflare.com/ajax/libs/pdf.js/${PDFJS_VERSION}
/pdf.worker.min.js';
0061       resolve();
0062     };
0063     script.onerror = () => {
0064       if (settled) return;
0065       settled = true;
0066       cleanup();
0067       reject(new Error("Failed to load PDF.js from
CDN"));
0068     };
0069     document.head.appendChild(script);
0070   });
0071 }
0072
0073 export async function extractMetadata(file: File):
Promise<DocMetadata> {
0074   const ext = file.name.split('.').pop()?.toLowerCase();
0075
0076   // PDF path: extract text directly, falling back to raster
OCR if needed.
0077   if (file.type === 'application/pdf' || ext === 'pdf') {
0078     try {
0079       let pdfjsLib = (window as any)['pdfjsLib'];
0080       if (!pdfjsLib) {
0081         console.warn('[ocr] PDF.js not loaded. Loading
from CDN...');
0082         await loadPdfJs();
0083         pdfjsLib = (window as any)['pdfjsLib'];

```

```

0084         if (!pdfjsLib) {
0085           console.warn('[ocr] Failed to load PDF.js.
Using default metadata.');
```

```

0086           return createDefaultMetadata();
0087         }
0088       }
0089
0090       // Set worker source for pdf.js (crucial for some
environments)
0091       if (!pdfjsLib.GlobalWorkerOptions.workerSrc) {
0092         const version = pdfjsLib.version || '3.11.174';
0093
0094         // Use the CDN if online (likely already cached
by the browser), otherwise
0095         // fall back to /pdf.worker.min.js locally (only
works if the user added it manually).
0096         if (navigator.onLine) {
0097           pdfjsLib.GlobalWorkerOptions.workerSrc =
'https://cdnjs.cloudflare.com/ajax/libs/pdf.js/${ve
0098           } else {
0099             pdfjsLib.GlobalWorkerOptions.workerSrc =
'/pdf.worker.min.js';
0100           }
0101         }
0102
0103         const arrayBuffer = await file.arrayBuffer();
0104         const pdf = await pdfjsLib.getDocument({ data:
arrayBuffer }).promise;
0105         const page = await pdf.getPage(1);
0106         const textContent = await page.getTextContent();
0107         const text = textContent.items.map((item: any) =>
item.str).join(' ');
0108         console.log(`[ocr] PDF text extracted (${text.length}
chars):`, text.substring(0, 200) + '...');
0109
0110         // If text is too short, fallback to raster OCR
0111         if (text.length < 50) {
0112           console.warn('[ocr] PDF text too sparse. Falling
back to raster OCR...');
0113           return await extractRasterMetadata(page,
pdfjsLib);
0114         }
0115
0116         const metadata = parseMetadata(text);
0117         return { ...metadata, rawText: text, confidence: 100,
language: 'Unknown' };
0118       } catch (err) {
0119         console.error('[ocr] PDF extraction failed:', err);
0120         return createDefaultMetadata();
0121       }
0122     }
0123
0124     // Image path: OCR via Tesseract (multilingual).
0125     if (!file.type.startsWith('image/')) {
0126       return createDefaultMetadata();
0127     }
0128
0129     const objectUrl = URL.createObjectURL(file);
0130
0131     try {
0132       // Tesseract.js automatically caches the worker and
language data in IndexedDB,
0133       // so as long as it runs once while online, it keeps
working offline.
0134       const worker = await createWorker('eng+fil', 1, {
0135         logger: (m: any) => console.log(m),
0136         errorHandler: (err: any) => console.error('[ocr]
Tesseract Worker Error:', err)

```

```

0137     });
0138
0139     try {
0140         // Manual thresholding for low-end mobile CPUs
0141         const { data: { text, confidence } } = await
worker.recognize(objectUrl);
0142
0143         // Post-process language detection from result
0144         const language = detectLanguage(text);
0145         const metadata = parseMetadata(text);
0146
0147         console.log('[ocr] Optimized single-pass OCR
results (Language: ${language}, Confidence: ${confidenc
0148         return { ...metadata, confidence, language };
0149     } finally {
0150         await worker.terminate();
0151     }
0152 } catch (err) {
0153     console.error('[ocr] Optimized Image OCR failed:',
err);
0154     return createDefaultMetadata();
0155 } finally {
0156     URL.revokeObjectURL(objectUrl);
0157 }
0158 }
0159
0160 function createDefaultMetadata(): DocMetadata {
0161     return {
0162         docType: 'Unknown',
0163         weekNumber: null,
0164         schoolYear: null,
0165         subject: null,
0166         subjectConfidence: 0,
0167         gradeLevel: null,
0168         rawText: '',
0169         confidence: 0,
0170         language: 'Unknown',
0171         school: null,
0172         teacher: null,
0173         date: null,
0174         dateRange: null,
0175         weekSource: 'none'
0176     };
0177 }
0178
0179 async function extractRasterMetadata(page: any, pdfjsLib:
any): Promise<DocMetadata> {
0180     try {
0181         console.log('[ocr] Starting raster OCR fallback...');
0182
0183         // Force a lower render scale on mobile to save
CPU/RAM.
0184         const isMobile =
/iPhone|iPad|iPod|Android/i.test(navigator.userAgent);
0185         const scale = isMobile ? 1.0 : 2.5;
0186
0187         const viewport = page.getViewport({ scale });
0188         const canvas = document.createElement('canvas');
0189         const context = canvas.getContext('2d');
0190         if (!context) throw new Error('Could not get canvas
context');
0191
0192         canvas.height = viewport.height;
0193         canvas.width = viewport.width;
0194
0195         await page.render({ canvasContext: context,
viewport }).promise;
0196
0197         // Convert to grayscale and threshold for low-end
CPUs; this makes Tesseract's job much easier and faster
0198         if (isMobile) {
0199             const imageData = context.getImageData(0, 0,
canvas.width, canvas.height);
0200             const data = imageData.data;
0201             for (let i = 0; i < data.length; i += 4) {
0202                 const avg = (data[i] + data[i + 1] + data[i + 2]) / 3;
0203                 // Thresholding: Black or White only (Binary)
0204                 const val = avg > 128 ? 255 : 0;
0205                 data[i] = data[i + 1] = data[i + 2] = val;
0206                 data[i + 3] = 255; // Ensure alpha is solid
0207             }
0208             context.putImageData(imageData, 0, 0);
0209         }
0210
0211         return new Promise<DocMetadata>((resolve, reject)
=> {
0212             canvas.toBlob(async (blob) => {
0213                 if (!blob) {
0214                     reject(new Error('Canvas toBlob failed'));
0215                     return;
0216                 }
0217                 const blobUrl = URL.createObjectURL(blob);
0218                 const { createWorker } = await
import('tesseract.js');
0219                 const worker = await createWorker('eng+fil');
0220
0221                 try {
0222                     const { data: { text, confidence } } = await
worker.recognize(blobUrl);
0223                     console.log('[ocr] Raster OCR complete
(${text.length} chars), scale: ${scale}, confidence: '
0224                     const metadata = parseMetadata(text);
0225                     resolve({ ...metadata, rawText: text,
confidence, language: detectLanguage(text) });
0226                 } catch (e) {
0227                     reject(e);
0228                 } finally {
0229                     await worker.terminate();
0230                     URL.revokeObjectURL(blobUrl);
0231                 }
0232             }, 'image/png');
0233         });
0234     } catch (err) {
0235         console.error('[ocr] Raster fallback failed:', err);
0236         return createDefaultMetadata();
0237     }
0238 }
0239
0240 function detectLanguage(text: string): 'English' | 'Filipino' |
'Unknown' {
0241     const upper = text.toUpperCase();
0242
0243     // Filipino/Tagalog indicators
0244     const filipinoPatterns = [
0245         /PAARALAN|GURO|PETA|ORAS|BAITANG|ASIGNATURA|M
ARKAHAN|LUNES|MARTES|MIYERKULES|HUWEBES|BIYER
NES/,
0246         /NILALAMAN|PAMANTAYAN|KASANAYAN|KURIKULUM|LAYU
NIN|GAWAIN/,
0247         /EDUKASYON|s*SA*s*PAGPAPAKATAO|ARALING*s*PANLIPU
NAN|PILIPINAS/
0248     ];
0249

```

```

0250 // English indicators
0251 const englishPatterns = [
0252 /SCHOOL|TEACHER|DATE|TIME|GRADE|SUBJECT|MARK|M
0253 ONDAY|TUESDAY|WEDNESDAY|THURSDAY|FRIDAY/,
0254 /CONTENT|STANDARDS|COMPETENCIES|ACTIVITIES|ASSE
0255 SMENT|RESOURCES/,
0256 /LEARNING\s*OBJECTIVES|PERFORMANCE|QUARTER|WEE
0257 K/
0258 ];
0259 let filipinoScore = 0;
0260 let englishScore = 0;
0261 for (const pattern of filipinoPatterns) {

```

## OCR Parsing and Date Handling

src/lib/utls/ocr.ts Lines 260-520

```

0260 for (const pattern of filipinoPatterns) {
0261   const matches = text.match(pattern);
0262   filipinoScore += matches ? matches.length : 0;
0263 }
0264 for (const pattern of englishPatterns) {
0265   const matches = text.match(pattern);
0266   englishScore += matches ? matches.length : 0;
0267 }
0268 if (filipinoScore > englishScore && filipinoScore > 0)
0269   return 'Filipino';
0270 if (englishScore > filipinoScore && englishScore > 0)
0271   return 'English';
0272 return 'Unknown';
0273 }
0274
0275 export function parseMetadata(text: string):
0276   Omit<DocMetadata, 'confidence' | 'language'> {
0277   const upper = text.toUpperCase();
0278   // Helper to convert Roman numerals to numbers
0279   const romanToNum = (roman: string): string => {
0280     const map: Record<string, string> = {
0281       'I': '1', 'II': '2', 'III': '3', 'IV': '4', 'V': '5', 'VI': '6',
0282       'VII': '7', 'VIII': '8', 'IX': '9', 'X': '10', 'XI': '11', 'XII': '12'
0283     };
0284     return map[roman] || roman;
0285   };
0286   // Detect document type
0287   let docType: DocMetadata['docType'] = 'Unknown';
0288   // Tier 1: Regex-based detection (highest confidence)
0289   if (/DAILY\s*LESSON\s*(LOG|PLAN)|D\.\?L\.\?|DETALYADON
0290     G\s*PLANO|ARAW-
0291     ARAW\s*LEKSYON|LINGGUHANG\s*ARALIN|BA
0292     (/INSTRUCTIONAL\s*SUPERVISORY\s*PLAN|I\.\?S\.\?P\.\?|SUP
0293     ERVISORY\s*PLAN/i.test(upper)) {
0294     docType = 'ISP';
0295   } else if (/INSTRUCTIONAL\s*SUPERVISORY\s*REPORT|I\.\?S\.\?R\.\?|S
0296     UPERVISORY\s*REPORT/i.test(upper)) {
0297     docType = 'ISR';
0298   }

```

```

0298 // Tier 2: Fuzzy fallback (handles OCR typos like "DALY
0299 LESSON LOG")
0300 if (docType === 'Unknown') {
0301   const fuzzyDt = predictDocType(text);
0302   if (fuzzyDt.value) {
0303     docType = fuzzyDt.value as 'DLL' | 'ISP' | 'ISR';
0304     console.log(`[ocr] Fuzzy matched doc type as
0305     ${docType} (confidence: ${fuzzyDt.confidence}%)`);
0306   }
0307   // For ISP/ISR documents, skip subject/grade/week
0308   // detection — they are supervisory docs
0309   let subject: string | null = null;
0310   let subjectConfidence = 0;
0311   let gradeLevel: string | null = null;
0312   let weekNumber: number | null = null;
0313   let weekSource: 'calendar' | 'header-date' | 'regex' |
0314     'none' = 'none';
0315   if (docType === 'DLL') {
0316     const explicitSubjectMatch =
0317       upper.match(/(?:ASIGNATURA|SUBJECT|LEARNING\s*AREA(?:
0318         :\s*[V]\s*ASIGNATURA)?
0319       if (explicitSubjectMatch) {
0320         const candidate = explicitSubjectMatch[1].trim();
0321         const subjectPatterns = buildSubjectPatterns(true);
0322         for (const p of subjectPatterns) {
0323           if (p.regex.test(candidate)) {
0324             subject = p.name;
0325             subjectConfidence = 100;
0326             break;
0327           }
0328         }
0329       }
0330       if (!subject) {
0331         const fuzzy = predictSubject(text);
0332         subject = fuzzy.value;
0333         subjectConfidence = fuzzy.confidence;
0334         if (subject) console.log(`[ocr] Fuzzy matched
0335         subject as ${subject} (confidence: ${fuzzy.confidence})
0336       }
0337       if (!subject) {
0338         const subjectPatterns =
0339           buildSubjectPatterns(false);
0340         for (const p of subjectPatterns) {
0341           if (p.regex.test(upper)) {
0342             subject = p.name;
0343             break;
0344           }
0345         }
0346       }
0347       // Detect Grade/Level
0348       const gradeMatch =
0349         upper.match(/(?:GRADE|GR\.\?|BAITANG|ANTAS)\s*[/]?[?]\s*(?:LE
0350         VEL|ANTAS)?\s*[:]\s*([IVX
0351         gradeLevel = gradeMatch ? gradeMatch[1].trim() :
0352         null;
0353         if (gradeLevel && isNaN(parseInt(gradeLevel))) {
0354           gradeLevel = romanToNum(gradeLevel);
0355         }
0356       }
0357       if (!gradeLevel) {

```

```

0355     const fuzzy = predictGradeLevel(text);
0356     if (fuzzy.value) {
0357         gradeLevel = fuzzy.value;
0358         console.log('[ocr] Fuzzy matched grade as
${gradeLevel} (confidence: ${fuzzy.confidence}%);
0359     }
0360 }
0361
0362 // Extract week number
0363 const weekMatch =
upper.match(/(?:WEEK|LINGGO)\s*[\#.]*\s*(\d+)/i) ||
0364     upper.match(/IKA-\s*(\d+)\s*LINGGO/i);
0365     weekNumber = weekMatch ? parseInt(weekMatch[1],
10) : null;
0366     weekSource = weekNumber ? 'regex' : 'none';
0367 }
0368
0369 // Extract school year
0370 const syMatch = text.match(/S\.\?Y\.\?s*(\d{4})s*[-
]\s*\d{4}/i) ||
0371     text.match(/(\d{4})s*[-]\s*\d{4}/i);
0372     const schoolYear = syMatch ? syMatch[1].replace(/s/g,
''); : null;
0373
0374 // Extract school name (Filipino: Paaralan, English:
School)
0375     const schoolMatch =
text.match(/(?:SCHOOL|PAARALAN)\s*[:\t]*s*([\^n\t]+)/i);
0376     const school = schoolMatch ? schoolMatch[1].trim() :
null;
0377
0378 // Extract teacher name (Filipino: Guro, English:
Teacher)
0379     const teacherMatch =
text.match(/(?:TEACHER|GURO|EDUCATOR)\s*[:\t]*s*([\^n\t]+)/
i);
0380     const teacher = teacherMatch ? teacherMatch[1].trim() :
null;
0381
0382 // Extract date string (Filipino: Petsa, English: Date)
0383     const dateMatch =
text.match(/(?:Teaching\s*Dates(?:\s*and\s*Time)?|DATE|PET
SA(?:\s*(?:at|[/])\s*(?:ORAS|D
0384     const date = dateMatch ? dateMatch[1].trim() : null;
0385
0386 // Tier 0: Parse date range from header for calendar-
based resolution
0387     const dateRange = parseDateRange(text);
0388     if (dateRange && !weekNumber) {
0389         weekSource = 'header-date';
0390         console.log('[ocr] Parsed date range:
${dateRange.start.toDateString()} -
${dateRange.end.toDateString()}
0391     }
0392
0393     console.log('[ocr] Extracted date raw:', dateMatch ?
dateMatch[0] : 'NONE');
0394     console.log('[ocr] Week detection source:',
weekSource);
0395
0396     return {
0397         docType,
0398         weekNumber,
0399         schoolYear,
0400         subject,
0401         subjectConfidence,
0402         gradeLevel,
0403         rawText: text,
0404         school,
0405         teacher,
0406         date,
0407         dateRange,
0408         weekSource
0409     };
0410 }
0411
0412 /**
0413  * Build subject regex patterns (shared between header
regex and full-text fallback).
0414  */
0415     function buildSubjectPatterns(_forHeader: boolean):
{ name: string; regex: RegExp }[] {
0416         return [
0417             { name: 'Makabansa', regex: /MAKABANSA/i },
0418             { name: 'English', regex:
/ENGLISH|(?<!WIKANG\s)WIKA(?:!NG\s*PILIPINO)/i },
0419             { name: 'Filipino', regex:
/FILIPINO|TAGALOG|WIKANG\s*PILIPINO/i },
0420             { name: 'Mathematics', regex:
/MATHEMATICS|MATH|MATEMATIKA/i },
0421             { name: 'Science', regex: /SCIENCE|AGHAM/i },
0422             { name: 'AP', regex:
/ARALING\s*PANLIPUNAN|bAP|b|SOCIAL\s*STUDIES/i },
0423             { name: 'GMRC', regex:
/EDUKASYON\s*SA\s*PAGPAPAKATAO|E\.\?S\.\?P\.\?|GMRC|V
ALUES|MORAL/i },
0424             { name: 'MAPEH', regex:
/MAPEH|ARTS|MUSIC|PHYSICAL|HEALTH|PE/i },
0425             { name: 'EPP', regex:
/E\.\?P\.\?P\.\?|EDUKASYON.*PRODUKTIBO|VOCATIONAL/i },
0426             { name: 'TLE', regex:
/T\.\?L\.\?E\.\?|TECHNOLOGY.*LIVELIHOOD/i },
0427             { name: 'Reading and Literacy', regex:
/READING\s*(?:AND|&)\s*LITERACY/i },
0428             { name: 'Language', regex:
/(?<!READING\s+(?:AND|&)\s+)+LANGUAGE(?:!s+(?:AND|&)\s+
LITERACY|LITERACY)/i },
0429             { name: 'Numeracy', regex: /NUMERACY/i },
0430             { name: 'Language Literacy and Communication',
regex:
/LANGUAGE\s+LITERACY\s+AND\s+COMMUNICATION/i },
0431             { name: 'Arts and Music', regex:
/ARTS\s+(?:AND|&)\s+MUSIC/i },
0432             { name: 'Physical Development', regex:
/PHYSICAL\s+DEVELOPMENT/i },
0433         ];
0434     }
0435
0436     const MONTH_MAP: Record<string, number> = {
0437         // English
0438         JANUARY: 0, FEBRUARY: 1, MARCH: 2, APRIL: 3,
MAY: 4, JUNE: 5,
0439         JULY: 6, AUGUST: 7, SEPTEMBER: 8, OCTOBER: 9,
NOVEMBER: 10, DECEMBER: 11,
0440         JAN: 0, FEB: 1, MAR: 2, APR: 3, JUN: 5, JUL: 6, AUG:
7, SEP: 8, OCT: 9, NOV: 10, DEC: 11,
0441         // Filipino
0442         ENERO: 0, PEBRERO: 1, MARSO: 2, ABRIL: 3, MAYO:
4, HUNYO: 5,
0443         HULYO: 6, AGOSTO: 7, SETYEMBRE: 8, OKTUBRE:
9, NOBYEMBRE: 10, DISYEMBRE: 11
0444     };
0445
0446     /**
0447      * Parse date ranges from DLL/ISP headers.
0448      * Handles formats like:

```

```

0449 * "FEBRUARY 9 - 13, 2026"
0450 * "ENERO 15 - 19, 2026"
0451 * "FEBRUARY 25 - 29, 2026"
0452 * "March 30 - April 3, 2026"
0453 * "FEBRUARY 9-13, 2026"
0454 */
0455 export function parseDateRange(text: string): DateRange
| null {
0456   const upper = text.toUpperCase();
0457
0458   // Build month name regex from our map
0459   const monthNames =
Object.keys(MONTH_MAP).join('|');
0460
0461   // Pattern 1: "MONTH DAY - DAY, YEAR" (same month)
0462   const sameMonth = new RegExp(
0463     `(${monthNames})\\s+(\\d{1,2})\\s*[-
]\\s*(\\d{1,2})[,\\s]+(\\d{4})`,
0464     'i'
0465   );
0466
0467   // Pattern 2: "MONTH DAY - MONTH DAY, YEAR"
(cross-month)
0468   const crossMonth = new RegExp(
0469     `(${monthNames})\\s+(\\d{1,2})\\s*[-
]\\s*(${monthNames})\\s+(\\d{1,2})[,\\s]+(\\d{4})`,
0470     'i'
0471   );
0472
0473   // Try cross-month first (more specific)
0474   let match = upper.match(crossMonth);
0475   if (match) {
0476     const startMonth = MONTH_MAP[match[1]];
0477     const startDay = parseInt(match[2]);
0478     const endMonth = MONTH_MAP[match[3]];
0479     const endDay = parseInt(match[4]);
0480     const year = parseInt(match[5]);
0481
0482     if (startMonth !== undefined && endMonth !==
undefined && !isNaN(year)) {
0483       const start = new Date(year, startMonth, startDay);
0484       const end = new Date(year, endMonth, endDay);
0485       if (!isNaN(start.getTime())
&& !isNaN(end.getTime())) {
0486         return { start, end, raw: match[0] };
0487       }
0488     }
0489   }
0490
0491   // Try same-month pattern
0492   match = upper.match(sameMonth);
0493   if (match) {
0494     const month = MONTH_MAP[match[1]];
0495     const startDay = parseInt(match[2]);
0496     const endDay = parseInt(match[3]);
0497     const year = parseInt(match[4]);
0498
0499     if (month !== undefined && !isNaN(year)) {
0500       const start = new Date(year, month, startDay);
0501       const end = new Date(year, month, endDay);
0502       if (!isNaN(start.getTime())
&& !isNaN(end.getTime())) {
0503         return { start, end, raw: match[0] };
0504       }
0505     }
0506   }
0507
0508   return null;

```

```

0509 }
0510
0511 /**
0512 * Resolve a parsed date range to an academic calendar
week.
0513 * Compares the document's date range against the
calendar's start/end dates.
0514 * Returns the matching week number and calendar ID, or
null if no match.
0515 */
0516 export function resolveWeekFromDates(
0517   dateRange: DateRange,
0518   calendar: { id?: string; week_number: number;
deadline_date?: string }[]
0519 ): { weekNumber: number; calendarId?: string } | null {
0520   if (!calendar || calendar.length === 0) return null;

```

## Fuzzy Classifier

src/lib/utils/fuzzyClassifier.ts Lines 1-220

```

0001 /**
0002 * Fuzzy String Matching Classifier — replaces Naive
Bayes for header extraction
0003 *
0004 * Uses Dice coefficient (bigram overlap) for fuzzy
matching against known values.
0005 * No training needed, zero model files, handles OCR
typos naturally.
0006 */
0007
0008 function diceCoefficient(a: string, b: string): number {
0009   const bigrams = new Map<string, number>();
0010   for (let i = 0; i < a.length - 1; i++) {
0011     const bg = a.substring(i, i + 2);
0012     bigrams.set(bg, (bigrams.get(bg) || 0) + 1);
0013   }
0014   let intersection = 0;
0015   for (let i = 0; i < b.length - 1; i++) {
0016     const bg = b.substring(i, i + 2);
0017     const count = bigrams.get(bg) || 0;
0018     if (count > 0) {
0019       bigrams.set(bg, count - 1);
0020       intersection++;
0021     }
0022   }
0023   const total = a.length + b.length - 2;
0024   return total === 0 ? 0 : (2 * intersection) / total;
0025 }
0026
0027 function normalize(s: string): string {
0028   return s.toLowerCase().replace(/[^a-z0-9\s]/g,
'').replace(/s+/g, ' ').trim();
0029 }
0030
0031 const SUBJECTS = [
0032   'GMRC', 'Mathematics', 'Math', 'Filipino', 'MAPEH',
'English',
0033   'Science', 'Makabansa', 'AP', 'EPP', 'Language',
0034   'Reading and Literacy', 'Numeracy', 'Arts and Music',
0035   'Physical Development', 'Language Literacy and
Communication',
0036   'TLE'
0037 ];
0038
0039 const GRADE_LEVELS: string[] = [];
0040 for (let g = 1; g <= 6; g++) {
0041   GRADE_LEVELS.push(`Grade ${g}`);
0042 }
0043

```

```

0044 const DOC_TYPES = ['DLL', 'ISP', 'ISR'];
0045
0046 const FIELD_LABELS = [
0047     'Learning Area', 'Asignatura', 'Subject',
0048     'Grade Level', 'Baitang', 'Grade',
0049     'Term', 'Week', 'Linggo',
0050     'Teacher', 'Guro',
0051     'School', 'Paaralan',
0052     'Date', 'Petsa'
0053 ];
0054
0055 function bestMatch(input: string, candidates: string[],
threshold: number = 0.3): { value: string | null; score:
0056     const norm = normalize(input);
0057     if (!norm) return { value: null, score: 0 };
0058
0059     let best: string | null = null;
0060     let bestScore = 0;
0061
0062     for (const candidate of candidates) {
0063         const score = diceCoefficient(norm,
normalize(candidate));
0064         if (score > bestScore) {
0065             bestScore = score;
0066             best = candidate;
0067         }
0068     }
0069
0070     return {
0071         value: bestScore >= threshold ? best : null,
0072         score: Math.round(bestScore * 100)
0073     };
0074 }
0075
0076 function fuzzyMatchAny(input: string, candidates: string[]):
{ value: string | null; score: number } {
0077     const norm = normalize(input);
0078     if (!norm) return { value: null, score: 0 };
0079
0080     // Try to find the candidate as a substring first (exact
match for short strings like "GMRC 1")
0081     for (const candidate of candidates) {
0082         if (norm.includes(normalize(candidate))) {
0083             return { value: candidate, score: 100 };
0084         }
0085     }
0086
0087     // Otherwise fuzzy match the whole input against each
candidate
0088     let best: string | null = null;
0089     let bestScore = 0;
0090     for (const candidate of candidates) {
0091         const score = diceCoefficient(norm,
normalize(candidate));
0092         if (score > bestScore) {
0093             bestScore = score;
0094             best = candidate;
0095         }
0096     }
0097
0098     return {
0099         value: bestScore >= 0.25 ? best : null,
0100         score: Math.round(bestScore * 100)
0101     };
0102 }
0103
0104 export interface FuzzyPrediction {
0105     value: string | null;

```

```

0106     confidence: number;
0107 }
0108
0109 export function predictSubject(text: string):
FuzzyPrediction {
0110     // Look for an explicit header field first, falling back to a
fuzzy scan of the whole text.
0111     const header =
text.match(/(?:ASIGNATURA|SUBJECT|LEARNING\s*AREA(?:\s*
[V]\s*ASIGNATURA)?)\s*[:\t]*s*([^\n\
0112     if (header) {
0113         const candidate = header[1].trim();
0114         // Extract the subject name (it's usually the first token
before any number)
0115         const firstWord = candidate.split(/[s,]+/)[0];
0116         const result = fuzzyMatchAny(firstWord, SUBJECTS);
0117         if (result.value) return { value: result.value,
confidence: result.score };
0118     }
0119
0120     const result = fuzzyMatchAny(text, SUBJECTS);
0121     return result.value
? { value: result.value, confidence: result.score }
: { value: null, confidence: 0 };
0124 }
0125
0126 export function predictGradeLevel(text: string):
FuzzyPrediction {
0127     // Try an explicit "Grade Level"/"Baitang" field, then a
bare "GRADE X" pattern, then fuzzy matching.
0128     const gradeField =
text.match(/(?:GRADE\s*(?:LEVEL)?|BAITANG)\s*[:\t]*s*(\d+|[I
VX]+)/i);
0129     if (gradeField) {
0130         const num = parseInt(gradeField[1]);
0131         if (num >= 1 && num <= 6) return { value: `Grade
${num}`, confidence: 100 };
0132     }
0133
0134     const gradeAnywhere =
text.match(/bGRADE\s*(\d+)\b/i);
0135     if (gradeAnywhere) {
0136         const num = parseInt(gradeAnywhere[1]);
0137         if (num >= 1 && num <= 6) return { value: `Grade
${num}`, confidence: 100 };
0138     }
0139
0140     const result = fuzzyMatchAny(text, GRADE_LEVELS);
0141     return result.value
? { value: result.value, confidence: result.score }
: { value: null, confidence: 0 };
0144 }
0145
0146 export function predictDocType(text: string):
FuzzyPrediction {
0147     const upper = text.toUpperCase();
0148
0149     // Try deterministic regex patterns first, falling back to
fuzzy matching.
0150     if
(/DAILY\s*LESSON\s*(LOG|PLAN)|D\.\?L\.\?|DETALYADON
Gs*PLANO|ARAW-
ARAW\s*LEKSYON|LINGGUHANG\s*ARALIN|BA
0151     return { value: 'DLL', confidence: 100 };
0152 }
0153
if
(/INSTRUCTIONAL\s*SUPERVISORY\s*PLAN|I\.\?S\.\?P\.\?|SUP
ERVISORY\s*PLAN/i.test(upper)) {

```

```

0154     return { value: 'ISP', confidence: 100 };
0155   }
0156   if
(/INSTRUCTIONAL\s*SUPERVISORY\s*REPORT|I\.?S\.?R\.?|S
UPERVISORY\s*REPORT/i.test(upper)) {
0157     return { value: 'ISR', confidence: 100 };
0158   }
0159
0160   const result = fuzzyMatchAny(text, DOC_TYPES);
0161   return result.value
0162     ? { value: result.value, confidence: result.score }
0163     : { value: null, confidence: 0 };
0164 }
0165
0166 /**
0167  * Fuzzy match a field label in OCR text to extract the
value that follows it.
0168  * Handles OCR typos like "Le@rning Ar3a" matching
"Learning Area".
0169  */
0170 export function extractFieldValue(text: string, fieldName:
string): string | null {
0171   const norm = normalize(fieldName);
0172
0173   const lines = text.split('\n');
0174   let bestLine: string | null = null;
0175   let bestScore = 0;
0176
0177   for (const line of lines) {
0178     const colonIdx = line.indexOf(':');
0179     if (colonIdx < 0) continue;
0180
0181     const labelPart = line.substring(0, colonIdx).trim();
0182     const valuePart = line.substring(colonIdx + 1).trim();
0183
0184     const score = diceCoefficient(normalize(labelPart),
norm);
0185     if (score > bestScore) {
0186       bestScore = score;
0187       bestLine = valuePart;
0188     }
0189   }
0190
0191   return bestScore >= 0.4 ? bestLine : null;
0192 }

```

## Offline Cache and Queue

### src/lib/utils/offline.ts Lines 1-260

```

0001 /**
0002  * Offline Sync Queue — IndexedDB via idb-keyval
0003  * Persists pending uploads in the browser.
0004  * Auto-resumes when connectivity is restored.
0005  */
0006
0007 import { get, set, del, keys } from 'idb-keyval';
0008 import { writable } from 'svelte/store';
0009 import { supabase } from './supabase';
0010 import { withTimeout } from './pipeline';
0011 import { getCurrentSchoolYear } from './schoolYear';
0012 import { env } from '$env/dynamic/public';
0013
0014 export const pendingSyncCount = writable<number>(0);
0015
0016 const CACHE_PREFIX_DOCS = 'cached_docs_';
0017 const CACHE_PREFIX_HISTORY = 'cached_history_';
0018 const CACHE_PREFIX_METADATA =
'cached_metadata_';
0019

```

```

0020 /**
0021  * Cache a verified document hash locally for offline
verification.
0022  * Part of Phase 4.7 Smart Caching.
0023  */
0024 export async function cacheVerifiedDoc(hash: string,
metadata: any) {
0025   await set(`${CACHE_PREFIX_DOCS}${hash}`,
metadata);
0026 }
0027
0028 /**
0029  * Lookup a document hash in the local offline cache.
0030  */
0031 export async function lookupOfflineDoc(hash: string) {
0032   return await get(`${CACHE_PREFIX_DOCS}${hash}`);
0033 }
0034
0035 /**
0036  * Strip Svelte 5 state proxies (and anything else
structured clone can't
0037  * store) before handing data to IndexedDB, which
otherwise throws
0038  * DataCloneError on a raw $state proxy.
0039  */
0040 function toCloneable(data: any) {
0041   return data === undefined ? data :
JSON.parse(JSON.stringify(data));
0042 }
0043
0044 /**
0045  * Cache current dashboard data for offline viewing.
0046  */
0047 export async function cacheDashboardData(userId: string,
data: any) {
0048   await set(`${CACHE_PREFIX_HISTORY}${userId}`, {
0049     data: toCloneable(data),
0050     timestamp: Date.now()
0051   });
0052 }
0053
0054 /**
0055  * Retrieve cached dashboard data.
0056  */
0057 export async function getCachedDashboardData(userId:
string) {
0058   return await
get(`${CACHE_PREFIX_HISTORY}${userId}`);
0059 }
0060
0061 /**
0062  * Generic metadata caching for offline fallbacks.
0063  */
0064 export async function cacheMetadata(key: string, data:
any) {
0065   await set(`${CACHE_PREFIX_METADATA}${key}`, {
0066     data: toCloneable(data),
0067     timestamp: Date.now()
0068   });
0069 }
0070
0071 /**
0072  * Retrieve generic cached metadata.
0073  */
0074 export async function getCachedMetadata(key: string) {
0075   return await
get(`${CACHE_PREFIX_METADATA}${key}`);
0076 }

```

```

0077
0078 /**
0079  * WBS 20.4 — Pre-fetch essential metadata for offline
upload manual selection.
0080  * Fetches teaching loads and academic calendar,
caching them in IndexedDB.
0081  */
0082 export async function prefetchOfflineMetadata(userId:
string, districtId?: string): Promise<void> {
0083   if (!navigator.onLine) return;
0084
0085   try {
0086     console.log('[offline] Pre-fetching metadata for offline
upload...');
0087
0088     // 1. Fetch ALL Teaching Loads for the user
0089     const { data: loads, error: loadsErr } = await
supabase
0090       .from('teaching_loads')
0091       .select(`
0092         id, subject, grade_level, is_active,
0093         profiles:user_id ( full_name )
0094       `)
0095       .eq('user_id', userId)
0096       .eq('is_active', true);
0097
0098     if (!loadsErr && loads) {
0099       await cacheMetadata(`teaching_loads_${userId}`,
loads);
0100       console.log(`[offline] Cached ${loads.length}
teaching loads`);
0101     }
0102
0103     // 2. Fetch ALL Academic Calendar weeks for the
district (for week resolution)
0104     let calendarData: any[] = [];
0105     if (districtId) {
0106       const { data: calendar, error: calErr } = await
supabase
0107         .from('academic_calendar')
0108         .select(`id, week_number, deadline_date,
description, is_active`)
0109         .eq('district_id', districtId)
0110         .order('week_number', { ascending: true });
0111
0112       if (!calErr && calendar && calendar.length > 0) {
0113         calendarData = calendar;
0114         await cacheMetadata(`calendar_${districtId}`,
calendar);
0115         await
cacheMetadata(`calendar_open_${districtId}`, calendar);
0116         console.log(`[offline] Cached ${calendar.length}
calendar weeks for district ${districtId}`);
0117       }
0118     }
0119
0120     // Fallback: If no district calendar found, fetch
global/common
0121     if (calendarData.length === 0) {
0122       const { data: globalCal, error: globalCalErr } =
await supabase
0123         .from('academic_calendar')
0124         .select(`id, week_number, deadline_date,
description, is_active`)
0125         .order('week_number', { ascending: true });
0126
0127       if (!globalCalErr && globalCal) {
0128         calendarData = globalCal;

```

```

0129         await cacheMetadata('calendar_all', globalCal);
0130         // Also cache as district-specific if possible to
avoid redundant fetches
0131         if (districtId) {
0132           await cacheMetadata(`calendar_${districtId}`,
globalCal);
0133           await
cacheMetadata(`calendar_open_${districtId}`, globalCal);
0134         }
0135         console.log(`[offline] Cached ${globalCal.length}
global calendar weeks (fallback)`);
0136       }
0137     }
0138
0139     // 3. Fetch Submission History (for Copilot context)
0140     const { data: subs, error: subsErr } = await supabase
0141       .from('submissions')
0142       .select(`teaching_load_id, week_number, doc_type,
compliance_status, created_at`)
0143       .eq('user_id', userId)
0144       .eq('school_year', getCurrentSchoolYear())
0145       .order('created_at', { ascending: false })
0146       .limit(100);
0147
0148     if (!subsErr && subs) {
0149       await
cacheMetadata(`submission_history_${userId}`, subs);
0150       console.log(`[offline] Cached ${subs.length}
submission history records`);
0151     }
0152
0153     // 4. Cache current year settings
0154     const currentYear = getCurrentSchoolYear();
0155     await cacheMetadata('current_school_year',
currentYear);
0156
0157     console.log('[offline] Offline metadata pre-fetch
complete.');
```

```

0158   } catch (err) {
0159     console.warn('[offline] prefetchOfflineMetadata error:',
err);
0160   }
0161 }
0162
0163 /**
0164  * Check if the device has all necessary metadata cached
for offline upload.
0165  */
0166 export async function verifyOfflineReadiness(userId:
string, districtId?: string): Promise<{
0167   ready: boolean;
0168   missing: string[];
0169   counts: Record<string, number>;
0170 }> {
0171   const missing: string[] = [];
0172   const counts: Record<string, number> = {};
0173
0174   const loads = await
getCachedMetadata(`teaching_loads_${userId}`);
0175   if (!loads?.data || (loads.data as any[]).length === 0) {
0176     missing.push('Teaching Loads');
0177   } else {
0178     counts.loads = (loads.data as any[]).length;
0179   }
0180
0181   const calendar = districtId
0182     ? await getCachedMetadata(`calendar_${districtId}`)
0183     : await getCachedMetadata('calendar_all');
```

```

0184
0185   if (!calendar?.data || (calendar.data as any[]).length ===
0186   0) {
0187       missing.push('Academic Calendar');
0188   } else {
0189       counts.weeks = (calendar.data as any[]).length;
0190   }
0191   const history = await
getCachedMetadata(`submission_history_${userId}`);
0192   if (!history?.data || (history.data as any[]).length === 0) {
0193       // History is optional but recommended
0194   } else {
0195       counts.history = (history.data as any[]).length;
0196   }
0197
0198   return {
0199       ready: missing.length === 0,
0200       missing,
0201       counts
0202   };
0203 }
0204
0205 /**
0206  * WBS 20.3 — Pre-cache ALL verified document hashes
for full offline verification.
0207  *
0208  * Fetches all submission hashes from Supabase and
stores them in IndexedDB.
0209  * This allows the verify page to work offline for ANY
document,
0210  * not just previously-verified ones.
0211  *
0212  * Should be called on login and periodically when online.
0213  */
0214 export async function preloadVerificationHashes(userId?:
string): Promise<number> {
0215     try {
0216         // Check if we're online
0217         if (!navigator.onLine) return 0;
0218
0219         let query = supabase
0220             .from('submissions')
0221             .select(`
0222                 file_hash, file_name, doc_type,
compliance_status, created_at,
0223                 file_size, week_number, subject, school_year,
0224                 profiles:user_id ( full_name ),
0225                 teaching_loads ( subject, grade_level )
0226             `)
0227             .not('file_hash', 'like', 'missing_%')
0228             .order('created_at', { ascending: false });
0229
0230         // Scope to current user to reduce cache size
0231         if (userId) {
0232             query = query.eq('user_id', userId);
0233         }
0234
0235         const { data, error } = await query.limit(200);
0236
0237         if (error || !data) return 0;
0238
0239         let cached = 0;
0240         for (const item of data) {
0241             const profileData = item.profiles as any;
0242             const teachingLoadData = item.teaching_loads as
any;
0243

```

```

0244         const metadata = {
0245             file_name: item.file_name,
0246             doc_type: item.doc_type,
0247             compliance_status: item.compliance_status,
0248             created_at: item.created_at,
0249             file_size: item.file_size,
0250             week_number: item.week_number,
0251             subject: item.subject,
0252             school_year: item.school_year,
0253             teacher_name: profileData?.full_name || null,
0254             school_name: profileData?.schools?.name ||
null,
0255             teaching_load_subject:
teachingLoadData?.subject || null,
0256             teaching_load_grade:
teachingLoadData?.grade_level || null,
0257         };
0258
0259         await cacheVerifiedDoc(item.file_hash, metadata);
0260         cached++;

```

## Offline Sync Processor

**src/lib/utils/offline.ts Lines 260-620**

```

0260         cached++;
0261     }
0262
0263     console.log(`[offline] Pre-cached ${cached}
document hashes for offline verification`);
0264     return cached;
0265 } catch (err) {
0266     console.warn(`[offline] preloadVerificationHashes
error:`, err);
0267     return 0;
0268 }
0269 }
0270
0271 const QUEUE_PREFIX = 'sync_queue_';
0272
0273 export interface QueueItem {
0274     fileName: string;
0275     filePath: string;
0276     fileHash: string;
0277     fileSize: number;
0278     pdfBytes: Uint8Array | Blob;
0279     rawText?: string;
0280     options: {
0281         userId: string;
0282         docType?: string;
0283         weekNumber?: number;
0284         schoolYear?: string;
0285         subject?: string;
0286         calendarId?: string;
0287         teachingLoadId?: string;
0288     };
0289     timestamp: number;
0290 }
0291
0292 export async function enqueue(item: QueueItem):
Promise<void> {
0293     const shortHash = item.fileHash.slice(0, 8);
0294
0295     // The same document must not be queued twice. This
used to return normally
0296     // when it found one, so the caller went on to report the
upload as archived
0297     // while nothing had been queued — a submission
could vanish with a success
0298     // message. It throws now, so the teacher is actually

```

```

told.
0299 //
0300 // This only ever catches the identical file. An additional
DLL for a
0301 // week/subject that already has one is a different file
with a different
0302 // hash: it queues normally and syncs as
'supplementary'.
0303 const allKeys = await keys();
0304 for (const k of allKeys) {
0305   if (typeof k === 'string' &&
k.startsWith(Queue.PREFIX) && k.endsWith(shortHash)) {
0306     throw new Error("This exact document is already
waiting to be uploaded. Choose a different file, or
0307   }
0308 }
0309
0310 // Resilience: Workers often return SharedArrayBuffer
which CANNOT be stored in IndexedDB.
0311 // We MUST copy it to a plain Uint8Array (backed by a
real ArrayBuffer) first.
0312 let pdfBlobPart: any = item.pdfBytes;
0313 if (!(item.pdfBytes instanceof Blob)) {
0314   const fresh = new Uint8Array(item.pdfBytes.length);
0315   fresh.set(item.pdfBytes);
0316   pdfBlobPart = fresh;
0317 }
0318
0319 const storeItem = {
0320   ...item,
0321   pdfBytes: new Blob([pdfBlobPart], { type:
'application/pdf' }),
0322   options: JSON.parse(JSON.stringify(item.options))
0323 };
0324
0325 const key =
`${Queue.PREFIX}${item.timestamp}_${shortHash}`;
0326 await set(key, storeItem);
0327 await updatePendingCount();
0328
0329 // NOTE: No auto-sync here. The sync engine handles
sync separately
0330 // via initOfflineSync listeners (online event, visibility
change, etc.)
0331 // This prevents unnecessary sync attempts while
offline.
0332 console.log(`[offline] Enqueued: ${item.fileName} (sync
deferred)`);
0333 }
0334
0335 export async function getQueueSize(): Promise<number>
{
0336   const allKeys = await keys();
0337   return allKeys.filter((k: any) =>
String(k).startsWith(Queue.PREFIX)).length;
0338 }
0339
0340 export async function updatePendingCount():
Promise<void> {
0341   const size = await getQueueSize();
0342   pendingSyncCount.set(size);
0343
0344   // Sync to Global App Icon Badge
0345   const { updateAppBadge } = await import('./badge');
0346   await updateAppBadge();
0347 }
0348
0349 export async function getQueueItems():

```

```

Promise<QueueItem[]> {
0350   const allKeys = await keys();
0351   const queueKeys = allKeys.filter((k: any) =>
String(k).startsWith(Queue.PREFIX));
0352   const items: QueueItem[] = [];
0353
0354   for (const key of queueKeys) {
0355     const item = await get<QueueItem>(key);
0356     if (item) items.push(item);
0357   }
0358
0359   return items.sort((a, b) => a.timestamp - b.timestamp);
0360 }
0361
0362 import { addToast } from '$lib/stores/toast';
0363 import { isMobileDevice } from './device';
0364
0365 /**
0366  * Background-sync chatter ("Syncing N offline file(s)...",
"Synced N file(s)
0367  * successfully!", and so on).
0368  *
0369  * On mobile every upload goes through the queue by
design — the document is
0370  * processed, stamped and reported as archived before
the transfer runs — so
0371  * these notices fire on every single upload and contradict
the success the
0372  * teacher was just shown. They're plumbing, not
information the teacher can
0373  * act on, so they stay silent there. Desktop only reaches
the queue when a
0374  * direct upload actually failed, where the same notices
are genuinely
0375  * informative, so they're kept.
0376  */
0377 function syncToast(type: 'info' | 'success' | 'warning' |
'error', message: string) {
0378   if (isMobileDevice()) return;
0379   addToast(type, message);
0380 }
0381
0382 let isSyncing = false;
0383
0384 // ... (getQueueSize, getQueueItems remain same)
0385
0386 // Robust connectivity check
0387 // Robust connectivity check with timeout
0388 async function checkConnection(): Promise<boolean> {
0389   if (typeof navigator !== 'undefined' && !navigator.onLine)
return false;
0390
0391   try {
0392     const controller = new AbortController();
0393     const timeoutId = setTimeout(() => controller.abort(),
5000); // 5s timeout
0394
0395     // Try to reach the app itself (HEAD request is
lightweight)
0396     const res = await fetch(window.location.origin, {
0397       method: 'HEAD',
0398       cache: 'no-store',
0399       signal: controller.signal
0400     });
0401
0402     clearTimeout(timeoutId);
0403     return res.ok || res.status === 405; // 405 is fine
(method not allowed), just means server responded

```

```

0404 } catch (e) {
0405   console.warn('[offline] Connection check failed:', e);
0406   return false;
0407 }
0408 }
0409
0410 export async function processQueue(force = false):
Promise<{ success: number; failed: number }> {
0411   // If not forcing, check connection status first
0412   const isOnline = force ? true : await checkConnection();
0413
0414   if (!isOnline) {
0415     return { success: 0, failed: 0 };
0416   }
0417
0418   if (isSyncing && !force) {
0419     console.log('[offline] processQueue skipped: Already
syncing');
0420     return { success: 0, failed: 0 };
0421   }
0422
0423   const allKeys = await keys();
0424   const queueKeys = allKeys.filter((k: any) =>
String(k).startsWith(Queue.PREFIX));
0425
0426   if (queueKeys.length === 0) {
0427     return { success: 0, failed: 0 };
0428   }
0429
0430   console.log('[offline] Found ${queueKeys.length} items
to sync...');
0431   isSyncing = true;
0432   let success = 0;
0433   let failed = 0;
0434   // Labels for the completion message, phrased the way
a direct upload
0435   // phrases it ("DLL - Week 1") rather than as a sync
summary.
0436   const archivedLabels: string[] = [];
0437
0438   // Notify start
0439   syncToast('info', `Syncing ${queueKeys.length} offline
file(s)...`);
0440
0441   // Lazy-import ledger for marking synced entries
0442   const { markSynced, removeLedgerEntry } = await
import('./offlineSubmissionLedger');
0443
0444   try {
0445     for (const key of queueKeys) {
0446       // Re-check connection before each item
0447       if (!force && !(await checkConnection())) {
0448         console.log('[offline] Connection lost during sync.
Pausing...');
0449         break;
0450       }
0451
0452       const item = await get<QueueItem>(key);
0453
0454       // Handle corrupted/missing data
0455       if (!item) {
0456         console.warn('[offline] Found ghost key ${key},
removing...');
0457         await del(key);
0458         await updatePendingCount();
0459         continue;
0460       }
0461

```

```

0462       // Set when the server already holds a document
for this slot, or
0463       // this exact content. Either makes the item an
additional copy, so
0464       // it is archived and marked supplementary rather
than discarded.
0465       let forceSupplementary = false;
0466
0467       try {
0468         // — Re-validate against server before upload
—
0469         // (Items may have been uploaded by another
device/session since queuing)
0470
0471         // 1. Strict Metadata Check (One per
load/week/type)
0472         if (item.options.teachingLoadId &&
item.options.weekNumber) {
0473           const metaCheckPromise = supabase
0474             .from('submissions')
0475             .select('id')
0476             .eq('teaching_load_id',
item.options.teachingLoadId)
0477             .eq('week_number',
item.options.weekNumber)
0478             .eq('school_year', item.options.schoolYear
|| getCurrentSchoolYear())
0479             .eq('doc_type', item.options.docType ||
'DLL')
0480             .maybeSingle();
0481
0482           const { data: metaMatch } = await
withTimeout(
0483             metaCheckPromise as any,
0484             20000,
0485             'Meta check timed out'
0486           ) as { data: any };
0487
0488           if (metaMatch) {
0489             // An additional document for a
week/subject that already
0490             // has one is archived as supplementary,
not discarded.
0491             // This used to del() the queued item and
report it as
0492             // synced, which threw the teacher's
second DLL away.
0493             console.info('[offline] Slot already covered
— filing as supplementary: ${item.fileName}
0494             forceSupplementary = true;
0495           }
0496         }
0497
0498         // 2. Strict File Hash Check (No identical content)
— cross-teacher,
0499         // so it goes through a narrow RPC rather than a
direct table
0500         // select (see migrations/20260910_*.sql).
0501         //
0502         // Plain array request rather than .maybeSingle():
PostgREST
0503         // only 406s when a singular representation is
requested and
0504         // 0 rows come back — routine here, not a real
failure.
0505         const hashCheckPromise = supabase
0506           .rpc('check_duplicate_submission_hash',
{ p_hash: item.fileHash });

```

```

0507
0508     const { data: hashMatches } = await
withTimeout(
0509         hashCheckPromise as any,
0510         20000,
0511         'Hash check timed out'
0512     ) as { data: any[] };
0513     const existing = hashMatches?.[0];
0514
0515     if (existing) {
0516         // The identical document is already archived
on the server.
0517         // UNIQUE (file_hash) means this row cannot
be inserted, and
0518         // retrying forever would never succeed — but
nothing is lost
0519         // by clearing it, because the very same bytes
are already on
0520         // file. Uploads are refused up front for this
reason, so
0521         // reaching here means the copy was
archived from another
0522         // device while this one sat queued.
0523         console.info('[offline] Identical document
already archived on server, clearing queued copy:
0524         await del(key);
0525         await updatePendingCount();
0526         await markSynced(item.fileHash);
0527         success++;
0528         continue;
0529     }
0530
0531     // — Upload to Backblaze B2 via Pre-signed
URL —
0532     const { data: sessionData } = await
supabase.auth.getSession();
0533     const accessToken =
sessionData?.session?.access_token;
0534     if (!accessToken) throw new Error('Not
authenticated for background sync');
0535
0536     const sanitizedPath =
item.filePath.replace(/s+/g, '_').replace(/[^a-zA-Z0-9./_-]/g, "");
0537     const contentType = 'application/pdf';
0538
0539     console.log('[sync] Requesting pre-signed B2
upload URL for: ${item.fileName}');
0540
0541     // 1. Get a pre-signed PUT url from our API
(signing never hits B2 CORS)
0542     const presignResponse = await withTimeout(
0543         fetch('/api/storage/presign', {
0544             method: 'POST',
0545             headers: {
0546                 'Authorization': `Bearer ${accessToken}`,
0547                 'Content-Type': 'application/json'
0548             },
0549             body: JSON.stringify({ key: sanitizedPath,
contentType, intent: 'upload' })
0550         }),
0551         20000,
0552         'Pre-signed URL request timed out'
0553     ).catch(err => {
0554         console.error('[sync] Fetch error during pre-
signed URL request.', err);
0555         throw err;
0556     });
0557
0558         if (!presignResponse.ok) {
0559             let errStr = presignResponse.statusText;
0560             try {
0561                 const errJson = await
presignResponse.json();
0562                 errStr = errJson.message || errStr;
0563             } catch { /* ignore */ }
0564             if (presignResponse.status === 401 ||
presignResponse.status === 403) {
0565                 throw new Error('Not authenticated for
background sync. Please sign in again.');
```

```

0611         }},
0612         120000,
0613         'B2 sync archive upload timed out'
0614     ).catch(err => {
0615         console.error('[sync] Fetch error during B2
upload fallback.', err);
0616         throw err;
0617     });
0618
0619     if (!putResponse.ok) {
0620         let errStr = putResponse.statusText;

```

## Submission Ledger

**src/lib/utls/offlineSubmissionLedger.ts Lines 1-260**

```

0001 /**
0002  * Offline Submission Ledger — IndexedDB-backed
0003  *
0004  * Tracks all submissions (online + offline) for strict
enforcement:
0005  * - One upload per teaching load per week per doc type
(no duplicates)
0006  * - SHA-256 hash deduplication (no identical content)
0007  * - Survives browser refresh, cache clear, and offline
periods
0008  * - Syncs with server when online for ground-truth
consistency
0009  *
0010  * Uses idb-keyval with dedicated prefixes to avoid
conflicts with
0011  * the sync queue or other cached data.
0012  */
0013
0014 import { get, set, del, keys } from 'idb-keyval';
0015 import { getCurrentSchoolYear } from './schoolYear';
0016
0017 // ——— Prefixes

```

---

```

0018
0019 /** Tracks (teachingLoadId + weekNumber + schoolYear +
docType) → submission info */
0020 const SLOT_PREFIX = 'ledger_slot_';
0021
0022 /** Tracks fileHash → submission info (content
deduplication) */
0023 const HASH_PREFIX = 'ledger_hash_';
0024
0025 // ——— Types

```

---

```

0026
0027 export interface LedgerEntry {
0028     teachingLoadId: string;
0029     weekNumber: number;
0030     schoolYear: string;
0031     docType: string;
0032     fileHash: string;
0033     fileName: string;
0034     timestamp: number;
0035     /** 'pending' = queued offline, 'synced' = confirmed on
server */
0036     status: 'pending' | 'synced';
0037 }
0038
0039 // ——— Key Builders

```

---

0040

```

0041 function slotKey(teachingLoadId: string, weekNumber:
number, schoolYear: string, docType: string): string {
0042     return
`${SLOT_PREFIX}${teachingLoadId}_w${weekNumber}_${scho
olYear}_${docType}`;
0043 }
0044
0045 function hashKey(fileHash: string): string {
0046     return `${HASH_PREFIX}${fileHash}`;
0047 }
0048
0049 // ——— Core Operations

```

---

```

0050
0051 /**
0052  * Record a submission in the local ledger.
0053  * Called after both online upload success and offline
queue enqueue.
0054  */
0055 export async function recordSubmission(entry:
LedgerEntry): Promise<void> {
0056     const slot = slotKey(entry.teachingLoadId,
entry.weekNumber, entry.schoolYear, entry.docType);
0057     const hash = hashKey(entry.fileHash);
0058
0059     await Promise.all([
0060         set(slot, entry),
0061         set(hash, entry)
0062     ]);
0063
0064     console.log(`[ledger] Recorded: ${entry.docType} Week
${entry.weekNumber} (${entry.status})`);
0065 }
0066
0067 /**
0068  * Check if a submission slot is already taken.
0069  * Returns the existing entry if found, null otherwise.
0070  *
0071  * This enforces: ONE upload per teaching load per week
per doc type.
0072  */
0073 export async function hasSubmission(
0074     teachingLoadId: string,
0075     weekNumber: number,
0076     schoolYear: string,
0077     docType: string
0078 ): Promise<LedgerEntry | null> {
0079     const slot = slotKey(teachingLoadId, weekNumber,
schoolYear, docType);
0080     const entry = await get<LedgerEntry>(slot);
0081     return entry || null;
0082 }
0083
0084 /**
0085  * Check if a file hash already exists in the ledger.
0086  * Returns the existing entry if found, null otherwise.
0087  *
0088  * This enforces: NO identical document content (even
across different weeks).
0089  */
0090 export async function hasHash(fileHash: string):
Promise<LedgerEntry | null> {
0091     const entry = await
get<LedgerEntry>(hashKey(fileHash));
0092     return entry || null;
0093 }
0094

```

```

0095 /**
0096  * Mark a pending ledger entry as synced (confirmed on
server).
0097  */
0098  export async function markSynced(fileHash: string):
Promise<void> {
0099      const entry = await
get<LedgerEntry>(hashKey(fileHash));
0100      if (entry) {
0101          entry.status = 'synced';
0102          await recordSubmission(entry);
0103          console.log(`[ledger] Marked synced:
${entry.fileName}`);
0104      }
0105  }
0106
0107  /**
0108  * Remove a ledger entry by file hash.
0109  * Used when a sync fails permanently and the item
should be retryable.
0110  */
0111  export async function removeLedgerEntry(fileHash: string):
Promise<void> {
0112      const entry = await
get<LedgerEntry>(hashKey(fileHash));
0113      if (entry) {
0114          const slot = slotKey(entry.teachingLoadId,
entry.weekNumber, entry.schoolYear, entry.docType);
0115          await Promise.all([
0116              del(slot),
0117              del(hashKey(fileHash))
0118          ]);
0119          console.log(`[ledger] Removed: ${entry.fileName}`);
0120      }
0121  }
0122
0123  /**
0124  * Get all ledger entries (for debugging or display).
0125  */
0126  export async function getAllLedgerEntries():
Promise<LedgerEntry[]> {
0127      const allKeys = await keys();
0128      const slotKeys = allKeys.filter((k: any) =>
String(k).startsWith(SLOT_PREFIX));
0129      const entries: LedgerEntry[] = [];
0130
0131      for (const key of slotKeys) {
0132          const entry = await get<LedgerEntry>(key);
0133          if (entry) entries.push(entry);
0134      }
0135
0136      return entries.sort((a, b) => b.timestamp - a.timestamp);
0137  }
0138
0139  /**
0140  * Sync the ledger FROM the server.
0141  * Fetches all confirmed submissions for the user and
populates the ledger.
0142  * This ensures the ledger is accurate after clearing cache
or on first install.
0143  *
0144  * Should be called:
0145  * - On login / app startup (when online)
0146  * - After coming back online from an offline period
0147  */
0148  export async function syncLedgerFromServer(userId:
string): Promise<number> {
0149      if (!navigator.onLine) return 0;
0150
0151      try {
0152          const { supabase } = await import('./supabase');
0153
0154          const { data, error } = await supabase
.from('submissions')
0155              .select('teaching_load_id, week_number,
school_year, doc_type, file_hash, file_name, created_at')
0156              .eq('user_id', userId)
0157              .eq('school_year', getCurrentSchoolYear())
0158              .order('created_at', { ascending: false })
0159              .limit(200);
0160
0161          if (error || !data) {
0162              console.warn(`[ledger] Server sync failed:`,
error?.message);
0163              return 0;
0164          }
0165
0166          let synced = 0;
0167          for (const row of data) {
0168              if (!row.teaching_load_id || !row.week_number
|| !row.file_hash) continue;
0169              await recordSubmission({
0170                  teachingLoadId: row.teaching_load_id,
0171                  weekNumber: row.week_number,
0172                  schoolYear: row.school_year ||
getCurrentSchoolYear(),
0173                  docType: row.doc_type || 'DLL',
0174                  fileHash: row.file_hash,
0175                  fileName: row.file_name || 'unknown',
0176                  timestamp: new Date(row.created_at).getTime(),
0177                  status: 'synced'
0178              });
0179              synced++;
0180          }
0181
0182          console.log(`[ledger] Synced ${synced} entries from
server`);
0183          return synced;
0184      } catch (err) {
0185          console.warn(`[ledger] syncLedgerFromServer error:`,
err);
0186          return 0;
0187      }
0188  }
0189
0190  /**
0191  * Request persistent storage from the browser.
0192  * Prevents the browser from evicting IndexedDB data
under storage pressure.
0193  * Should be called once on app startup.
0194  */
0195  export async function requestPersistentStorage():
Promise<boolean> {
0196      if (typeof navigator === 'undefined') return false;
0197
0198      try {
0199          if (navigator.storage && navigator.storage.persist) {
0200              const granted = await navigator.storage.persist();
0201              console.log(`[ledger] Persistent storage ${granted ?
'GRANTED' : 'denied'}`);
0202              return granted;
0203          }
0204      } catch (err) {
0205          console.warn(`[ledger] requestPersistentStorage

```

```

error:', err);
0209   }
0210   return false;
0211 }
0212
0213 /**
0214  * Full integrity check: validates both slot uniqueness and
content hash uniqueness.
0215  * Returns a detailed result object for the UI to display.
0216  */
0217 export async function validateUploadIntegrity(
0218   teachingLoadId: string,
0219   weekNumber: number,
0220   schoolYear: string,
0221   docType: string,
0222   fileHash: string
0223 ): Promise<{
0224   allowed: boolean;
0225   reason?: string;
0226   existingEntry?: LedgerEntry;
0227   blockType?: 'slot_taken' | 'duplicate_content';
0228 }> {
0229   // Check 1: Slot uniqueness (one per teaching load per
week per doc type)
0230   const slotEntry = await hasSubmission(teachingLoadId,
weekNumber, schoolYear, docType);
0231   if (slotEntry) {
0232     return {
0233       allowed: false,
0234       reason: `A ${docType} document for Week
${weekNumber} has already been ${slotEntry.status === 'sync'
0235       existingEntry: slotEntry,
0236       blockType: 'slot_taken'
0237     };
0238   }
0239
0240   // Check 2: Content hash uniqueness (no identical
documents)
0241   const hashEntry = await hasHash(fileHash);
0242   if (hashEntry) {
0243     return {
0244       allowed: false,
0245       reason: `This exact document has already been
submitted as "${hashEntry.fileName}" (${hashEntry.docT
0246       existingEntry: hashEntry,
0247       blockType: 'duplicate_content'
0248     };
0249   }
0250
0251   return { allowed: true };
0252 }

```

## Analytics Queries

**src/lib/utils/analyticsQueries.ts Lines 1-260**

```

0001 import { supabase } from '$lib/utils/supabase';
0002 import { calculateCompliance } from
'$lib/utils/useDashboardData';
0003
0004 export async function getSchoolHeadAnalytics(schoolId:
string, districtId: string) {
0005   // NOTE: every query below embeds `profiles!inner(...)`
(not the plain
0006   // `profiles(...)` left-join form) so that
`.eq('profiles.school_id', ...)`
0007   // actually restricts the top-level `submissions` rows
returned. Without
0008   // `!inner`, PostgREST treats it as a left join and the
school_id filter

```

```

0009   // does not narrow the result set — every query here
previously returned
0010   // submissions from every school in the system instead
of just this one.
0011   const [complianceTrend, teacherPerformance,
atRiskTeachers, rosterProfiles, rosterLoads] = await Promise.all
0012     // Compliance trend over last 12 weeks
0013     supabase
0014       .from('submissions')
0015       .select(`
0016         id,
0017         created_at,
0018         compliance_status,
0019         doc_type,
0020         week_number,
0021         teaching_load_id,
0022         profiles!inner(school_id)
0023       `)
0024       .eq('profiles.school_id', schoolId)
0025       .order('created_at', { ascending: true })
0026       .limit(10000),
0027
0028     // Teacher performance metrics (week_number +
teaching_load_id are
0029     // required for calculateCompliance's slot-dedup —
see
0030     // getPerformanceDistribution below)
0031     supabase
0032       .from('submissions')
0033       .select(`
0034         user_id,
0035         compliance_status,
0036         doc_type,
0037         created_at,
0038         week_number,
0039         teaching_load_id,
0040         profiles!inner(full_name, role, school_id)
0041       `)
0042       .eq('profiles.school_id', schoolId)
0043       .order('created_at', { ascending: false })
0044       .limit(10000),
0045
0046     // Teachers below compliance threshold (< 70%)
0047     supabase
0048       .from('submissions')
0049       .select(`
0050         user_id,
0051         compliance_status,
0052         profiles!inner(full_name, role, avatar_url,
school_id)
0053       `)
0054       .eq('profiles.school_id', schoolId)
0055       .order('user_id')
0056       .limit(10000),
0057
0058     // Full teacher/master-teacher roster for this school,
so an entity
0059     // with zero submissions still shows up (at 0%)
instead of being
0060     // silently absent from clustering/rankings/at-risk.
0061     supabase
0062       .from('profiles')
0063       .select('id, full_name')
0064       .eq('school_id', schoolId)
0065       .in('role', ['Teacher', 'Master Teacher']),
0066
0067     // Active teaching load counts per teacher, for the
expected-total

```

```

0068 // denominator in getPerformanceDistribution
0069 supabase
0070   .from('teaching_loads')
0071   .select('id, user_id, profiles!inner(school_id)')
0072   .eq('profiles.school_id', schoolId)
0073 });
0074
0075 const loadCountByUser = new Map<string, number>();
0076 for (const l of (rosterLoads.data || []) as any[]) {
0077   loadCountByUser.set(l.user_id,
0078     (loadCountByUser.get(l.user_id) || 0) + 1);
0079 }
0080 const roster = (rosterProfiles.data || []).map((p: any) =>
0081 ({
0082   id: p.id,
0083   name: p.full_name,
0084   loadCount: loadCountByUser.get(p.id) || 0
0085 }));
0086
0087 return {
0088   complianceTrend: complianceTrend.data || [],
0089   teacherPerformance: teacherPerformance.data || [],
0090   atRiskTeachers: atRiskTeachers.data || [],
0091   roster
0092 };
0093 }
0094 export async function
0095 getDistrictSupervisorAnalytics(districtId: string) {
0096   // See the inner note in getSchoolHeadAnalytics above
0097   — same fix applies
0098   // here, scoped to district_id instead of school_id.
0099   const [complianceTrend, schoolPerformance,
0100   teacherDistribution, alertData, rosterProfiles, rosterLoads] = await
0101   // District compliance trend
0102   supabase
0103     .from('submissions')
0104     .select(`
0105       id,
0106       created_at,
0107       compliance_status,
0108       doc_type,
0109       week_number,
0110       teaching_load_id,
0111       profiles!inner(district_id, schools(name))
0112     `)
0113     .eq('profiles.district_id', districtId)
0114     .order('created_at', { ascending: true })
0115     .limit(10000),
0116   // School performance metrics
0117   supabase
0118     .from('submissions')
0119     .select(`
0120       compliance_status,
0121       profiles!inner(district_id, school_id,
0122       schools(name))
0123     `)
0124     .eq('profiles.district_id', districtId)
0125     .limit(10000),
0126   // Teacher performance distribution (for k-means
0127   clustering).
0128   // week_number + teaching_load_id are required for
0129   // calculateCompliance's slot-dedup — see
0130   getPerformanceDistribution.
0131   supabase
0132     .from('submissions')
0133     .select(`
0134       user_id,
0135       compliance_status,
0136       doc_type,
0137       created_at,
0138       week_number,
0139       teaching_load_id,
0140       profiles!inner(full_name, district_id, school_id,
0141       schools(name))
0142     `)
0143     .eq('profiles.district_id', districtId)
0144     .order('user_id')
0145     .limit(10000),
0146   // Critical alerts
0147   supabase
0148     .from('submissions')
0149     .select(`
0150       id,
0151       compliance_status,
0152       created_at,
0153       profiles!inner(full_name, district_id,
0154       schools(name))
0155     `)
0156     .eq('profiles.district_id', districtId)
0157     .in('compliance_status', ['late', 'missing'])
0158     .order('created_at', { ascending: false })
0159     .limit(100),
0160   // Full teacher/master-teacher roster across the
0161   district, so an
0162   // entity with zero submissions still shows up (at 0%)
0163   instead of
0164   // being silently absent from clustering/rankings/at-
0165   risk.
0166   supabase
0167     .from('profiles')
0168     .select('id, full_name')
0169     .eq('district_id', districtId)
0170     .in('role', ['Teacher', 'Master Teacher']),
0171   // Active teaching load counts per teacher, for the
0172   expected-total
0173   // denominator in getPerformanceDistribution
0174   supabase
0175     .from('teaching_loads')
0176     .select('id, user_id, profiles!inner(district_id)')
0177     .eq('profiles.district_id', districtId)
0178   });
0179
0180 const loadCountByUser = new Map<string, number>();
0181 for (const l of (rosterLoads.data || []) as any[]) {
0182   loadCountByUser.set(l.user_id,
0183     (loadCountByUser.get(l.user_id) || 0) + 1);
0184 }
0185 const roster = (rosterProfiles.data || []).map((p: any) =>
0186 ({
0187   id: p.id,
0188   name: p.full_name,
0189   loadCount: loadCountByUser.get(p.id) || 0
0190 }));
0191
0192 return {
0193   complianceTrend: complianceTrend.data || [],
0194   schoolPerformance: schoolPerformance.data || [],
0195   teacherDistribution: teacherDistribution.data || [],
0196   alerts: alertData.data || [],
0197   roster

```

```

0188   };
0189 }
0190
0191 export function calculateComplianceMetrics(submissions:
any[]) {
0192   if (submissions.length === 0) return { compliant: 0, late:
0, missing: 0, rate: 0, total: 0 };
0193
0194   const compliant = submissions.filter(s =>
s.compliance_status === 'compliant').length;
0195   const late = submissions.filter(s => s.compliance_status
=== 'late').length;
0196   const missing = submissions.filter(s =>
s.compliance_status === 'missing').length;
0197   const total = submissions.length;
0198
0199   return {
0200     compliant,
0201     late,
0202     missing,
0203     total,
0204     rate: Math.round((compliant / total) * 100)
0205   };
0206 }
0207
0208 export function generateComplianceTrend(submissions:
any[], granularity: 'week' | 'month' = 'week') {
0209   const trendMap = new Map<string, { compliant: number;
late: number; missing: number; total: number }>();
0210
0211   submissions.forEach(sub => {
0212     const date = new Date(sub.created_at);
0213     let key: string;
0214
0215     if (granularity === 'week') {
0216       const weekNum = Math.ceil((date.getDate() -
date.getDay()) / 7);
0217       const month = String(date.getMonth() +
1).padStart(2, '0');
0218       key = `W${weekNum}-${month}`;
0219     } else {
0220       const month = date.toLocaleDateString('en-PH',
{ month: 'short', year: '2-digit' });
0221       key = month;
0222     }
0223
0224     if (!trendMap.has(key)) {
0225       trendMap.set(key, { compliant: 0, late: 0, missing:
0, total: 0 });
0226     }
0227
0228     const stats = trendMap.get(key)!;
0229     stats.total += 1;
0230
0231     if (sub.compliance_status === 'compliant')
stats.compliant += 1;
0232     else if (sub.compliance_status === 'late') stats.late
+= 1;
0233     else if (sub.compliance_status === 'missing')
stats.missing += 1;
0234   });
0235
0236   return Array.from(trendMap.entries()).map(([key, stats])
=> ({
0237     period: key,
0238     ...stats,
0239     rate: Math.round((stats.compliant / stats.total) * 100)
0240   }));

```

```

0241 }
0242
0243 /**
0244  * Performance distribution for k-means clustering
0245  * Calculates compliance rate and submission frequency
for each teacher/school.
0246  *
0247  * `roster` + `definedWeeks` give each entity its true
expected total
0248  * (loadCount * definedWeeks), so the rate is computed
the same
0249  * capped, expected-total-based way as the rest of the
app (via
0250  * calculateCompliance) instead of compliant/submitted-
only — and an entity
0251  * with zero submissions still gets an entry instead of
being silently
0252  * absent from clustering/rankings/at-risk.
0253  */
0254 export function getPerformanceDistribution(
0255   submissions: any[],
0256   groupBy: 'teacher' | 'school' = 'teacher',
0257   roster: { id: string; name: string; loadCount: number }[] =
[],
0258   definedWeeks: number = 0
0259 ) {
0260   const idOf = (sub: any) => groupBy === 'teacher' ?
sub.user_id : sub.profiles?.school_id;

```

## Compliance Dashboard

**src/routes/dashboard/compliance/+page.svelte Lines 1-360**

```

0001 <script lang="ts">
0002   import PageHeader from
"$lib/components/PageHeader.svelte";
0003   import SkeletonLoader from
"$lib/components/SkeletonLoader.svelte";
0004   import { addToast } from "$lib/stores/toast";
0005   import { profile } from "$lib/utis/auth";
0006   import { supabase } from "$lib/utis/supabase";
0007   import {
0008     calculateCompliance,
0009     getCurrentWeekFromCalendar,
0010     getDefinedWeeksCount,
0011     getDynamicSchoolYear,
0012     isComplianceTrackedDocType,
0013   } from "$lib/utis/useDashboardData";
0014   import { canCluster, extractFeatures,
runKMeansClustering, type ClusterSummary } from
"$lib/utis/clusterAna
0015   import {
0016     ArrowLeft,
0017     ArrowUpDown,
0018     CheckCircle2,
0019     Clock3,
0020     Eye,
0021     FileCheck2,
0022     Search,
0023     ShieldAlert,
0024     Sparkles,
0025     Users,
0026   } from "lucide-svelte";
0027   import { onDestroy, onMount } from "svelte";
0028
0029   type ScopeMode = "school" | "district";
0030
0031   type TeacherRow = {
0032     id: string;
0033     name: string;

```

```

0034     loadLabel: string;
0035     expected: number;
0036     compliant: number;
0037     missing: number;
0038     late: number;
0039     forChecking: number;
0040     checked: number;
0041     rate: number;
0042     schoolId?: string | null;
0043     schoolName?: string;
0044   };
0045
0046   type SchoolRow = {
0047     id: string;
0048     name: string;
0049     teacherCount: number;
0050     compliant: number;
0051     missing: number;
0052     late: number;
0053     forChecking: number;
0054     checked: number;
0055     rate: number;
0056   };
0057
0058   type Summary = {
0059     compliant: number;
0060     missing: number;
0061     late: number;
0062     forChecking: number;
0063     checked: number;
0064   };
0065
0066   let loading = $state(true);
0067   let loadError = $state<string | null>(null);
0068   let mode = $state<ScopeMode>("school");
0069   let teacherRows = $state<TeacherRow[]>([]);
0070   let schoolRows = $state<SchoolRow[]>([]);
0071   let selectedSchoolId = $state<string | null>(null);
0072   let selectedSchoolName = $state<string | null>(null);
0073   let clusterSummaries = $state<ClusterSummary[]>([]);
0074   let canShowClusters = $state(false);
0075   let clusterTeachers = $state<any[]>([]);
0076   let clusterSubmissions = $state<any[]>([]);
0077   let clusterWeeks = $state(1);
0078   let search = $state("");
0079   let statusFilter = $state("all");
0080   let weekFilter = $state("all");
0081   let sortField = $state<keyof TeacherRow>("name");
0082   let sortDir = $state<"asc" | "desc">("asc");
0083   let currentPage = $state(1);
0084   let currentWeek = $state(1);
0085   let realtimeChannel: ReturnType<typeof
supabase.channel> | null = null;
0086   let refreshTimer: ReturnType<typeof setTimeout> | null
= null;
0087
0088   const pageSize = 8;
0089
0090   const visibleTeacherRows = $derived.by(() => {
0091     if (mode !== "district" || !selectedSchoolId) return
teacherRows;
0092     return teacherRows.filter((row) => row.schoolId ===
selectedSchoolId);
0093   });
0094
0095   const summary = $derived.by(() : Summary =>
summarizeRows(visibleTeacherRows));
0096   const districtSummary = $derived.by(() : Summary =>

```

```

summarizeRows(teacherRows));
0097
0098   const filteredRows = $derived.by(() => {
0099     const q = search.trim().toLowerCase();
0100     let rows = visibleTeacherRows.filter((row) => {
0101       const matchesSearch =
0102         !q ||
0103         row.name.toLowerCase().includes(q) ||
0104         row.loadLabel.toLowerCase().includes(q) ||
0105         (row.schoolName ||
0106         "").toLowerCase().includes(q);
0107       const matchesStatus =
0108         statusFilter === "all" ||
0109         (statusFilter === "compliant" && row.missing
=== 0) ||
0110         (statusFilter === "missing" && row.missing > 0)
||
0111         (statusFilter === "late" && row.late > 0) ||
0112         (statusFilter === "forChecking" &&
row.forChecking > 0) ||
0113         (statusFilter === "checked" && row.checked >
0);
0114       return matchesSearch && matchesStatus;
0115     });
0116     rows = [...rows].sort((a, b) => {
0117       const aVal = a[sortField];
0118       const bVal = b[sortField];
0119       if (typeof aVal === "number" && typeof bVal ===
"number") {
0120         return sortDir === "asc" ? aVal - bVal : bVal -
aVal;
0121       }
0122       return sortDir === "asc"
0123         ? String(aVal).localeCompare(String(bVal))
0124         : String(bVal).localeCompare(String(aVal));
0125     });
0126     return rows;
0127   });
0128
0129   const totalPages = $derived(Math.max(1,
Math.ceil(filteredRows.length / pageSize)));
0130   $effect(() => {
0131     if (currentPage > totalPages) currentPage =
totalPages;
0132   });
0133   const pageRows =
$derived(filteredRows.slice((currentPage - 1) * pageSize,
currentPage * pageSize));
0134
0135   const statCards = $derived([
0136     { label: "Compliant", value: (mode === "district"
&& !selectedSchoolId ? districtSummary : summary).comp
0137     { label: "Missing", value: (mode === "district"
&& !selectedSchoolId ? districtSummary : summary).missin
0138     { label: "Late", value: (mode === "district"
&& !selectedSchoolId ? districtSummary : summary).late, ico
0139     { label: "For Checking", value: (mode === "district"
&& !selectedSchoolId ? districtSummary : summary).f
0140     { label: "Checked", value: (mode === "district"
&& !selectedSchoolId ? districtSummary : summary).checke
0141   ]});
0142
0143   onMount(async () => {
0144     await loadCompliance();
0145     realtimeChannel = supabase
0146       .channel("compliance-monitoring-live")
0147       .on("postgres_changes", { event: "*", schema:

```

```

"public", table: "submissions" }, queueRefresh)
0148   .on("postgres_changes", { event: "*", schema:
"public", table: "dll_reviews" }, queueRefresh)
0149   .on("postgres_changes", { event: "*", schema:
"public", table: "teaching_loads" }, queueRefresh)
0150   .on("postgres_changes", { event: "*", schema:
"public", table: "profiles" }, queueRefresh)
0151   .subscribe();
0152   });
0153
0154   onDestroy(() => {
0155     if (refreshTimer) clearTimeout(refreshTimer);
0156     if (realtimeChannel)
supabase.removeChannel(realtimeChannel);
0157   });
0158
0159   function queueRefresh() {
0160     if (refreshTimer) clearTimeout(refreshTimer);
0161     refreshTimer = setTimeout(() => {
0162       loadCompliance();
0163       refreshTimer = null;
0164     }, 350);
0165   }
0166
0167   async function loadCompliance() {
0168     const userProfile = $profile;
0169     if (!userProfile || !["Master Teacher", "School Head",
"District Supervisor"].includes(userProfile.role
0170     loadError = "Compliance Monitoring is available to
supervisors.";
0171     loading = false;
0172     return;
0173   }
0174
0175   loading = true;
0176   loadError = null;
0177
0178   try {
0179     mode = userProfile.role === "District Supervisor" ?
"district" : "school";
0180     if (mode === "district") await
loadDistrictCompliance(userProfile);
0181     else await loadSchoolCompliance(userProfile);
0182   } catch (err) {
0183     console.error("[compliance-monitoring] load failed",
err);
0184     loadError = "Failed to load compliance monitoring
data. Please try again.";
0185     addToast("error", loadError);
0186   } finally {
0187     loading = false;
0188   }
0189 }
0190
0191   async function loadSchoolCompliance(userProfile: any)
{
0192     const [teachersRes, loadsRes, submissionsRes,
reviewsRes] = await Promise.all([
0193       supabase
0194         .from("profiles")
0195         .select("id, full_name, role, school_id")
0196         .eq("school_id", userProfile.school_id)
0197         .in("role", ["Teacher", "Master Teacher"])
0198         .order("full_name"),
0199       supabase
0200         .from("teaching_loads")
0201         .select("id, user_id, subject, grade_level,
profiles!inner(school_id)")
0202         .eq("profiles.school_id", userProfile.school_id),
0203       supabase
0204         .from("submissions")
0205         .select("id, user_id, file_name, doc_type,
compliance_status, created_at, week_number, teaching_
0206         .eq("profiles.school_id", userProfile.school_id)
0207         .order("created_at", { ascending: false }),
0208       supabase.from("dll_reviews").select("submission_id,
reviewer_comment"),
0209     ]);
0210
0211     if (teachersRes.error) throw teachersRes.error;
0212     if (loadsRes.error) throw loadsRes.error;
0213     if (submissionsRes.error) throw
submissionsRes.error;
0214
0215     currentWeek = await
getCurrentWeekFromCalendar(supabase,
getDynamicSchoolYear(), userProfile.district_id
0216     const definedWeeks = await
getDefinedWeeksCount(supabase);
0217     const activeWeeks = Math.max(1,
Math.min(currentWeek || 1, definedWeeks || 1));
0218     const teachers = teachersRes.data || [];
0219     const loads = loadsRes.data || [];
0220     const submissions = (submissionsRes.data ||
[])?.filter((s: any) => isComplianceTrackedDocType(s.doc_type
0221     const reviewBySubmission = new
Map((reviewsRes.data || []).map((r: any) => [r.submission_id,
r.reviewer_
0222
0223     schoolRows = [];
0224     selectedSchoolId = null;
0225     selectedSchoolName = null;
0226     teacherRows = buildTeacherRows(teachers, loads,
submissions, reviewBySubmission, activeWeeks);
0227     clusterTeachers = teachers;
0228     clusterSubmissions = submissions;
0229     clusterWeeks = activeWeeks;
0230     buildClusters(teachers, submissions, activeWeeks);
0231   }
0232
0233   async function loadDistrictCompliance(userProfile: any)
{
0234     const schoolsRes = await supabase
0235       .from("schools")
0236       .select("id, name, district_id")
0237       .eq("district_id", userProfile.district_id)
0238       .order("name");
0239     if (schoolsRes.error) throw schoolsRes.error;
0240
0241     const schools = schoolsRes.data || [];
0242     const schoolIds = schools.map((school: any) =>
school.id);
0243     if (schoolIds.length === 0) {
0244       teacherRows = [];
0245       schoolRows = [];
0246       clusterSummaries = [];
0247       canShowClusters = false;
0248       return;
0249     }
0250
0251     const [teachersRes, loadsRes, submissionsRes,
reviewsRes, calendarRes] = await Promise.all([
0252       supabase
0253         .from("profiles")
0254         .select("id, full_name, role, school_id")

```

```

0255     .in("school_id", schoolIds)
0256     .in("role", ["Teacher", "Master Teacher"])
0257     .order("full_name"),
0258     supabase
0259     .from("teaching_loads")
0260     .select("id, user_id, subject, grade_level,
profiles!inner(school_id)")
0261     .in("profiles.school_id", schoolIds),
0262     supabase
0263     .from("submissions")
0264     .select("id, user_id, file_name, doc_type,
compliance_status, created_at, week_number, teaching_
0265     .in("profiles.school_id", schoolIds)
0266     .order("created_at", { ascending: false }},
0267     supabase.from("dll_reviews").select("submission_id,
reviewer_comment"),
0268     supabase
0269     .from("academic_calendar")
0270     .select("week_number, is_active, district_id,
school_year")
0271     .eq("school_year", getDynamicSchoolYear())
0272     .order("week_number", { ascending: true }},
0273     ]);
0274
0275     if (teachersRes.error) throw teachersRes.error;
0276     if (loadsRes.error) throw loadsRes.error;
0277     if (submissionsRes.error) throw
submissionsRes.error;
0278
0279     currentWeek = await
getCurrentWeekFromCalendar(supabase,
getDynamicSchoolYear(), userProfile.district_id
0280     const districtWeeks = (calendarRes.data ||
[])?.filter((week: any) => week.district_id === userProfile.di
0281     const activeWeeks = Math.max(1,
Math.min(currentWeek || 1, districtWeeks.length || currentWeek
|| 1));
0282     const schoolNameById = new
Map(schools.map((school: any) => [school.id, school.name]));
0283     const teachers = (teachersRes.data ||
[])?.map((teacher: any) => ({
0284         ...teacher,
0285         school_name:
schoolNameById.get(teacher.school_id) || "Unassigned School",
0286     }));
0287     const loads = loadsRes.data || [];
0288     const submissions = (submissionsRes.data ||
[])?.filter((s: any) => isComplianceTrackedDocType(s.doc_type
0289     const reviewBySubmission = new
Map((reviewsRes.data || []).map((r: any) => [r.submission_id,
r.reviewer_
0290
0291     teacherRows = buildTeacherRows(teachers, loads,
submissions, reviewBySubmission, activeWeeks);
0292     clusterTeachers = teachers;
0293     clusterSubmissions = submissions;
0294     clusterWeeks = activeWeeks;
0295     schoolRows = schools.map((school: any) => {
0296         const rows = teacherRows.filter((row) =>
row.schoolId === school.id);
0297         const stats = summarizeRows(rows);
0298         const expected = rows.reduce((sum, row) => sum
+ row.expected, 0);
0299         const actual = stats.compliant;
0300         return {
0301             id: school.id,
0302             name: school.name,

```

```

0303             teacherCount: rows.length,
0304             ...stats,
0305             rate: expected > 0 ? Math.round((actual /
expected) * 100) : 0,
0306         });
0307     });
0308     if (selectedSchoolId)
buildClustersForSchool(selectedSchoolId);
0309     else buildClusters(teachers, submissions,
activeWeeks);
0310 }
0311
0312     function buildTeacherRows(teachers: any[], loads: any[],
submissions: any[], reviewBySubmission: Map<any, an
0313     return teachers.map((teacher: any) => {
0314         const teacherLoads = loads.filter((load: any) =>
load.user_id === teacher.id);
0315         const teacherSubs = submissions.filter((sub: any)
=> sub.user_id === teacher.id);
0316         const stats = calculateCompliance(teacherSubs,
teacherLoads.length * activeWeeks);
0317         const checked = teacherSubs.filter((sub: any) =>
reviewBySubmission.get(sub.id)).length;
0318         const forChecking = teacherSubs.filter((sub: any)
=> !reviewBySubmission.get(sub.id)).length;
0319
0320         return {
0321             id: teacher.id,
0322             name: teacher.full_name || "Unnamed teacher",
0323             loadLabel: summarizeLoads(teacherLoads),
0324             expected: stats.expected,
0325             compliant: stats.Compliant,
0326             missing: stats.NonCompliant,
0327             late: stats.Late,
0328             forChecking,
0329             checked,
0330             rate: stats.rate,
0331             schoolId: teacher.school_id || null,
0332             schoolName: teacher.school_name,
0333         };
0334     });
0335 }
0336
0337     function buildClusters(teachers: any[], submissions:
any[], activeWeeks: number) {
0338         const clusterInput = submissions.map((sub: any) =>
({
0339             user_id: sub.user_id,
0340             compliance_status: sub.compliance_status,
0341             week_number: sub.week_number,
0342             created_at: sub.created_at,
0343         }));
0344         canShowClusters = canCluster(teachers.length,
clusterInput.length);
0345         clusterSummaries = canShowClusters
? runKMeansClustering(extractFeatures(teachers,
clusterInput, activeWeeks), 3).summaries
0346         : [];
0347     }
0348 }
0349
0350     function buildClustersForSchool(schoolId: string) {
0351         const teachers = clusterTeachers.filter((teacher: any)
=> teacher.school_id === schoolId);
0352         const teacherIds = new Set(teachers.map((teacher:
any) => teacher.id));
0353         const submissions = clusterSubmissions.filter((sub:
any) => teacherIds.has(sub.user_id));
0354         buildClusters(teachers, submissions, clusterWeeks);

```

```

0355   }
0356
0357   function summarizeRows(rows: TeacherRow[]):
Summary {
0358     return rows.reduce(
0359       (acc, row) => {
0360         acc.compliant += row.compliant;

```

## Archive View

**src/routes/dashboard/archive/+page.svelte Lines 1-300**

```

0001   <script lang="ts">
0002     import { supabase, getRows } from "$lib/utis/supabase";
0003     import PageHeader from
'$lib/components/PageHeader.svelte';
0004     import { profile } from "$lib/utis/auth";
0005     import StatusBadge from
"$lib/components/StatusBadge.svelte";
0006     import FolderCard from
"$lib/components/FolderCard.svelte";
0007     import { normalizeComplianceStatus } from
"$lib/utis/useDashboardData";
0008     import {
0009       canAddReviewRemarks,
0010       canAddRemarkToISPISR,
0011       canViewUploadedISPISR,
0012     } from "$lib/utis/documentPermissions";
0013     import { shareVerification } from
"$lib/utis/shareIntegration";
0014     import { addToast } from "$lib/stores/toast";
0015     import { fly, fade } from "svelte/transition";
0016     import {
0017       FolderOpen,
0018       FileText,
0019       ChevronRight,
0020       Home,
0021       Search,
0022       Download,
0023       Eye,
0024       MessageSquare,
0025       Share2,
0026       X,
0027       ArrowLeft,
0028       ArrowUpDown,
0029     } from "lucide-svelte";
0030     import { cacheMetadata, getCacheMetadata } from
"$lib/utis/offline";
0031     import EmptyState from
"$lib/components/EmptyState.svelte";
0032     import SkeletonLoader from
"$lib/components/SkeletonLoader.svelte";
0033     import { focusTrap } from "$lib/actions/focusTrap";
0034     import { getDocumentLabel } from
"$lib/utis/documentLabels";
0035
0036     // â€œâ€œ Types â€œâ€œ
0037     interface Submission {
0038       id: string;
0039       user_id: string;
0040       file_name: string;
0041       file_hash: string;
0042       file_size: number;
0043       doc_type: string;
0044       compliance_status: string;
0045       week_number: number | null;
0046       subject: string | null;
0047       teaching_load_id: string | null;
0048       created_at: string;
0049       file_path: string;

```

```

0050     uploader?: {
0051       full_name: string;
0052       school_id: string | null;
0053       district_id: string | null;
0054       avatar_url?: string | null;
0055       role: string;
0056     };
0057     school_name?: string;
0058     school_avatar?: string | null;
0059   }
0060
0061   interface PathSegment {
0062     type: "root" | "docType" | "school" | "teacher" |
"subject" | "week";
0063     id: string;
0064     label: string;
0065   }
0066
0067   interface FolderItem {
0068     id: string;
0069     label: string;
0070     count: number;
0071     type: "docType" | "school" | "teacher" | "subject" |
"week";
0072     icon?: string;
0073     avatar_url?: string | null;
0074   }
0075
0076   // â€œâ€œ State â€œâ€œ
0077   let allSubmissions = $state<Submission[]>([]);
0078   let schoolsMap = $state<Record<string, { label: string;
avatar_url: string | null }>>({});
0079   let teachersMap = $state<Record<string, { label: string;
avatar_url: string | null }>>({});
0080   let loading = $state(true);
0081   let loadError = $state<string | null>(null);
0082   let searchQuery = $state("");
0083   let currentPage = $state(1);
0084   let folderPage = $state(1);
0085   const pageSize = 8;
0086   const folderPageSize = 10;
0087   let currentPath = $state<PathSegment[]>([
0088     { type: "root", id: "root", label: "Archive" },
0089   ]);
0090
0091   // â€œâ€œ Remarks & Review Status â€œâ€œ
0092   let reviewsMap = $state<Record<string,
{ reviewer_comment: string | null; status: string | null;
return_reas
0093     let statusFilter = $state<"all" | "for-checking" |
"checked">("all");
0094   let sortField = $state<"date" | "name" | "size">("date");
0095   let sortDir = $state<"asc" | "desc">("desc");
0096
0097   let remarkModalOpen = $state(false);
0098   let remarkTarget = $state<Submission | null>(null);
0099   let remarkText = $state("");
0100   let existingRemark = $state<string | null>(null);
0101   let savingRemark = $state(false);
0102   let openingSubmissionId = $state<string | null>(null);
0103
0104   const canReview = $derived(
0105     $profile ? canAddReviewRemarks($profile.role) :
false,
0106   );
0107
0108   function canAddRemarkToSubmission(sub:
Submission): boolean {

```

```

0109     if (!$profile) return false;
0110     // For ISP/ISR, check the specific uploader role
0111     if (sub.doc_type === 'ISP' || sub.doc_type === 'ISR')
0112     {
0113         return canAddRemarkToISPISR(
0114             $profile.role,
0115             sub.uploader?.role || 'Teacher',
0116             sub.doc_type
0117         );
0118     }
0119     // For DLL, anyone who can review can add remarks
0120     return canReview;
0121 }
0122 function canViewRemarkForSubmission(sub:
Submission): boolean {
0123     if (!$profile) return false;
0124     if (!reviewsMap[sub.id]?.reviewer_comment) return
canAddRemarkToSubmission(sub);
0125
0126     if ($profile.role === "Teacher") {
0127         return sub.user_id === $profile.id;
0128     }
0129
0130     return canAddRemarkToSubmission(sub);
0131 }
0132
0133 function openRemarkModal(sub: Submission) {
0134     if (!canViewRemarkForSubmission(sub)) return;
0135     remarkTarget = sub;
0136     remarkText = "";
0137     existingRemark =
reviewsMap[sub.id]?.reviewer_comment || null;
0138     remarkText = existingRemark || "";
0139     remarkModalOpen = true;
0140 }
0141
0142 function closeRemarkModal() {
0143     remarkModalOpen = false;
0144     remarkTarget = null;
0145 }
0146
0147 async function saveRemark() {
0148     if (!remarkTarget
|| !canAddRemarkToSubmission(remarkTarget)) {
0149         addToast(
0150             "error",
0151             "You don't have permission to add remarks to
this document",
0152         );
0153         return;
0154     }
0155     if (!remarkText.trim() || !$profile) return;
0156     savingRemark = true;
0157     const { error } = await
supabase.from("dll_reviews").upsert({
0158         submission_id: remarkTarget.id,
0159         reviewer_id: $profile.id,
0160         reviewer_comment: remarkText.trim(),
0161         status: "needs-check",
0162     }, { onConflict: "submission_id" });
0163     savingRemark = false;
0164     if (error) {
0165         addToast("error", "Failed to save remark: " +
error.message);
0166     } else {
0167         reviewsMap[remarkTarget.id] = {
0168             reviewer_comment: remarkText.trim(),

```

```

0169         status: "needs-check",
0170         return_reason: null,
0171         reviewer_id: $profile.id,
0172         reviewer_name: $profile.full_name || "Checker",
0173     };
0174     addToast("success", "Remark saved
successfully");
0175     closeRemarkModal();
0176 }
0177 }
0178
0179 // â€œâ€œ Lifecycle â€œâ€œ
0180 // Reactive on $profile?.id (not just onMount) as
defense-in-depth: if the
0181 // resolved profile ever changes after mount (auth state
event, token
0182 // refresh resolving a different cached vs. server profile,
etc.), this
0183 // re-fetches scoped to whichever account is actually
current instead of
0184 // being locked to whatever was true at the moment this
page first mounted.
0185 let loadedForProfileId: string | null = null;
0186 $effect(() => {
0187     const currentId = $profile?.id ?? null;
0188     if (currentId && currentId !== loadedForProfileId) {
0189         loadedForProfileId = currentId;
0190         loading = true;
0191         allSubmissions = [];
0192         loadData().then(() => {
0193             loading = false;
0194         });
0195     }
0196 });
0197
0198 // â€œâ€œ Data Fetching â€œâ€œ
0199 async function loadData() {
0200     loadError = null;
0201     const userProfile = $profile;
0202     if (!userProfile) return;
0203
0204     const role = userProfile.role;
0205     console.log('[archive] Loading with role:', role,
'userId:', userProfile.id, 'schoolId:', userProfile.sc
0206
0207     // Offline: restore cached archive view
0208     if (typeof navigator !== 'undefined'
&& !navigator.onLine) {
0209         const cached = await
getCachedMetadata('archive_state_${role}_${userProfile.id}');
0210         if (cached?.data) {
0211             allSubmissions = cached.data.submissions || [];
0212             schoolsMap = cached.data.schoolsMap || {};
0213             teachersMap = cached.data.teachersMap || {};
0214             reviewsMap = cached.data.reviewsMap || {};
0215             console.log("[archive] Loaded archive from
offline cache");
0216             return;
0217         }
0218         allSubmissions = [];
0219         loadError = "You are offline and no cached archive
is available on this device yet.";
0220         return;
0221     }
0222
0223     try {
0224
0225         if (role === "Teacher") {

```

```

0226     const { data } = await supabase
0227       .from("submissions")
0228       .select(".*")
0229       .eq("user_id", userProfile.id)
0230       .eq("doc_type", "DLL")
0231       .order("created_at", { ascending: false });
0232     allSubmissions =
getRows<Submission>(data).filter((s) => s.user_id ===
userProfile.id);
0233   } else if (role === "Master Teacher") {
0234     if (!userProfile.school_id) return;
0235
0236     const { data } = await supabase
0237       .from("submissions")
0238       .select(
0239         `*, uploader:profiles!inner(full_name,
school_id, district_id, avatar_url, role)`,
0240       )
0241       .eq("profiles.school_id", userProfile.school_id)
0242       .in("doc_type", ["DLL", "ISP", "ISR"])
0243       .order("created_at", { ascending: false });
0244
0245     allSubmissions = getRows<Submission>(data)
0246       .filter(s => canViewUploadedISPISR(
0247         role,
0248         userProfile.id,
0249         s.user_id,
0250         s.uploader?.role || 'Teacher',
0251         s.doc_type,
0252         userProfile.school_id,
0253         s.uploader?.school_id || null,
0254         userProfile.district_id,
0255         s.uploader?.district_id || null
0256       ));
0257     console.log("[archive] Master Teacher loaded",
allSubmissions.length, 'submissions:', allSubmissions
0258     teachersMap = buildTeachersMap(allSubmissions);
0259   } else if (role === "School Head") {
0260     if (!userProfile.school_id) return;
0261
0262     const { data } = await supabase
0263       .from("submissions")
0264       .select(
0265         `*, uploader:profiles!inner(full_name,
school_id, district_id, avatar_url, role)`,
0266       )
0267       .eq("profiles.school_id", userProfile.school_id)
0268       .in("doc_type", ["DLL", "ISP", "ISR"])
0269       .order("created_at", { ascending: false });
0270
0271     allSubmissions = getRows<Submission>(data)
0272       .filter(s => canViewUploadedISPISR(
0273         role,
0274         userProfile.id,
0275         s.user_id,
0276         s.uploader?.role || 'Teacher',
0277         s.doc_type,
0278         userProfile.school_id,
0279         s.uploader?.school_id || null,
0280         userProfile.district_id,
0281         s.uploader?.district_id || null
0282       ));
0283     console.log("[archive] School Head loaded",
allSubmissions.length, 'submissions:', allSubmissions.ma
0284     teachersMap = buildTeachersMap(allSubmissions);
0285   } else if (role === "District Supervisor") {
0286     // District-scoped: fetch submissions from all
schools in district

```

```

0287     if (!userProfile.district_id) return;
0288
0289     // First get all schools in the district
0290     const { data: schoolsData } = await supabase
0291       .from("schools")
0292       .select("id, name, avatar_url")
0293       .eq("district_id", userProfile.district_id);
0294
0295     const schools = schoolsData || [];
0296     const sMap: Record<string, { label: string;
avatar_url: string | null }> = {};
0297     for (const s of schools) {
0298       sMap[s.id] = {
0299         label: s.name,
0300         avatar_url: s.avatar_url || null

```

## Storage Upload API

**src/routes/api/storage/upload/+server.ts Lines 1-140**

```

0001 import { json, error } from '@sveltejs/kit';
0002 import { Buffer } from 'buffer';
0003 import { S3Client, PutObjectCommand } from '@aws-
sdk/client-s3';
0004 import { env } from '$env/dynamic/private';
0005 import { supabase } from '$lib/utils/supabase';
0006
0007 const b2 = new S3Client({
0008   region: 'us-east-005',
0009   endpoint: env.B2_ENDPOINT || 'https://s3.us-east-
005.backblazeb2.com',
0010   forcePathStyle: false,
0011   requestChecksumCalculation: 'WHEN_REQUIRED',
0012   responseChecksumValidation: 'WHEN_REQUIRED',
0013   credentials: {
0014     accessKeyId: env.B2_APPLICATION_KEY_ID || "",
0015     secretAccessKey: env.B2_APPLICATION_KEY || "",
0016   },
0017 });
0018
0019 export async function POST({ request }) {
0020   // 1. Authenticate with Supabase
0021   const authHeader =
request.headers.get('Authorization');
0022   if (!authHeader) throw error(401, 'Unauthorized');
0023
0024   const token = authHeader.replace('Bearer ', "");
0025   const { data: { user }, error: authError } = await
supabase.auth.getUser(token);
0026
0027   if (authError || !user) {
0028     console.error("[upload-api] Auth error:", authError);
0029     throw error(401, 'Unauthorized');
0030   }
0031
0032   // 2. Parse FormData
0033   const formData = await request.formData();
0034   const file = formData.get('file');
0035   const key = formData.get('key');
0036
0037   if (!file || !(file instanceof Blob)) throw error(400, 'Missing
or invalid file');
0038   if (!key || typeof key !== 'string') throw error(400,
'Missing key');
0039
0040   // Check size limit (Vercel Serverless Function limit is
4.5MB, we use 4.4MB to be safe)
0041   const MAX_SIZE = 4.4 * 1024 * 1024;
0042   if (file.size > MAX_SIZE) {
0043     throw error(413, 'File exceeds the 4.2MB limit for

```

```

server-side uploads. Please compress your PDF.');
```

0044 }
0045
0046 try {
0047 const arrayBuffer = await file.arrayBuffer();
0048 const buffer = Buffer.from(arrayBuffer);
0049
0050 const command = new PutObjectCommand({
0051 Bucket: env.B2\_BUCKET\_NAME,
0052 Key: key,
0053 Body: buffer,
0054 ContentType: file.type || 'application/pdf',
0055 });
0056
0057 await b2.send(command);
0058 console.log(`[upload-api] Successfully proxied
upload for: \${key}`);
0059
0060 return json({ success: true, key });
0061 } catch (err: any) {
0062 console.error(`[upload-api] B2 Upload Error:`, err);
0063 throw error(500, `Failed to proxy upload to cloud
storage: \${err.message}`);
0064 }
0065 }

## Storage Presign API

**src/routes/api/storage/presign/+server.ts Lines 1-180**

```

0001 import { json, error } from '@sveltejs/kit';
0002 import { getPresignedUploadUrl,
getPresignedDownloadUrl } from '$lib/utls/b2.server';
0003 import { supabase } from '$lib/utls/supabase';
0004 import { env } from '$env/dynamic/private';
0005
0006 export async function POST({ request }) {
0007     // 1. Authenticate with Supabase
0008     const authHeader =
request.headers.get('Authorization');
0009     if (!authHeader) throw error(401, 'Unauthorized');
0010
0011     const token = authHeader.replace('Bearer ', '');
0012     const { data: { user }, error: authError } = await
supabase.auth.getUser(token);
0013
0014     if (authError || !user) {
0015         console.error(`[presign] Auth error:`, authError);
0016         throw error(401, 'Unauthorized');
0017     }
0018
0019     // 2. Get request body
0020     const { key, contentType, intent = 'upload' } = await
request.json();
0021     if (!key) throw error(400, 'Missing key');
0022
0023     // 3. Generate pre-signed URL
0024     try {
0025         let url;
0026         if (intent === 'download') {
0027             url = await getPresignedDownloadUrl(key);
0028         } else {
0029             if (!contentType) throw error(400, 'Missing
contentType for upload');
0030
0031             // Use SvelteKit's server environment source so
local Vite and
0032             // production adapters validate the same
credentials.
0033             const missing = ['B2_ENDPOINT',

```

```

'B2_BUCKET_NAME', 'B2_APPLICATION_KEY_ID',
'B2_APPLICATION_KEY']
0034             .filter((name) => !env[name as keyof typeof
env]);
0035             if (missing.length > 0) {
0036                 console.error(`[presign] Missing B2 environment
variables:`, missing.join(', '));
0037                 throw error(500, `Cloud storage is not
configured: missing ${missing.join(', ')});
0038             }
0039
0040             url = await getPresignedUploadUrl(key,
contentType);
0041         }
0042
0043         console.log(`[presign] Generated URL for ${key}:
${url.split('?')[0]}...`);
0044         return json({ url });
0045     } catch (err: any) {
0046         const detail = err?.body?.message || err?.message ||
err?.name || 'Unknown storage signing error';
0047         console.error(`[presign] Generation error:`, detail, err);
0048         if (err?.status && err?.body) throw err;
0049         throw error(500, `Failed to generate pre-signed URL:
${detail}`);
0050     }
0051 }

```

## Verification Page

**src/routes/verify/[hash]/+page.svelte Lines 1-260**

```

0001 <script lang="ts">
0002     import { page } from "$app/stores";
0003     import { goto } from "$app/navigation";
0004     import { supabase } from "$lib/utls/supabase";
0005     import StatusBadge from
"$lib/components/StatusBadge.svelte";
0006     import { getDocumentLabel } from
"$lib/utls/documentLabels";
0007     import QRScanner from
"$lib/components/QRScanner.svelte";
0008     import { onMount } from "svelte";
0009     import { lookupOfflineDoc, cacheVerifiedDoc } from
"$lib/utls/offline";
0010     import { profile } from "$lib/utls/auth";
0011     import { addToast } from "$lib/stores/toast";
0012     import {
0013         CheckCircle2,
0014         ShieldCheck,
0015         Clock,
0016         FileText,
0017         AlertCircle,
0018         WifiOff,
0019         Calendar,
0020         ScanLine,
0021         Download,
0022         Eye,
0023         Lock,
0024         LogIn,
0025         Loader2,
0026     } from "lucide-svelte";
0027
0028     interface VerifyResult {
0029         file_name: string;
0030         file_path: string;
0031         doc_type: string;
0032         compliance_status: string | null;
0033         created_at: string;
0034         file_size: number;

```

```

0035     week_number: number | null;
0036     subject: string | null;
0037     school_year: string | null;
0038     teacher_name: string | null;
0039     school_name: string | null;
0040     teaching_load_subject: string | null;
0041     teaching_load_grade: string | null;
0042     uploader_id: string | null;
0043     uploader_school_id: string | null;
0044   }
0045
0046   let result = $state<VerifyResult | null>(null);
0047   let notFound = $state(false);
0048   let loading = $state(true);
0049   let isOfflineData = $state(false);
0050   let showScanner = $state(false);
0051   let openingDoc = $state(false);
0052
0053   let canAccessFile = $derived.by(() => {
0054     const viewer = $profile;
0055     if (!viewer || !result) return false;
0056     if (viewer.role === 'District Supervisor') return true;
0057     if ((viewer.role === 'School Head' || viewer.role ===
'Master Teacher') && result.uploader_school_id) {
0058       return viewer.school_id ===
result.uploader_school_id;
0059     }
0060     if (viewer.role === 'Teacher' && result.uploader_id) {
0061       return viewer.id === result.uploader_id;
0062     }
0063     return false;
0064   });
0065
0066   let isLoggedIn = $derived(!!$profile);
0067
0068   const hash = $derived($page.params.hash);
0069
0070   onMount(async () => {
0071     // 1. Check offline cache first (Phase 20.3)
0072     const cached = await lookupOfflineDoc(hash as
string);
0073     if (cached) {
0074       result = cached;
0075       isOfflineData = true;
0076       loading = false;
0077     }
0078
0079     // 2. Try fetching fresh data from Supabase.
0080     //
0081     // This calls a SECURITY DEFINER RPC
0082     // of selecting from `submissions` directly. The table's
RLS used to
0083     // carry a blanket "USING (true)" SELECT policy
meant to support this
0084     // exact page, but a per-row policy can't be scoped to
"only when the
0085     // client filters by hash" — it just made the whole
table readable by
0086     // anyone regardless of what a query actually asked
for. The RPC
0087     // achieves the same public-verification goal without
that exposure:
0088     // it only ever returns the single row matching the
exact hash given.
0089     try {
0090       // Plain array request rather than .maybeSingle():
PostgreSQL only

```

```

0091     // 406s when a singular representation is
requested and 0 rows
0092     // come back — an expected outcome for an
unrecognized/mistyped
0093     // hash, not a real failure worth a console error on
this public
0094     // page.
0095     const { data: rows, error } = (await supabase.rpc(
0096       "verify_submission_by_hash",
0097       { p_hash: hash },
0098     )) as { data: VerifyResult[] | null; error: unknown };
0099     const data = rows?.[0] ?? null;
0100
0101     if (data) {
0102       const freshResult: VerifyResult = {
0103         file_name: data.file_name,
0104         file_path: data.file_path,
0105         doc_type: data.doc_type,
0106         compliance_status: data.compliance_status,
0107         created_at: data.created_at,
0108         file_size: data.file_size,
0109         week_number: data.week_number,
0110         subject: data.subject,
0111         school_year: data.school_year,
0112         teacher_name: data.teacher_name || null,
0113         school_name: data.school_name || null,
0114         teaching_load_subject:
data.teaching_load_subject || null,
0115         teaching_load_grade:
data.teaching_load_grade || null,
0116         uploader_id: data.uploader_id || null,
0117         uploader_school_id: data.uploader_school_id
|| null,
0118       };
0119
0120       result = freshResult;
0121       isOfflineData = false;
0122
0123       // Update cache for future offline use
0124       await cacheVerifiedDoc(hash as string,
freshResult);
0125     } else if (!result) {
0126       notFound = true;
0127     }
0128   } catch (err) {
0129     if (!result) notFound = true;
0130   } finally {
0131     loading = false;
0132   }
0133   });
0134
0135   function formatDate(dateStr: string): string {
0136     const date = new Date(dateStr);
0137     const dateFormatted = date.toLocaleDateString("en-
PH", {
0138       year: "numeric",
0139       month: "long",
0140       day: "numeric",
0141     });
0142     const timeFormatted = date.toLocaleTimeString("en-
PH", {
0143       hour: "2-digit",
0144       minute: "2-digit",
0145       hour12: true,
0146     });
0147     return `${dateFormatted} at ${timeFormatted}`;
0148   }
0149

```

```

0150 function handleScanResult(data: string) {
0151     showScanner = false;
0152     // Extract hash from verification URL or use raw data
0153     const hashMatch = data.match(/verifyV([a-f0-
0154     9]{64})/i);
0155     const scannedHash = hashMatch ? hashMatch[1] :
0156     data;
0157     if (scannedHash && /^[a-f0-
0158     9]{64}$/i.test(scannedHash)) {
0159         goto(`/verify/${scannedHash}`);
0160     }
0161     function openScanner() {
0162         showScanner = true;
0163     }
0164     async function handleOpenDocument() {
0165         if (!result?.file_path || !canAccessFile) return;
0166         openingDoc = true;
0167         try {
0168             const { data: { session } } = await
0169             supabase.auth.getSession();
0170             const token = session?.access_token;
0171             if (!token) {
0172                 addToast('error', 'Please sign in to view this
0173                 document.');
```

```

0172         return;
0173     }
0174     const res = await fetch('/api/storage/presign', {
0175         method: 'POST',
0176         headers: {
0177             'Content-Type': 'application/json',
0178             'Authorization': `Bearer ${token}`
0179         },
0180         body: JSON.stringify({ key: result.file_path,
0181         intent: 'download' })
0182     });
0183     if (!res.ok) throw new Error("Failed to get download
0184     URL");
0185     const { url } = await res.json();
0186     if (url) {
0187         window.open(url, '_blank');
0188     } else {
0189         throw new Error("No URL returned");
0190     }
0191     } catch (err) {
0192         console.error('[CEDIMS] Failed to open
0193         document:', err);
0194         addToast('error', 'Unable to open document.
0195         Please try again.');
```

```

0194     } finally {
0195         openingDoc = false;
0196     }
0197 }
0198 }
0199 </script>
0200
0201 <svelte:head>
0202     <title>Verify Document: CEDIMS · Powered by Smart
0203     E-VISION</title>
0204 </svelte:head>
0205 <!-- QR Scanner Overlay for continuous scanning -->
0206 {#if showScanner}
0207     <QRScanner
```

```

0208     onScan={handleScanResult}
0209     onClose={() => (showScanner = false)}
0210 />
0211 {/if}
0212
0213 <div class="min-h-dvh gradient-mesh flex items-center
0214 justify-center p-6">
0215     <div class="w-full max-w-lg animate-slide-up">
0216         <!-- Logo -->
0217         <div class="text-center mb-8">
0218             
0225             <h1 class="text-xl font-bold text-text-
0226             primary">CEDIMS</h1>
0227             <p class="text-sm text-text-muted">Document
0228             Verification</p>
0229             <div>
0230                 {#if loading}
0231                     <div
0232                         class="gov-card-static p-12 text-center"
0233                         role="status"
0234                         aria-label="Loading verification"
0235                     >
0236                         <div
0237                             class="w-12 h-12 border-4 border-gov-
0238                             blue/20 border-t-gov-blue rounded-full animate-spin mx-
0239                             ></div>
0240                             <p>
0241                                 Authenticating Hash...
0242                             </p>
0243                         {<div>
0244                             class="gov-card-static bg-gov-red/5 p-10 text-
0245                             center animate-shake"
0246                             role="alert"
0247                             <AlertCircle size={48} class="text-gov-red mx-
0248                             auto mb-4" />
0249                             <h2>
0250                                 class="text-xl font-semibold text-gov-red mb-
0251                                 2 uppercase tracking-tight"
0252                                 <p>
0253                                     Invalid Document
0254                                     </h2>
0255                                     <p class="text-sm text-text-secondary mb-6
0256                                     font-medium">
0257                                         The scanned hash does not match any
0258                                         official record in our
0259                                         secure registry.
0260                                     </p>
0261                                     <div
0262                                         class="p-3 bg-gov-red/5 rounded-lg border
0263                                         border-gov-red/10"
0264                                     >
0265                                         <code class="text-xs text-gov-red font-mono
0266                                         break-all"
```

## Database Security RPC

supabase/migrations/20260910\_fix\_verify\_by\_hash\_ri\_hol  
e.sql Lines 1-170

```
0001 --
=====
0002 -- SECURITY FIX: "Anyone can verify by hash" was
implemented as
0003 -- CREATE POLICY "Anyone can verify by hash" ON
submissions FOR SELECT USING (true);
0004 --
0005 -- RLS policies are evaluated per-row and combined with
OR across all
0006 -- permissive policies on the same command. A `USING
(true)` SELECT policy
0007 -- does not (and cannot) restrict itself to "only when the
query filters by
0008 -- file_hash" — Postgres has no way to see the client's
WHERE clause from
0009 -- inside a row policy. In practice this meant ANY
authenticated (and
0010 -- possibly anonymous) request could run
0011 -- supabase.from('submissions').select('*')
0012 -- with no filter at all and receive every row in the table —
including
0013 -- other teachers' file paths, raw OCR text, and
compliance data — because
0014 -- this one policy alone was enough to satisfy RLS
regardless of the other,
0015 -- correctly-scoped policies (Teachers own rows only,
School Head/Master
0016 -- Teacher same school, District Supervisor same district).
0017 --
0018 -- The public "scan a QR code to verify" feature only ever
needs a single
0019 -- row's worth of already-non-sensitive verification fields
for one known
0020 -- hash. That's exactly what a SECURITY DEFINER
function is for: it runs
0021 -- with the privileges of its owner (bypassing RLS
internally) but only
0022 -- exposes the narrow, specific result its own SQL asks for
— a client can
0023 -- never turn it into an unfiltered table dump.
0024 --
=====
0025
0026 DROP POLICY IF EXISTS "Anyone can verify by hash"
ON submissions;
0027
0028 CREATE OR REPLACE FUNCTION
verify_submission_by_hash(p_hash TEXT)
0029 RETURNS TABLE (
0030   file_name TEXT,
0031   file_path TEXT,
0032   doc_type TEXT,
0033   compliance_status TEXT,
0034   created_at TIMESTAMPTZ,
0035   file_size INTEGER,
0036   week_number INTEGER,
0037   subject TEXT,
0038   school_year TEXT,
0039   uploader_id UUID,
0040   teacher_name TEXT,
0041   uploader_school_id UUID,
0042   school_name TEXT,
0043   teaching_load_subject TEXT,
0044   teaching_load_grade TEXT
0045 )
0046 LANGUAGE sql
0047 SECURITY DEFINER
0048 SET search_path = public
0049 STABLE
0050 AS $$
0051   SELECT
0052     s.file_name,
0053     s.file_path,
0054     s.doc_type,
0055     s.compliance_status,
0056     s.created_at,
0057     s.file_size,
0058     s.week_number,
0059     s.subject,
0060     s.school_year,
0061     s.user_id AS uploader_id,
0062     p.full_name AS teacher_name,
0063     p.school_id AS uploader_school_id,
0064     sch.name AS school_name,
0065     tl.subject AS teaching_load_subject,
0066     tl.grade_level AS teaching_load_grade
0067   FROM submissions s
0068   LEFT JOIN profiles p ON p.id = s.user_id
0069   LEFT JOIN schools sch ON sch.id = p.school_id
0070   LEFT JOIN teaching_loads tl ON tl.id =
s.teaching_load_id
0071   WHERE s.file_hash = p_hash
0072   LIMIT 1;
0073 $$;
0074
0075 -- The verification page is intentionally public (per the
landing page copy:
0076 -- "Anyone can scan the code to verify document
authenticity"), so both
0077 -- anonymous visitors and signed-in users may call this
function. It cannot
0078 -- be used to enumerate or dump data — it only ever
returns the single row
0079 -- matching the exact hash passed in.
0080 GRANT EXECUTE ON FUNCTION
verify_submission_by_hash(TEXT) TO anon, authenticated;
0081
0082 --
=====
0083 -- The dropped policy was ALSO the only thing letting the
app's cross-teacher
0084 -- duplicate-content check work: "reject files that have
already been
0085 -- archived" (by anyone, not just the current uploader)
genuinely needs to
0086 -- look up a hash regardless of who owns the row. With
that policy gone, an
0087 -- authenticated Teacher's own query is limited to their
own rows (correct —
0088 -- that's what fixed the leak), so this legitimate check
needs the same
0089 -- narrow-RPC treatment: authenticated-only, and it
returns just enough to
0090 -- show "this was already archived", never the full row.
0091 --
=====
0092
0093 CREATE OR REPLACE FUNCTION
```

```

check_duplicate_submission_hash(p_hash TEXT)
0094 RETURNS TABLE (
0095     id UUID,
0096     file_name TEXT,
0097     doc_type TEXT,
0098     week_number INTEGER
0099 )
0100 LANGUAGE sql
0101 SECURITY DEFINER
0102 SET search_path = public
0103 STABLE
0104 AS $$
0105     SELECT s.id, s.file_name, s.doc_type, s.week_number
0106     FROM submissions s
0107     WHERE s.file_hash = p_hash
0108     LIMIT 1;
0109 $$;
0110
0111 GRANT EXECUTE ON FUNCTION
check_duplicate_submission_hash(TEXT) TO authenticated;

```

## Compliance Report RPC

**supabase/migrations/20260917\_compliance\_report\_rpc.sql**  
**Lines 1-220**

```

0001 --
=====
0002 -- Flat compliance-report rows, for the Google Apps Script
Sheets export.
0003 --
0004 -- Same security-scoping convention as
get_analytics_comparison() (see
0005 -- 20260821_analytics_aggregate_functions.sql):
SECURITY DEFINER, and the
0006 -- caller's own profile (looked up via auth.uid()) decides
what they can see
0007 -- — School Head / Master Teacher get their own school,
District Supervisor
0008 -- gets their own district, or every district when their
district_id is
0009 -- NULL. That last case is what lets a dedicated reporting
account (role =
0010 -- 'District Supervisor', district_id = NULL) see everything
without ever
0011 -- touching the service_role key — it's the same
fallthrough
0012 -- get_analytics_comparison() already relies on for a
district-wide view.
0013 --
0014 -- Apply with: supabase db push (or paste into the SQL
editor of the dashboard)
0015 --
=====
0016
0017 create or replace function
public.get_compliance_report_rows(p_school_year text default
null)
0018 returns table (
0019     teacher_name text,
0020     school_name text,
0021     district_name text,
0022     doc_type text,
0023     week_number integer,
0024     school_year text,
0025     compliance_status text,
0026     submitted_at timestampz
0027 )

```

```

0028 language plpgsql
0029 security definer
0030 set search_path = public
0031 as $$
0032 declare
0033     v_role text;
0034     v_school_id uuid;
0035     v_district_id uuid;
0036     v_sy text;
0037 begin
0038     select p.role, p.school_id, p.district_id
0039     into v_role, v_school_id, v_district_id
0040     from public.profiles p
0041     where p.id = auth.uid();
0042
0043     if v_role is null then
0044         raise exception 'Not authorized';
0045     end if;
0046
0047     v_sy := coalesce(p_school_year,
public.get_current_school_year());
0048
0049     return query
0050     select
0051         t.full_name as teacher_name,
0052         sch.name as school_name,
0053         d.name as district_name,
0054         s.doc_type,
0055         s.week_number,
0056         s.school_year,
0057         s.compliance_status,
0058         s.created_at as submitted_at
0059     from public.submissions s
0060     join public.profiles t on t.id = s.user_id
0061     left join public.schools sch on sch.id = t.school_id
0062     left join public.districts d on d.id = sch.district_id
0063     where s.school_year = v_sy
0064         and (
0065             (v_role in ('School Head', 'Master Teacher') and
t.school_id = v_school_id)
0066             or (v_role = 'District Supervisor' and (v_district_id
is null or sch.district_id = v_district_id))
0067         )
0068     order by d.name, sch.name, t.full_name,
s.week_number;
0069 end;
0070 $$;
0071
0072 revoke execute on function
public.get_compliance_report_rows(text) from public;
0073 grant execute on function
public.get_compliance_report_rows(text) to authenticated;

```

## Chatbot Utility

**src/lib/utils/chatbot.ts Lines 1-220**

```

0001 import intentModel from
'../models/intent_classifier_model.json';
0002 import type { SupabaseClient } from
'@supabase/supabase-js';
0003 import { getCurrentSchoolYear as
getDynamicSchoolYear } from './schoolYear';
0004
0005 // Text Normalization
0006 // Makes the bot robust to typos, wrong grammar,
repeated characters, emojis,
0007 // and diacritics. All light-weight, pure string ops — no
heavy model.
0008

```



```

0116 const lower = normalizeText(text);
0117 const tokens = lower.split(/\s+/).filter(Boolean);
0118 if (tokens.length === 0) return 'en';
0119 let tlCount = 0;
0120 for (const tok of tokens) {
0121     const clean = tok.replace(/[^a-z]/g, "");
0122     if (clean && TAGALOG_MARKERS.has(clean))
tlCount++;
0123 }
0124 if (tlCount >= 2) return 'tl';
0125 return tlCount / tokens.length >= 0.2 ? 'tl' : 'en';
0126 }
0127
0128 // Human-like Phrasing
0129 function pick<T>(arr: T[]): T {
0130     return arr[Math.floor(Math.random() * arr.length)];
0131 }
0132
0133 const OPENERS: Record<Lang, string[]> = {
0134     "en": [
0135         "Certainly."
0136     ],
0137     "tl": [
0138         "Narito po ang impormasyon."
0139     ]
0140 };
0141 const LOW_CONFIDENCE_PREFIXES: Record<Lang,
string[]> = {
0142     "en": [
0143         "Please confirm whether your question concerns"
0144     ],
0145     "tl": [
0146         "Pakikumpirma po kung ang inyong tanong ay
tungkol sa"
0147     ]
0148 };
0149 const LOW_CONFIDENCE_SUFFIXES: Record<Lang,
string[]> = {
0150     "en": [
0151         "If you meant a different topic, please clarify your
question."
0152     ],
0153     "tl": [
0154         "Kung ibang paksa po ang inyong tinutukoy,
pakilina ang inyong tanong."
0155     ]
0156 };
0157 const INTENT_TOPIC_LABELS: Record<Lang,
Record<Intent, string[]>> = {
0158     en: {
0159         ask_compliance: 'your compliance status',
0160         check_deadline: 'upcoming deadlines',
0161         find_dll: 'finding a Daily Lesson Plan',
0162         school_compare: 'school comparisons',
0163         teacher_stats: 'teacher statistics',
0164         calendar_info: 'the academic calendar',
0165         how_to_upload: 'how to upload a document',
0166         create_report: 'generating a report',
0167         general_help: 'general help'
0168     },
0169     tl: {
0170         ask_compliance: 'compliance status mo',
0171         check_deadline: 'mga paparating na deadline',
0172         find_dll: 'paghahanap ng Daily Lesson Plan',
0173         school_compare: 'paghahambing ng paaralan',
0174         teacher_stats: 'istatistika ng guro',
0175         calendar_info: 'academik calendar',
0176         how_to_upload: 'kung paano mag-upload ng

```

```

dokumento',
0177         create_report: 'paggawa ng report',
0178         general_help: 'pangalahatang tulong'
0179     }
0180 };
0181 const CONFUSED_RESPONSES: Record<Lang, string[]>
= {
0182     "en": [
0183         "I could not determine your request. Please rephrase
your question or ask about CEDIMS submissions, dead
0184     ],
0185     "tl": [
0186         "Paumanhin po, hindi ko matukoy ang inyong
kahilingan. Pakilina ang inyong tanong tungkol sa submission
0187     ]
0188 };
0189
0190 // Knowledge Base (lightweight FAQ corpus)
0191 // Every topic maps to *multiple* phrasings per language,
picked at random
0192 // each time, so asking the same question twice in one
session doesn't come
0193 // back sounding copy-pasted — and the language picked
matches how the user
0194 // actually asked, not a coin flip.
0195 interface KnowledgeEntry {
0196     keywords: string[];
0197     answers: Record<Lang, string[]>;
0198 }
0199
0200 const KNOWLEDGE_BASE: KnowledgeEntry[] = [
0201     {
0202         keywords: ['dll', 'daily lesson plan', 'weekly lesson log',
'lingguhang aralin', 'lesson plan', 'banghay
0203         answers: {
0204             en: [
0205                 'A Daily Lesson Plan is the DepEd weekly
lesson planning document that teachers prepare and subm
0206                 'Daily Lesson Plan: it's the weekly lesson plan
every teacher submits per subject. The system ch
0207                 'Think of a Daily Lesson Plan as your weekly
proof of teaching plans: one per subject, per week.
0208             ],
0209             tl: [
0210                 'Ang Daily Lesson Plan ay ang lingguhang
banghay-aralin na kailangan ninyong i-submit para sa ba
0211                 'Ang Daily Lesson Plan ay ang lesson plan
ninyo bawat linggo, per subject. Chinecheck ng system
0212                 'Daily Lesson Plan ang tawag sa lingguhang
lesson plan: isa ito bawat subject, bawat linggo, kai
0213             ]
0214         },
0215     },
0216     {
0217         keywords: ['isp', 'instructional supervisory plan'],
0218         answers: {
0219             en: [
0220                 'An ISP (Instructional Supervisory Plan) is the
school's supervisory blueprint: it lists program

```

## Intent Classifier Training

### chatbot/train\_intent\_classifier.py Lines 1-230

```

0001 """
0002 SmartE Vision Intent Classifier Training Pipeline
0003 Trains a character n-gram Logistic Regression intent
classifier.
0004 Robust to typos, Tagalog/Filipino, and varied phrasings.
0005 Generates: model JSON, confusion matrix, ROC curves,

```

classification report, top words chart.

0006 ""

0007

0008 import json, os, re, warnings, random

0009 from itertools import product

0010 import numpy as np

0011 import pandas as pd

0012 import matplotlib

0013 matplotlib.use('Agg')

0014 import matplotlib.pyplot as plt

0015 import seaborn as sns

0016 from sklearn.feature\_extraction.text import

CountVectorizer

0017 from sklearn.linear\_model import LogisticRegression

0018 from sklearn.model\_selection import train\_test\_split,

StratifiedKfold

0019 from sklearn.metrics import (

0020 confusion\_matrix, classification\_report, roc\_curve, auc,

0021 precision\_recall\_fscore\_support

0022 )

0023

0024 warnings.filterwarnings('ignore')

0025

0026 OUTPUT\_DIR = os.path.join(os.path.dirname(\_\_file\_\_), '..',

'src', 'lib', 'models')

0027 RESULTS\_DIR = os.path.join(os.path.dirname(\_\_file\_\_),

'results')

0028 os.makedirs(OUTPUT\_DIR, exist\_ok=True)

0029 os.makedirs(RESULTS\_DIR, exist\_ok=True)

0030

0031 # ~700+ labeled questions across 8 intents

0032 # Includes English, Tagalog/Filipino, and common

typographical errors

0033

0034 TRAINING\_DATA = [

0035 # ask\_compliance (~90 samples)

0036 {"text": "What is my compliance rate?", "intent":

"ask\_compliance"},

0037 {"text": "Am I compliant this week?", "intent":

"ask\_compliance"},

0038 {"text": "How many submissions are late?", "intent":

"ask\_compliance"},

0039 {"text": "What percentage of my DLLs are compliant?",

"intent": "ask\_compliance"},

0040 {"text": "Show my compliance status", "intent":

"ask\_compliance"},

0041 {"text": "Am I on track this term?", "intent":

"ask\_compliance"},

0042 {"text": "How compliant am I?", "intent":

"ask\_compliance"},

0043 {"text": "What is my compliance percentage?", "intent":

"ask\_compliance"},

0044 {"text": "Are all my submissions on time?", "intent":

"ask\_compliance"},

0045 {"text": "Check my submission status", "intent":

"ask\_compliance"},

0046 {"text": "How many DLLs did I submit on time?", "intent":

"ask\_compliance"},

0047 {"text": "Am I missing any submissions?", "intent":

"ask\_compliance"},

0048 {"text": "What is my compliance for Grade 5 Math?",

"intent": "ask\_compliance"},

0049 {"text": "Show me my late submissions", "intent":

"ask\_compliance"},

0050 {"text": "How many weeks am I compliant?", "intent":

"ask\_compliance"},

0051 {"text": "What is my overall compliance score?", "intent":

"ask\_compliance"},

0052 {"text": "My compliance status please", "intent":

"ask\_compliance"},

0053 {"text": "Have I submitted all required DLLs?", "intent":

"ask\_compliance"},

0054 {"text": "Which subjects am I compliant in?", "intent":

"ask\_compliance"},

0055 {"text": "Show my non-compliant weeks", "intent":

"ask\_compliance"},

0056 {"text": "How am I doing with my submissions?", "intent":

"ask\_compliance"},

0057 {"text": "What percent of my DLLs are in?", "intent":

"ask\_compliance"},

0058 {"text": "Give me my compliance summary", "intent":

"ask\_compliance"},

0059 {"text": "Show me where I am at with submissions",

"intent": "ask\_compliance"},

0060 {"text": "How many do I still need to submit?", "intent":

"ask\_compliance"},

0061 {"text": "Am I falling behind on DLLs?", "intent":

"ask\_compliance"},

0062 {"text": "What is my submission status?", "intent":

"ask\_compliance"},

0063 {"text": "My compliance rate for this quarter", "intent":

"ask\_compliance"},

0064 {"text": "How many am I missing this week?", "intent":

"ask\_compliance"},

0065 {"text": "Are there any pending submissions?", "intent":

"ask\_compliance"},

0066 {"text": "Show me my record so far", "intent":

"ask\_compliance"},

0067 {"text": "Do I have any late submissions?", "intent":

"ask\_compliance"},

0068 {"text": "What subjects am I behind on?", "intent":

"ask\_compliance"},

0069 {"text": "Any missing DLLs this week?", "intent":

"ask\_compliance"},

0070 {"text": "How is my compliance looking?", "intent":

"ask\_compliance"},

0071 {"text": "Do I need to submit anything?", "intent":

"ask\_compliance"},

0072 {"text": "Are all my DLLs submitted on time?", "intent":

"ask\_compliance"},

0073 {"text": "Am I good for this week?", "intent":

"ask\_compliance"},

0074 {"text": "Check if I am compliant", "intent":

"ask\_compliance"},

0075 {"text": "How am I performing with DLL submissions?",

"intent": "ask\_compliance"},

0076 {"text": "Give me my stats", "intent": "ask\_compliance"},

0077 {"text": "Show my record", "intent": "ask\_compliance"},

0078 {"text": "What is my score?", "intent": "ask\_compliance"},

0079 {"text": "Am I doing okay with submissions?", "intent":

"ask\_compliance"},

0080 {"text": "How many DLLs have I submitted?", "intent":

"ask\_compliance"},

0081 # Typo variants

0082 {"text": "What is my compliance rate?", "intent":

"ask\_compliance"},

0083 {"text": "Am I compliant this week?", "intent":

"ask\_compliance"},

0084 {"text": "how meny submissions are late", "intent":

"ask\_compliance"},

0085 {"text": "Show my compliance statis", "intent":

"ask\_compliance"},

0086 {"text": "Am I on trak this term?", "intent":

"ask\_compliance"},

0087 {"text": "How compliant am I?", "intent":

"ask\_compliance"},

0088 {"text": "Check my submission status", "intent": "ask\_compliance"},  
0089 {"text": "how meny DLLs did I submit on time", "intent": "ask\_compliance"},  
0090 {"text": "Am I missing any submissions?", "intent": "ask\_compliance"},  
0091 {"text": "What is my overall compliance score?", "intent": "ask\_compliance"},  
0092 {"text": "Have I submitted all required DLLs?", "intent": "ask\_compliance"},  
0093 {"text": "Show my non-compliant weeks", "intent": "ask\_compliance"},  
0094 {"text": "my compliance statis please", "intent": "ask\_compliance"},  
0095 {"text": "What is my compliance percentage?", "intent": "ask\_compliance"},  
0096 {"text": "Are all my submissions on time?", "intent": "ask\_compliance"},  
0097 {"text": "Which subjects am I compliant in?", "intent": "ask\_compliance"},  
0098 {"text": "Whats my compliance four this weak?", "intent": "ask\_compliance"},  
0099 {"text": "How am I doing with my submissions?", "intent": "ask\_compliance"},  
0100 {"text": "Give me my compliance summary", "intent": "ask\_compliance"},  
0101 {"text": "Am I falling behind on DLLs?", "intent": "ask\_compliance"},  
0102 # Tagalog variants  
0103 {"text": "Ano ang compliance rate ko?", "intent": "ask\_compliance"},  
0104 {"text": "Compliant ba ako ngayong linggo?", "intent": "ask\_compliance"},  
0105 {"text": "Ilan ang late submissions ko?", "intent": "ask\_compliance"},  
0106 {"text": "Ipakita ang compliance status ko", "intent": "ask\_compliance"},  
0107 {"text": "Nasa tamang landas ba ako?", "intent": "ask\_compliance"},  
0108 {"text": "Gaano ako ka-compliant?", "intent": "ask\_compliance"},  
0109 {"text": "Suriin ang submission status ko", "intent": "ask\_compliance"},  
0110 {"text": "Ilang DLL ang na-submit ko on time?", "intent": "ask\_compliance"},  
0111 {"text": "Mayroon ba akong missing submissions?", "intent": "ask\_compliance"},  
0112 {"text": "Ano ang overall compliance score ko?", "intent": "ask\_compliance"},  
0113 {"text": "Compliance status ko please", "intent": "ask\_compliance"},  
0114 {"text": "Na-submit ko na ba lahat ng DLL?", "intent": "ask\_compliance"},  
0115 {"text": "Aling subjects ang compliant ako?", "intent": "ask\_compliance"},  
0116 {"text": "Ipakita ang mga non-compliant weeks ko", "intent": "ask\_compliance"},  
0117 {"text": "Ano ang compliance percentage ko?", "intent": "ask\_compliance"},  
0118 {"text": "On time ba lahat ng submissions ko?", "intent": "ask\_compliance"},  
0119 {"text": "Kumusta ang compliance ko?", "intent": "ask\_compliance"},  
0120 {"text": "Ilan pa ba ang kailangan kong i-submit?", "intent": "ask\_compliance"},  
0121 {"text": "May late ba akong submission?", "intent": "ask\_compliance"},  
0122 {"text": "Saan ako kulang sa submissions?", "intent": "ask\_compliance"},  
0123 {"text": "Nasubmit ko na ba lahat?", "intent": "ask\_compliance"},  
0124 {"text": "Kumusta ang performance ko sa DLL compliance?", "intent": "ask\_compliance"},  
0125 {"text": "Compliance ko para sa linggong ito", "intent": "ask\_compliance"},  
0126 {"text": "May pending pa ba akong submissions?", "intent": "ask\_compliance"},  
0127 {"text": "Ipakita ang record ko ng submissions", "intent": "ask\_compliance"},  
0128 {"text": "Ilang percent ang compliance ko?", "intent": "ask\_compliance"},  
0129 {"text": "Compliant ba ako ngayon?", "intent": "ask\_compliance"},  
0130  
0131 # check\_deadline (~80 samples)  
0132 {"text": "When is the deadline for Week 5?", "intent": "check\_deadline"},  
0133 {"text": "What is the submission deadline for this week?", "intent": "check\_deadline"},  
0134 {"text": "When is the next deadline?", "intent": "check\_deadline"},  
0135 {"text": "Deadline for this week", "intent": "check\_deadline"},  
0136 {"text": "Is the deadline this Friday?", "intent": "check\_deadline"},  
0137 {"text": "How many days until the deadline?", "intent": "check\_deadline"},  
0138 {"text": "When is the deadline for Week 8?", "intent": "check\_deadline"},  
0139 {"text": "What time is the deadline?", "intent": "check\_deadline"},  
0140 {"text": "Deadline for this term", "intent": "check\_deadline"},  
0141 {"text": "What is the deadline this week?", "intent": "check\_deadline"},  
0142 {"text": "When is submission deadline?", "intent": "check\_deadline"},  
0143 {"text": "Upcoming deadline dates", "intent": "check\_deadline"},  
0144 {"text": "Is there a deadline today?", "intent": "check\_deadline"},  
0145 {"text": "Deadline for Week 10", "intent": "check\_deadline"},  
0146 {"text": "How much time left before deadline?", "intent": "check\_deadline"},  
0147 {"text": "Give me the deadline for this week", "intent": "check\_deadline"},  
0148 {"text": "When is the deadline due?", "intent": "check\_deadline"},  
0149 {"text": "Deadline schedule", "intent": "check\_deadline"},  
0150 {"text": "What deadline is coming up?", "intent": "check\_deadline"},  
0151 {"text": "Show me all deadlines", "intent": "check\_deadline"},  
0152 {"text": "Next submission deadline", "intent": "check\_deadline"},  
0153 {"text": "Is anything due this week?", "intent": "check\_deadline"},  
0154 {"text": "What is the cut-off date for submissions?", "intent": "check\_deadline"},  
0155 {"text": "When should I submit by?", "intent": "check\_deadline"},  
0156 {"text": "How long until the next deadline?", "intent": "check\_deadline"},  
0157 {"text": "What is due this week?", "intent": "check\_deadline"},

0158 {"text": "By when do I need to submit?", "intent": "check\_deadline"},  
0159 {"text": "What is the due date?", "intent": "check\_deadline"},  
0160 {"text": "Are there any deadlines this week?", "intent": "check\_deadline"},  
0161 {"text": "How many days left before submissions are due?", "intent": "check\_deadline"},  
0162 {"text": "Is anything expiring soon?", "intent": "check\_deadline"},  
0163 {"text": "What is the last day to submit DLLs?", "intent": "check\_deadline"},  
0164 {"text": "Tell me about this weeks deadline", "intent": "check\_deadline"},  
0165 {"text": "Submission cut-off", "intent": "check\_deadline"},  
0166 {"text": "When is the last day to submit?", "intent": "check\_deadline"},  
0167 {"text": "What is the submission date?", "intent": "check\_deadline"},  
0168 {"text": "Any deadlines coming up?", "intent": "check\_deadline"},  
0169 {"text": "What needs to be submitted this week?", "intent": "check\_deadline"},  
0170 {"text": "When is the cut-off?", "intent": "check\_deadline"},  
0171 {"text": "Do I have a deadline this week?", "intent": "check\_deadline"},  
0172 # Typo variants  
0173 {"text": "When is the dedline for Week 5?", "intent": "check\_deadline"},  
0174 {"text": "What is the submission dedline for this week?", "intent": "check\_deadline"},  
0175 {"text": "When is the next dedline?", "intent": "check\_deadline"},  
0176 {"text": "Is the dedline this Friday?", "intent": "check\_deadline"},  
0177 {"text": "how meny days until the dedline", "intent": "check\_deadline"},  
0178 {"text": "When is the dedline for Week 8?", "intent": "check\_deadline"},  
0179 {"text": "Upcomng dedline dates", "intent": "check\_deadline"},  
0180 {"text": "Is there a dedline today?", "intent": "check\_deadline"},  
0181 {"text": "how mutch time left before dedline", "intent": "check\_deadline"},  
0182 {"text": "What dedline is coming up?", "intent": "check\_deadline"},  
0183 {"text": "Next submission dedline", "intent": "check\_deadline"},  
0184 {"text": "Is anything due this weak?", "intent": "check\_deadline"},  
0185 {"text": "Whats the cut-off date four submissions?", "intent": "check\_deadline"},  
0186 {"text": "how long until the next dedline", "intent": "check\_deadline"},  
0187 {"text": "When is the last day to sumbit?", "intent": "check\_deadline"},  
0188 # Tagalog variants  
0189 {"text": "Kailan ang deadline para sa Week 5?", "intent": "check\_deadline"},  
0190 {"text": "Kailan ang submission deadline ngayong linggo?", "intent": "check\_deadline"},  
0191 {"text": "Kailan ang susunod na deadline?", "intent": "check\_deadline"},  
0192 {"text": "Deadline para sa linggong ito", "intent": "check\_deadline"},  
0193 {"text": "Ba't sa Friday ba ang deadline?", "intent": "check\_deadline"},  
0194 {"text": "Ilang araw bago ang deadline?", "intent": "check\_deadline"},  
0195 {"text": "Kailan ang deadline para sa Week 8?", "intent": "check\_deadline"},  
0196 {"text": "Kailan ang submission deadline?", "intent": "check\_deadline"},  
0197 {"text": "Mga deadline dates na darating", "intent": "check\_deadline"},  
0198 {"text": "May deadline ba ngayon?", "intent": "check\_deadline"},  
0199 {"text": "Deadline para sa Week 10", "intent": "check\_deadline"},  
0200 {"text": "Gaano karaming oras pa bago ang deadline?", "intent": "check\_deadline"},  
0201 {"text": "Ano ang deadline ngayong linggo?", "intent": "check\_deadline"},  
0202 {"text": "Kailan ang huling araw ng submission?", "intent": "check\_deadline"},  
0203 {"text": "May deadline ba ngayong linggo?", "intent": "check\_deadline"},  
0204 {"text": "Hanggang kailan ako puwedeng mag-submit?", "intent": "check\_deadline"},  
0205 {"text": "Anong araw ang deadline?", "intent": "check\_deadline"},  
0206 {"text": "Puwede pa ba akong mag-submit?", "intent": "check\_deadline"},  
0207 {"text": "Deadline ba ngayon?", "intent": "check\_deadline"},  
0208  
0209 # find\_dll (~80 samples)  
0210 {"text": "Find DLLs about fractions", "intent": "find\_dll"},  
0211 {"text": "Search for DLLs on reading comprehension", "intent": "find\_dll"},  
0212 {"text": "Show DLLs for Grade 3 Science", "intent": "find\_dll"},  
0213 {"text": "Find lesson plans about addition", "intent": "find\_dll"},  
0214 {"text": "Search DLLs for mathematics", "intent": "find\_dll"},  
0215 {"text": "Show DLLs for Week 4", "intent": "find\_dll"},  
0216 {"text": "Find DLLs uploaded by Teacher Santos", "intent": "find\_dll"},  
0217 {"text": "Search my DLLs", "intent": "find\_dll"},  
0218 {"text": "Show DLLs about plants", "intent": "find\_dll"},  
0219 {"text": "Find all DLLs for Grade 2", "intent": "find\_dll"},  
0220 {"text": "Look for DLLs about grammar", "intent": "find\_dll"},  
0221 {"text": "Find DLLs with group activities", "intent": "find\_dll"},  
0222 {"text": "Show DLLs for this week", "intent": "find\_dll"},  
0223 {"text": "Find DLLs about Filipino culture", "intent": "find\_dll"},  
0224 {"text": "Search documents for assessment tools", "intent": "find\_dll"},  
0225 {"text": "Look up DLLs on fractions", "intent": "find\_dll"},  
0226 {"text": "Show DLLs about animals", "intent": "find\_dll"},  
0227 {"text": "Find DLLs for Grade 1 Math", "intent": "find\_dll"},  
0228 {"text": "Search for DLLs on verbs", "intent": "find\_dll"},  
0229 {"text": "Show DLLs related to weather", "intent": "find\_dll"},  
0230 {"text": "Find DLLs with experiments", "intent": "find\_dll"},